UNIWERSYTET KARDYNAŁA STEFANA WYSZYŃSKIEGO
W WARSZAWIE

WYDZIAŁ MATEMATYCZNO-PRZYRODNICZY
SZKOŁA NAUK ŚCISŁYCH

Łukasz Łukaszewski

Nr alb.: 119244

Kierunek studiów: informatyka

Opracowanie systemu widzenia komputerowego do automatycznej detekcji i zliczania przesyłek pocztowych na taśmie transportowej

Praca magisterska wykonana
pod kierunkiem naukowym
dr. Katsiaryna Kosarava

Warszawa, 2026

Streszczenie

Praca dotyczy opracowania inteligentnego systemu detekcji, śledzenia i zliczania przesyłek pocztowych przemieszczających się na taśmie transportowej z wykorzystaniem metod komputerowego widzenia i uczenia maszynowego, ze szczególnym uwzględnieniem dwóch klas obiektów: małych przesyłek typu mailer oraz większych paczek typu box.

Celem pracy było zaprojektowanie, implementacja i ewaluacja rozwiązania typu *proof-of-concept* (dowód słuszności koncepcji), zdolnego do działania w warunkach zbliżonych do czasu rzeczywistego oraz umożliwiającego porównanie jakości detekcji, skuteczności zliczania i wydajności obliczeniowej różnych konfiguracji systemu.

W ramach badań przeanalizowano trzy modele detekcji obiektów: YOLOv11, RF-DETR oraz Faster R-CNN, a także algorytmy śledzenia wieloobiektowego ByteTrack, BoT-SORT i SORT. System zbudowano jako modularny potok przetwarzania wideo (ang. *pipeline*) obejmujący detekcję, normalizację wyników, filtrację z wykorzystaniem ROI, śledzenie obiektów, logikę zliczania oraz eksport metryk i materiałów wynikowych, a całość uzupełniono aplikacją webową w technologii Streamlit wspierającą prowadzenie eksperymentów.

Badania oparto na własnym zbiorze danych zawierającym 395 obrazów i 1403 adnotacje typu *bounding box* dla klas box i mailer, dla którego przeprowadzono analizę statystyczną oraz zastosowano preprocessing i augmentację danych. Eksperymenty objęły porównanie modeli detekcyjnych, analizę wpływu rozmiaru modelu, ocenę różnic pomiędzy inferencją na CPU i GPU oraz weryfikację wpływu *tracker*ów na końcową dokładność zliczania.

Uzyskane wyniki wykazały, że RF-DETR osiągnął najwyższą jakość detekcji offline, natomiast YOLOv11 zapewnił najkorzystniejszy kompromis pomiędzy dokładnością a szybkością działania. Najlepszą dokładność zliczania uzyskała konfiguracja łącząca RF-DETR z algorytmem ByteTrack, a przeprowadzone badania potwierdziły, że metody głębokiego uczenia mogą stanowić praktyczną podstawę budowy systemów automatycznego zliczania paczek, choć wyniki należy interpretować z uwzględnieniem ograniczeń zbioru danych, generalizacji oraz *proof-of-concept*owego charakteru rozwiązania.

Abstract

This thesis presents the development of an intelligent system for the detection, tracking, and counting of parcels moving on a conveyor belt using computer vision and machine learning methods, with a particular focus on two object classes: small mailer parcels and larger box packages. The main objective of the study was to design, implement, and evaluate a proof-of-concept solution capable of operating in near real time while enabling a systematic comparison of detection quality, counting accuracy, and computational performance across different system configurations.

The research included a comparative analysis of three object detection models — YOLOv11, RF-DETR, and Faster R-CNN — as well as multiple object tracking algorithms: ByteTrack, BoT-SORT, and SORT. The proposed solution was implemented as a modular video-processing pipeline covering detection, output normalization, ROI-based

filtering, object tracking, counting logic, and export of metrics and result videos, and it was complemented by a Streamlit-based web application supporting experiment management and result inspection.

The experiments were conducted on a custom dataset containing 395 images and 1403 bounding box annotations for the box and mailer classes, preceded by statistical dataset analysis as well as preprocessing and augmentation steps. The evaluation covered detector comparison, the impact of model size, CPU versus GPU inference performance, and the influence of tracking algorithms on final counting accuracy.

The results showed that RF-DETR achieved the highest offline detection quality, while YOLOv11 provided the most favorable trade-off between accuracy and inference speed. The best counting performance was obtained with the RF-DETR and ByteTrack combination, and the study confirmed that deep learning methods can provide a practical foundation for automatic parcel counting systems, although the findings should be interpreted with regard to dataset limitations, generalization constraints, and the proof-of-concept nature of the developed solution.

Słowa kluczowe

detekcja obiektów, śledzenie wieloobiektowe, zliczanie obiektów, komputerowe widzenie, uczenie maszynowe, głębokie uczenie, taśma transportowa, przesyłki pocztowe, YOLOv11, RF-DETR, Faster R-CNN, ByteTrack

Keywords

object detection, multi-object tracking, object counting, computer vision, machine learning, deep learning, conveyor belt, postal parcels, YOLOv11, RF-DETR, Faster R-CNN, ByteTrack

Spis treści

Spis rysunków VI

Spis tabel VIII

Spis skrótów IX

Rozdział 1. Wstęp 1

1.1. Motywacja i kontekst problemu 1
1.2. Cel i hipoteza badawcza pracy 2
1.3. Zakres pracy i przyjęte ograniczenia 3
1.4. Struktura pracy 4

Rozdział 2. Analiza problemu i przegląd literatury 6

2.1. Przegląd algorytmów detekcji obiektów 6
2.1.1. YOLO 7
2.1.2. RF-DETR 7
2.1.3. Faster R-CNN 8
2.2. Metody śledzenia i zliczania obiektów na taśmach transportowych 9
2.3. Przegląd algorytmów śledzenia obiektów 11
2.3.1. ByteTrack 12
2.3.2. BoT-SORT 13
2.3.3. SORT 14
2.4. Porównanie podejść — wady, zalety, ograniczenia 15
2.5. Uzasadnienie wyboru metod zastosowanych w pracy 17

Rozdział 3. Dane i ich przygotowanie 21

3.1. Charakterystyka problemu: małe i duże paczki pocztowe 21
3.2. Źródła i charakterystyka zbioru danych 23
3.3. Adnotacje danych — narzędzia i proces etykietowania 26
3.4. Wstępna analiza statystyczna zbioru 28
3.5. Augmentacja i preprocessing danych 31

Rozdział 4. Implementacja systemu 35

4.1. Architektura systemu — schemat ogólny *pipeline*'u 35
4.2. Modele detekcji — konfiguracja i trening 38
4.2.1. YOLOv11 39
4.2.2. RF-DETR 40
4.2.3. Faster R-CNN 43
4.3. Algorytmy śledzenia obiektów i zliczania 44
4.3.1. ByteTrack i BoT-SORT dla YOLOv11 45

4.3.2. ByteTrack i SORT dla RF-DETR i Faster R-CNN . . . 46
4.4. Środowisko programistyczne . . . 47
4.4.1. Architektura kodu źródłowego i zapewnienie jakości . . . 48
4.4.2. Frameworki uczenia maszynowego i widzenia komputerowego . . . 48
4.4.3. Śledzenie, ewaluacja i przetwarzanie danych . . . 49
4.4.4. Infrastruktura eksperymentalna i interfejs webowy . . . 50
4.5. Sprzęt i środowisko obliczeniowe . . . 50
4.6. Aplikacja webowa — architektura i funkcjonalności . . . 52
4.6.1. Architektura oprogramowania i separacja zagadnień . . . 52
4.6.2. Moduły aplikacji i cykl życia eksperymentu . . . 53
4.6.3. Stos technologiczny i optymalizacje . . . 57

Rozdział 5. Eksperymenty i wyniki . . . 59
5.1. Plan eksperymentów i zastosowane metryki . . . 59
5.1.1. Środowisko eksperymentalne i kontrolowane warunki . . . 59
5.1.2. Macierz eksperymentalna i mapowanie pytań badawczych . . . 60
5.1.3. Zastosowane metryki . . . 61
5.2. Eksperyment I — porównanie modeli detekcji . . . 62
5.2.1. Konfiguracja eksperymentu . . . 62
5.2.2. Metryki offline: jakość detekcji . . . 62
5.2.3. Metryki online: wydajność inferencji na GPU . . . 64
5.2.4. Dyskusja i odpowiedź na PB1 . . . 66
5.3. Eksperyment II — porównanie dużych i małych wariantów modeli . . . 67
5.3.1. Konfiguracja eksperymentu . . . 67
5.3.2. Metryki offline: jakość detekcji na zbiorze walidacyjnym . . . 68
5.3.3. Metryki online: wydajność inferencji i gęstość detekcji . . . 69
5.3.4. Dyskusja i odpowiedź na PB2 . . . 72
5.4. Eksperyment III — porównanie inferencji na CPU vs. GPU . . . 72
5.5. Eksperyment IV — porównanie *trackerów* w kontekście dokładności zliczania . . . 76
5.5.1. Dyskusja wyników i odpowiedź na PB4 . . . 78
5.6. Dyskusja wyników — interpretacja, porównanie z literaturą, analiza błędów . . . 79

Rozdział 6. Wnioski . . . 82
6.1. Podsumowanie osiągniętych celów . . . 83
6.2. Odpowiedź na hipotezę badawczą . . . 83
6.3. Ograniczenia systemu i pracy . . . 84
6.4. Kierunki dalszego rozwoju . . . 87

Literatura . . . 89

Spis rysunków

3.1 Przykładowe obrazy wchodzące w skład zbioru danych: pojedyncze fotografie oraz klatki wyodrębnione z publicznie dostępnych nagrań przedstawiających pracę sortowni i linii transportowych. . . . 24
3.2 Przykłady zróżnicowania wizualnego zbioru danych: odmienne warunki oświetleniowe, geometria sceny. . . . 25
3.3 Przykłady scen trudnych w zbiorze danych: częściowe przesłonięcia, niewielkie odstępy między obiektami. . . . 26
3.4 Interfejs platformy Roboflow wykorzystywany w procesie ręcznej adnotacji obrazów z użyciem ramek bounding box dla klas mailer i box. . . . 27
3.5 Przykładowa klatka ze zbioru treningowego z naniesionymi adnotacjami ground truth — widoczne *bounding boxy* klas *box* oraz *mailer* wraz z ich etykietami. . . . 27
3.6 Rozkład adnotacji pomiędzy klasami box i mailer. . . . 29
3.7 Rozkład liczby obiektów przypadających na pojedynczy obraz. . . . 30
3.8 Przykład wpływu preprocessingu i augmentacji na pojedynczy obraz ze zbioru danych: (a) obraz oryginalny, (b) obraz po preprocessingu obejmującym standaryzację orientacji oraz przeskalowanie do rozdzielczości 640 × 640 pikseli z zachowaniem proporcji i czarnym wypełnieniem, (c) obraz po augmentacji w postaci niewielkiej rotacji, (d) obraz po połączonych transformacjach odbicia poziomego i rotacji. . . . 33

4.1 Schemat blokowy pipeline przetwarzania wideo systemu detekcji i zliczania przesyłek pocztowych. . . . 37
4.2 Widok zakładki *Training* w aplikacji webowej — panel uruchamiania notatników treningowych dla modeli YOLOv11, RF-DETR oraz Faster R-CNN. . . . 39
4.3 Widok zakładki *Model Comparison* w aplikacji webowej — wybór konfiguracji model–tracker oraz porównanie adnotowanych wideo dla wybranego nagrania. . . . 54
4.4 Porównanie adnotowanych klatek w zakładce *Model Comparison* — konfiguracje RF-DETR z ByteTrack oraz Faster R-CNN z ByteTrack dla tego samego nagrania. 54
4.5 Sekcja *Performance Metrics* w zakładce *Model Comparison* — tabela porównawcza oraz wykresy słupkowe FPS, czasu inferencji, gęstości detekcji i średniego zaufania. 55
4.6 Widok zakładki *Inference Runner* w aplikacji webowej — konfiguracja zadań wnioskowania oraz podsumowanie ich wykonania. . . . 56
4.7 Widok zakładki *ROI Editor* w aplikacji webowej — interaktywne definiowanie wielokątnego obszaru zainteresowania na wybranej klatce nagrania. . . . 57

5.1 Porównanie mAP@0.5 dla klas `box` i `mailer` w ocenianych modelach detekcji. . . . 64
5.2 Trade-off pomiędzy szybkością inferencji a jakością detekcji dla trzech modeli. . . . 66

5.3 Porównanie mAP@0.5 dla klas `box` i `mailer` dla wariantów nano i small obu architektur. 69
5.4 Porównanie średniego FPS inferencji dla wariantów nano i small w podziale na trackery. 70
5.5 Trade-off pomiędzy szybkością inferencji a jakością detekcji dla wariantów nano i small. 71
5.6 Porównanie średniego FPS inferencji na GPU i CPU dla wszystkich konfiguracji model–*tracker*. 74

Spis tabel

3.1 Podstawowe statystyki zbioru danych oraz podział na zbiory treningowy, walidacyjny i testowy. 29
3.2 Kategorie rozmiaru i proporcji obrazów w analizowanym zbiorze danych. 30

5.1 Porównanie metryk jakości detekcji offline na zbiorze walidacyjnym dla ocenianych modeli. 63
5.2 Średnia wydajność inferencji online na GPU dla ocenianych modeli. 64
5.3 Szczegółowa wydajność inferencji dla poszczególnych nagrań (FPS / czas inferencji w ms). 65
5.4 Porównanie metryk jakości detekcji offline na zbiorze walidacyjnym dla wariantów YOLOv11 i RF-DETR. 68
5.5 Metryki offline per klasa dla wariantów RF-DETR (zbiór walidacyjny). 69
5.6 Średnia wydajność inferencji online na GPU dla wariantów modeli (*tracker*: *ByteTrack*). 70
5.7 Średnia wydajność inferencji dla analizowanych konfiguracji model–*tracker* na GPU i CPU. 73
5.8 Liczba obiektów klasy `box` w podziale na nagrania i konfiguracje. 76
5.9 Liczba obiektów klasy `mailer` w podziale na nagrania i konfiguracje. 77
5.10 Zestawienie błędów zliczania dla analizowanych konfiguracji. 78

Spis skrótów

AI Artificial Intelligence, sztuczna inteligencja

BoT-SORT algorytm śledzenia obiektów analizowany w pracy i używany z modelem YOLOv11

ByteTrack algorytm śledzenia wieloobiektowego używany w porównaniach konfiguracji detektor-tracker

CNN Convolutional Neural Network, konwolucyjna sieć neuronowa

CPU Central Processing Unit, procesor wykorzystywany jako jedna z platform inferencji w eksperymencie porównującym CPU i GPU

EXIF Exchangeable Image File Format, standard metadanych osadzanych w plikach graficznych, zawierający m.in. informację o orientacji obrazu

Faster R-CNN dwuetapowy model detekcji obiektów porównywany z YOLOv11 i RF-DETR

FP16 tryb obliczeń o zmniejszonej precyzji używany podczas treningu modeli

FPS Frames Per Second, klatki na sekundę; podstawowa miara wydajności inferencji stosowana w pracy

GPU Graphics Processing Unit, akcelerator graficzny używany do treningu i inferencji modeli

GT Ground Truth, prawda podstawowa, czyli rzeczywista liczba fizycznych przesyłek w nagraniu

IDF1 miara oceny śledzenia wielu obiektów wspomniana jako metryka nieuwzględniona w planie ewaluacji

IoU Intersection over Union, miara stosowana w ewaluacji detekcji i asocjacji obiektów

MAE miara błędu użyta przy ocenie dokładności zliczania

mAP mean Average Precision, średnia precyzja uśredniona

MAPE procentowa miara błędu użyta przy ocenie dokładności zliczania

MOT Multiple Object Tracking, śledzenie wielu obiektów

MOTA Multiple Object Tracking Accuracy, miara oceny śledzenia wielu obiektów wspomniana w ograniczeniach metodologicznych

NMS Non-Maximum Suppression, heurystyka postprocessingu, której RF-DETR nie wymaga

OCR Optical Character Recognition, optyczne rozpoznawanie znaków

OOM Out-Of-Memory, błąd braku pamięci

PB pytanie badawcze

PR krzywe PR używane do oceny balansu między fałszywymi alarmami a pominiętymi detekcjami

RF-DETR transformatorowa architektura detekcji obiektów porównywana w pracy

ROI Region of Interest, obszar zainteresowania ograniczający analizę do taśmy transportowej

SDK skrót występujący w kontekście Roboflow Python SDK używanego do zarządzania zbiorem danych

SORT Simple Online And Realtime Tracking, algorytm śledzenia obiektów

SSD Single-Shot Detector, jednoetapowy detektor obiektów

tracker algorytm śledzenia wieloobiektowego (multi-object tracker); w pracy termin używany jako skrótowe określenie algorytmów ByteTrack, BoT-SORT i SORT

VRAM pamięć karty graficznej istotna dla konfiguracji treningu i inferencji

YOLO You Only Look Once, jednoetapowa rodzina modeli detekcji obiektów

Rozdział 1

Wstęp

1.1. Motywacja i kontekst problemu

Gwałtowny wzrost e-commerce w ostatnich kilkunastu latach spowodował, że operatorzy pocztowi i logistyczni muszą obsługiwać stale rosnący wolumen przesyłek przy jednoczesnym zachowaniu wysokiej wydajności i niezawodności procesów. Wolumen przesyłek obsłużonych przez firmy kurierskie w Polsce wzrósł z poziomu 1,214 mld w 2024 roku do blisko 1,34 mld w roku 2025 [23]. Na tym tle wyróżnia się wynik InPostu, który wypracował 25-procentowy wzrost liczby dostarczonych paczek względem poprzedniego roku [13].

Współczesne sortownie stoją przed wyzwaniem szybkiego przetwarzania dużych strumieni przesyłek przy jednoczesnym zachowaniu bezbłędności i ciągłości pracy. Odpowiedzią na te wymagania są zaawansowane systemy przenośników, które dzięki integracji z wizyjnymi systemami kamer, czujnikami oraz algorytmami AI, stanowią dziś fundament automatyzacji centrów logistycznych [26].

Tradycyjne metody liczenia i monitorowania paczek w dużym stopniu opierają się na pracy ręcznej lub prostych rozwiązaniach czujnikowych, takich jak bariery fotoelektryczne czy systemy wagowe. Chociaż oferują one podstawową funkcjonalność, borykają się z poważnymi problemami w zakresie skalowalności oraz odporności na pracę w zatłoczonym i dynamicznym środowisku linii transportowych [12].

Kluczową barierą technologiczną pozostaje skuteczna i niezawodna detekcja obiektów w czasie rzeczywistym – szczególnie w sytuacjach, gdy przesyłki mają skrajnie różne gabaryty, nachodzą na siebie lub znajdują się na trudnym wizualnie tle. Rozróżnianie małych form (np. kopert bąbelkowych) od paczek ponadgabarytowych pozostaje nietrywialnym wyzwaniem dla systemów opartych na sztywnych regułach. Takie ograniczenia sprawiają, że klasyczne algorytmy przetwarzania obrazu nie radzą sobie w wymagającym otoczeniu przemysłowym, co wymusza przejście na modele wizyjne wykorzystujące uczenie maszynowe.

Potrzebę tę dostrzega branża: w odpowiedzi na te wyzwania aż 60% europejskich decydentów w sektorze magazynowym planuje wdrożyć oprogramowanie oparte na sztucznej inteligencji w ciągu najbliższych pięciu lat [47].

W bardziej zaawansowanych systemach stosuje się metody śledzenia, które pozwalają zachować tożsamość obiektów w kolejnych klatkach wideo i umożliwiają ich zliczanie w dynamicznych scenach. Z praktycznego punktu widzenia stanowi to silną motywację do

opracowywania inteligentnych rozwiązań w zakresie detekcji i liczenia przesyłek w systemach logistycznych.

Niniejsza praca koncentruje się zatem na projektowaniu i ewaluacji systemu wizji komputerowej oraz uczenia maszynowego, przeznaczonego do wykrywania i zliczania małych oraz dużych paczek pocztowych na przenośniku taśmowym.

1.2. Cel i hipoteza badawcza pracy

Głównym celem niniejszej pracy jest zaprojektowanie, implementacja oraz ewaluacja inteligentnego systemu wizyjnego, zdolnego do automatycznej detekcji i zliczania przesyłek pocztowych – zarówno małych, jak i dużych – przemieszczających się po przenośniku taśmowym w czasie rzeczywistym. System ten ma pełnić rolę rozwiązania demonstracyjnego typu *proof-of-concept* dla zastosowań w zautomatyzowanych sortowniach i centrach logistycznych, w których kluczowe znaczenie mają skuteczność detekcji, szybkość przetwarzania oraz możliwość działania w różnych środowiskach obliczeniowych [16].

W celu realizacji tego założenia w pracy przeprowadzono analizę porównawczą trzech modeli detekcji obiektów: YOLOv11 [14], RF-DETR [33] oraz Faster R-CNN [30]. Każdy z nich oceniany jest zarówno pod względem jakości detekcji – przy użyciu metryk takich jak mAP i precyzja – jak i szybkości inferencji wyrażonej w klatkach na sekundę (FPS), co pozwala określić jego przydatność w zadaniu analizy przesyłek poruszających się na taśmie transportowej w warunkach zbliżonych do czasu rzeczywistego.

Część detekcyjna została rozszerzona o algorytmy śledzenia wieloobiektowego (*trackery*, ang. *tracker*), które umożliwiają zachowanie tożsamości przesyłek w kolejnych klatkach obrazu, a tym samym ich poprawne zliczanie. Dla modelu YOLOv11 analizowane są algorytmy ByteTrack oraz BoT-SORT, natomiast dla modeli RF-DETR i Faster R-CNN uwzględniono ByteTrack oraz SORT [48].

Takie podejście umożliwia ocenę nie tylko skuteczności samych detektorów, lecz również wpływu warstwy śledzenia na jakość końcowego zliczania obiektów w dynamicznej scenie wideo [48].

Praca została oparta na następującej hipotezie badawczej:

Możliwe jest opracowanie systemu detekcji i śledzenia obiektów, opartego na metodach głębokiego uczenia, który osiąga wystarczająco wysoką skuteczność detekcji oraz szybkość działania, aby mógł stanowić praktyczne rozwiązanie do automatycznego zliczania przesyłek na taśmie transportowej, przy jednoczesnym występowaniu istotnych różnic pomiędzy architekturami modeli detekcyjnych, wariantami ich rozmiaru, algorytmami śledzenia oraz platformami sprzętowymi wykorzystywanymi do inferencji [14, 33, 48].

Z hipotezy głównej wynikają następujące szczegółowe pytania badawcze:

PB1: Który z analizowanych modeli detekcji — YOLOv11, RF-DETR czy Faster R-CNN — zapewnia najlepszy kompromis pomiędzy skutecznością wykrywania obiektów a szybkością inferencji w zadaniu detekcji paczek?

PB2: W jaki sposób rozmiar modelu wpływa na jakość detekcji i czy większe warianty modeli zapewniają istotną poprawę wyników względem mniejszych odpowiedników?

PB3: Jak duża jest różnica wydajności pomiędzy inferencją realizowaną na CPU i GPU oraz czy wdrożenie systemu wyłącznie na CPU może być uznane za praktycznie użyteczne w kontekście systemów działających w czasie rzeczywistym?

PB4: Który z algorytmów śledzenia — ByteTrack, BoT-SORT czy SORT — zapewnia najwyższą poprawność zliczania paczek we współpracy z poszczególnymi modelami detekcji?

Podstawę weryfikacji hipotezy oraz odpowiedzi na postawione pytania badawcze stanowić będą eksperymenty obejmujące porównanie modeli detekcyjnych, wariantów ich rozmiaru, algorytmów śledzenia oraz środowisk wykonawczych. Uzyskane wyniki pozwolą określić, które zestawienie metod zapewnia najkorzystniejszą równowagę pomiędzy dokładnością detekcji, poprawnością zliczania oraz wydajnością obliczeniową w zadaniu analizy przesyłek poruszających się po przenośniku taśmowym.

1.3. Zakres pracy i przyjęte ograniczenia

Zakres niniejszej pracy obejmuje pełny proces tworzenia inteligentnego systemu detekcji i zliczania paczek – od przygotowania zbioru danych, poprzez trenowanie modelu, aż po ocenę skuteczności detekcji i śledzenia obiektów na taśmociągu. System został zaprojektowany i przetestowany ze szczególnym uwzględnieniem rozróżniania dwóch głównych kategorii przesyłek: małych paczek pocztowych (np. kopert bąbelkowych, foliopaków) oraz dużych paczek (pudełek), co odzwierciedla najczęstsze wyzwania klasyfikacyjne w rzeczywistych sortowniach.

Wymiar praktyczny tej pracy opiera się na kilku kluczowych elementach. Po pierwsze, detekcja wspomnianych klas przesyłek bazuje na odpowiednio wyadnotowanym zbiorze danych obrazowych i wideo, przygotowanym do celów treningowych i testowych. Po drugie, badanie obejmuje ocenę porównawczą trzech modeli wykrywania obiektów: YOLOv11, RF-DETR oraz Faster R-CNN. Po trzecie, warstwa detekcyjna została rozszerzona o algorytmy śledzenia wielu obiektów. Umożliwia to zliczanie paczek z zachowaniem przypisanej im tożsamości na kolejnych klatkach wideo, co w pełni odpowiada roli mechanizmów śledzących w analizie dynamicznych scen wizualnych.

Z kolei zakres eksperymentalny obejmuje porównanie jakości detekcji, precyzji zliczania oraz szybkości wnioskowania w różnych konfiguracjach systemu. W badaniach przeanalizowano w szczególności różnice wynikające z zastosowania odmiennych architektur detektorów, wariantów modeli (mniejszych i większych), konfiguracji algorytmów śledzących, a także środowisk sprzętowych opartych na procesorach (CPU) oraz kartach graficznych (GPU). Takie podejście jest podyktowane koniecznością znalezienia optymalnego kompromisu pomiędzy dokładnością a wydajnością obliczeniową, co stanowi dobrze znane wyzwanie w systemach detekcji obiektów działających w czasie rzeczywistym [16].

Pod kątem implementacji, projekt przyjmuje formę dowodu słuszności koncepcji i nie aspiruje do odtworzenia pełnej złożoności przemysłowej linii sortowniczej. Opracowana aplikacja ma za zadanie zweryfikować techniczną wykonalność zaproponowanego rozwiązania, a nie dostarczyć gotowy, komercyjny produkt wdrożeniowy [16, 26]. Z tego względu w pracy skupiono się wyłącznie na głównym procesie wizyjnej detekcji, śledzenia i zlicza-

nia paczek. Celowo pominięto integrację z zewnętrznymi systemami operacyjnymi, takimi jak czytniki kodów kreskowych, oprogramowanie OCR, systemy zarządzania magazynem, ramiona robotyczne czy mechanizmy zautomatyzowanego kierowania ruchem przesyłek.

Wyniki uzyskane w niniejszej pracy należy interpretować w kontekście przyjętego scenariusza badawczego. Eksperymenty zostały przeprowadzone na publicznie dostępnych materiałach wideo oraz zbiorach danych pozyskanych z internetu, co oznacza, że ich bezpośrednie przenoszenie na inne sortownie, inne typy przenośników lub odmienne warunki przemysłowe wymaga dodatkowej weryfikacji eksperymentalnej. Ograniczeniem pracy jest również to, że analizie poddano wyłącznie wybrane architektury detekcji i śledzenia, reprezentatywne dla badanego problemu, lecz nieobejmujące pełnego spektrum dostępnych metod. Z tego względu przedstawione rezultaty należy traktować jako wiążące przede wszystkim dla badanego układu eksperymentalnego oraz jako podstawę do dalszych badań nad uogólnieniem zaproponowanego rozwiązania. Kod źródłowy opracowanej aplikacji wraz z instrukcją uruchomienia został udostępniony w publicznym repozytorium GitHub pod adresem: https://github.com/lukmiik/masters_thesis_parcels_object_detection_web_app.

1.4. Struktura pracy

Niniejsza praca została podzielona na sześć rozdziałów, z których każdy podejmuje odrębny aspekt projektowanego systemu.

Rozdział 1. (Wstęp) zarysowuje motywację oraz kontekst przemysłowy podjętej tematyki. Ponadto, definiuje główny cel badawczy, stawia hipotezę wraz ze szczegółowymi pytaniami badawczymi, określa zakres i ograniczenia projektu, a na końcu prezentuje niniejsze zestawienie struktury dokumentu.

Rozdział 2. (Analiza problemu i przegląd literatury) stanowi systematyczne omówienie dotychczasowych metod związanych z badanym problemem. Przedstawiono w nim ewolucję architektur detekcji obiektów – od tradycyjnych technik przetwarzania obrazu po nowoczesne modele głębokiego uczenia, ze szczególnym uwzględnieniem algorytmów YOLOv11, RF-DETR oraz Faster R-CNN. Rozdział ten przybliża również teoretyczne podstawy śledzenia wieloobiektowego oraz charakteryzuje wykorzystane w pracy algorytmy: ByteTrack, BoT-SORT i SORT. Całość podsumowano merytorycznym uzasadnieniem wyborów metodologicznych dokonanych na potrzeby badań.

Rozdział 3. (Dane i ich przygotowanie) opisuje zestaw danych wykorzystany do trenowania i ewaluacji modeli. Znalazły się w nim informacje o źródłach i charakterystyce materiałów wizyjnych (zdjęć i wideo), procesie adnotacji, a także analiza statystyczna rozkładu klas. Omówiono ponadto zastosowane techniki augmentacji i preprocessingu obrazu oraz sposób podziału zebranych danych na zbiory: treningowy, walidacyjny i testowy.

Rozdział 4. (Implementacja systemu) skupia się na technicznych detalach zaprojektowanego rozwiązania. Zaprezentowano w nim ogólną architekturę potoku przetwarzania danych, parametry procesu uczenia dla każdego z trzech detektorów, a także sposób integracji algorytmów śledzących z poszczególnymi modelami. Dopełnieniem tej części jest

opis wykorzystanego środowiska sprzętowo-programowego oraz demonstracyjnej aplikacji internetowej, pełniącej rolę interfejsu dla użytkownika końcowego.

Rozdział 5. (Eksperymenty i wyniki) przedstawia rezultaty czterech głównych analiz porównawczych, stanowiących bezpośrednią odpowiedź na postawione wcześniej pytania badawcze: zestawienie architektur modeli detekcyjnych (PB1), badanie wpływu rozmiaru wariantów modeli na jakość wyników (PB2), porównanie wydajności inferencji pomiędzy procesorami CPU i GPU (PB3) oraz weryfikację dokładności algorytmów śledzących w zadaniu zliczania paczek (PB4). Każdy z przeprowadzonych eksperymentów zilustrowano wynikami ilościowymi, wizualizacjami oraz szczegółową interpretacją.

Rozdział 6. (Wnioski) to syntetyczne zebranie wyników całej pracy. Zawiera on ostateczne odpowiedzi na pytania badawcze, weryfikację głównej hipotezy, a także dyskusję nad ograniczeniami zaproponowanego systemu i potencjalnymi kierunkami jego dalszego rozwoju.

Rozdział 2

Analiza problemu i przegląd literatury

2.1. Przegląd algorytmów detekcji obiektów

Detekcja obiektów należy do podstawowych i zarazem najintensywniej rozwijanych zadań widzenia komputerowego. W odróżnieniu od klasyfikacji obrazu, która przypisuje całemu obrazowi pojedynczą etykietę, detekcja obiektów wymaga jednoczesnej odpowiedzi na dwa pytania: jakie obiekty występują na obrazie oraz gdzie dokładnie się znajdują. Wynikiem działania detektora jest zatem zbiór ramek ograniczających (ang. *bounding boxes*), którym przypisywane są klasy obiektów oraz wartości ufności predykcji [49]. W praktycznych zastosowaniach jakość detektora ocenia się nie tylko przez skuteczność predykcji, lecz również przez szybkość inferencji, koszt obliczeniowy oraz łatwość wdrożenia w rzeczywistym systemie [14].

Z punktu widzenia budowy praktycznych systemów wizyjnych detekcja obiektów pełni rolę warstwy pośredniej między surowym obrazem a dalszym przetwarzaniem decyzyjnym. To właśnie od jakości detektora zależy skuteczność kolejnych etapów, takich jak śledzenie obiektów, ich zliczanie, analiza obrazu czy automatyczne podejmowanie decyzji. Jeżeli obiekty są wykrywane niestabilnie, z błędną lokalizacją albo nie są wykrywane wcale, błędy te propagują się dalej do następnych modułów systemu [18]. Problem ten nabiera szczególnego znaczenia w systemach przemysłowych i logistycznych — detektor musi pracować stabilnie i szybko, a jednocześnie poprawnie radzić sobie ze zmienną skalą obiektów, częściowymi przesłonięciami, nakładaniem się paczek oraz niejednorodnymi warunkami oświetlenia.

Rozwój algorytmów detekcji obiektów przebiegał od metod opartych na ręcznie projektowanych deskryptorach cech, takich jak HOG[1] czy SIFT[2], do architektur głębokiego uczenia, które samodzielnie uczą się reprezentacji obrazu bezpośrednio z danych treningowych [46]. Przełom nastąpił wraz z popularyzacją konwolucyjnych sieci neuronowych, które umożliwiły automatyczne uczenie hierarchicznych reprezentacji cech bez konieczności ręcznego definiowania reguł opisujących obraz [49].

[1] HOG (Histograms of Oriented Gradients, histogramy zorientowanych gradientów) — „Siatki deskryptorów histogramów zorientowanych gradientów znacząco przewyższają istniejące zbiory cech w detekcji ludzi"[8].

[2] SIFT (Scale-Invariant Feature Transform, skalo-niezmiennicza transformacja cech) — „W artykule przedstawiono metodę ekstrakcji charakterystycznych, niezmienniczych cech z obrazów, które mogą być wykorzystywane do niezawodnego dopasowywania różnych widoków obiektu lub sceny"[19].

Współczesne rozwiązania można podzielić na trzy główne nurty: detektory dwuetapowe, detektory jednoetapowe oraz nowsze architektury transformatorowe wykorzystujące mechanizmy uwagi. Ocena ich jakości opiera się na ustandaryzowanych miarach, takich jak Intersection over Union (IoU), precyzja, pełność oraz mean Average Precision (mAP), przy czym w zastosowaniach praktycznych równie istotne są szybkość inferencji i przepustowość systemu [49].

2.1.1. YOLO

Rodzina YOLO (You Only Look Once) stanowi najbardziej rozpoznawalny przykład jednoetapowych detektorów obiektów. Jej podstawowa idea polega na tym, że cały problem detekcji jest traktowany jako pojedyncze zadanie regresyjne realizowane w jednym przebiegu sieci, bez wyraźnego oddzielania etapu generowania propozycji regionów od końcowej klasyfikacji i lokalizacji. Takie podejście znacząco upraszcza *pipeline* przetwarzania i pozwala osiągać bardzo wysoką szybkość inferencji, co sprawiło, że modele z tej rodziny stały się szczególnie popularne w zastosowaniach bliskich czasu rzeczywistego [14].

Kolejne wersje YOLO rozwijały pierwotną koncepcję przez wprowadzanie coraz bardziej zaawansowanych mechanizmów ekstrakcji cech, agregacji informacji wieloskalowej oraz optymalizacji obliczeniowej. Generacja YOLOv11 kontynuuje ten kierunek rozwoju, zachowując charakterystyczny dla rodziny podział na *backbone* (główna część ekstrakcji cech), *neck* (moduł agregacji cech) i *head* (głowica predykcyjna), a jednocześnie wprowadzając szereg zmian architektonicznych mających poprawić skuteczność i efektywność modelu [14].

Wśród najczęściej wskazywanych elementów wyróżniających YOLOv11 znajdują się między innymi bloki C3k2[3], moduł SPPF[4] oraz komponent C2PSA[5], które mają usprawniać ekstrakcję cech, agregację kontekstu i selektywne wzmacnianie istotnych obszarów obrazu [3].

Istotną zaletą YOLOv11 jest także skalowalność. Model występuje w kilku wariantach rozmiarowych, od lekkich wersji przeznaczonych do środowisk o ograniczonych zasobach obliczeniowych po większe konfiguracje nastawione na maksymalizację jakości predykcji. Skalowalność rodziny YOLOv11 umożliwia dobór wariantu modelu adekwatnego do dostępnych zasobów sprzętowych, od lekkich wersji działających na urządzeniach brzegowych po większe konfiguracje przeznaczone do serwerów GPU [42].

2.1.2. RF-DETR

Odmienną filozofię reprezentuje RF-DETR, należący do nowszej grupy detektorów transformatorowych. Podejście to wywodzi się z architektury DETR, w której detekcja obiektów została sformułowana jako problem bezpośredniej predykcji zbioru obiektów, a

[3] „Blok C3k2 jest bardziej efektywną obliczeniowo implementacją wąskiego gardła Cross Stage Partial (CSP). Wykorzystuje dwie mniejsze konwolucje zamiast jednej dużej konwolucji" [14].

[4] „Warstwa SPPF wykonuje kilka równoległych operacji max-poolingu o różnych rozmiarach jąder na tej samej mapie cech, przechwytując kontekst na wielu skalach" [14].

[5] „Blok C2PSA stanowi istotne rozszerzenie, które wzmacnia uwagę przestrzenną w mapach cech. Mechanizm ten pozwala modelowi skuteczniej koncentrować się na ważnych regionach obrazu" [14].

nie jako sekwencja heurystycznych kroków obejmujących kotwice, przypisywanie propozycji i końcowe filtrowanie wyników [6]. W praktyce oznacza to bardziej spójną architekturę *end-to-end*, w której model bezpośrednio przewiduje końcowy zestaw detekcji, co ogranicza zależność od klasycznych elementów postprocessingu, takich jak *anchor boxes*[6] czy non-maximum suppression[7].

RF-DETR bazuje na fundamencie Vision Transformera i jest przedstawiany jako architektura czasu rzeczywistego wykorzystująca *backbone* DINOv2, czyli model uczony w sposób samonadzorowany na dużych zbiorach danych wizualnych [27]. Tego typu wstępne uczenie ma duże znaczenie dla transferowalności modelu, ponieważ poprawia zdolność uogólniania na nowe domeny, także wtedy, gdy zbiór danych do dostrajania jest ograniczony. W literaturze RF-DETR łączony jest również z lekkimi rozwiązaniami dekodera inspirowanymi pracą LW-DETR, których celem jest obniżenie kosztu obliczeniowego i przyspieszenie inferencji bez rezygnacji z zalet mechanizmu uwagi.

Szczególnie ważną cechą RF-DETR jest jego charakter *end-to-end*. W odróżnieniu od wielu starszych detektorów model ten dąży do bezpośredniej predykcji końcowego zbioru obiektów, dzięki czemu *pipeline* jest bardziej jednolity koncepcyjnie i mniej zależny od ręcznie projektowanych heurystyk. Według materiałów Roboflow model RF-DETR osiąga bardzo wysokie wyniki w benchmarkach, a jednocześnie jest pozycjonowany jako rozwiązanie nadające się do zastosowań praktycznych, nie tylko do porównań laboratoryjnych [33].

2.1.3. Faster R-CNN

Klasycznym przedstawicielem detektorów dwuetapowych jest Faster R-CNN, będący jednym z najważniejszych punktów odniesienia w historii badań nad detekcją obiektów. Jego najistotniejszą innowacją było wprowadzenie Region Proposal Network (RPN), czyli modułu generującego propozycje regionów potencjalnie zawierających obiekty bezpośrednio na współdzielonych mapach cech sieci konwolucyjnej [30]. Dzięki temu rozwiązaniu udało się ograniczyć koszt obliczeniowy obecny we wcześniejszych podejściach region-based, gdzie generowanie propozycji było realizowane oddzielnie od właściwego detektora.

Zasada działania Faster R-CNN opiera się na dwuetapowym przetwarzaniu. W pierwszym etapie sieć wskazuje obszary obrazu, które z dużym prawdopodobieństwem zawierają obiekty, natomiast w drugim etapie te propozycje są klasyfikowane, a ich ramki ograniczające są dodatkowo doprecyzowywane. Taka organizacja obliczeń zwykle sprzyja wysokiej jakości lokalizacji i dobrej precyzji detekcji, zwłaszcza w trudniejszych scenach z obiektami o zróżnicowanej skali lub częściowo nakładającymi się na siebie. Ceną za tę dokładność

[6] *Anchor boxes* (ramki kotwicowe) — „W każdej pozycji okna przesuwnego jednocześnie przewidywane jest k propozycji regionów [...]. Owe k propozycji jest parametryzowanych względem k ramek referencyjnych, które nazywamy kotwicami. Kotwica jest wycentrowana w rozpatrywanym oknie przesuwnym i powiązana z określoną skalą oraz proporcją boków" [30].

[7] Non-maximum suppression (ang. non-maximum suppression, NMS) — technika postprocessingu stosowana w detekcji obiektów w celu wyeliminowania nadmiarowych, nakładających się na siebie ramek ograniczających.

jest jednak większa złożoność obliczeniowa i wyższe opóźnienie inferencji niż w przypadku wielu modeli jednoetapowych oraz najnowszych lekkich architektur transformatorowych.

Mimo pojawienia się nowszych rozwiązań Faster R-CNN nadal pozostaje bardzo ważnym punktem odniesienia metodologicznego. Reprezentuje bowiem wcześniejszy, ale nadal niezwykle wpływowy paradygmat, zgodnie z którym detekcja korzysta z wyraźnego rozdzielenia etapu wyszukiwania kandydatów od etapu ich dokładnego rozpoznawania. W niniejszej pracy jego uwzględnienie jest uzasadnione nie tylko historycznie, ale również praktycznie, ponieważ umożliwia porównanie współczesnych modeli czasu rzeczywistego z silnym i dobrze ugruntowanym detektorem dwuetapowym.

Zestawienie YOLOv11, RF-DETR i Faster R-CNN tworzy tym samym układ porównawczy obejmujący trzy odmienne filozofie projektowania detektorów, co pozwala wyjaśnić nie tylko, który model osiąga najlepszy wynik liczbowy, ale również, z jakich założeń architektonicznych wynikają różnice w skuteczności, szybkości działania i kosztach obliczeniowych.

2.2. Metody śledzenia i zliczania obiektów na taśmach transportowych

Problem zliczania obiektów poruszających się na taśmie transportowej nie sprowadza się do samej detekcji obiektów w pojedynczych klatkach obrazu. Detektor pozwala określić, czy i gdzie w danej klatce znajduje się paczka, natomiast poprawne zliczanie w strumieniu wideo wymaga dodatkowo zachowania tożsamości obiektu w czasie. W przeciwnym razie ta sama przesyłka, widoczna przez wiele kolejnych klatek, mogłaby zostać policzona wielokrotnie. Z tego względu w nowoczesnych systemach wizyjnych zliczanie jest traktowane jako wynik połączenia detekcji, śledzenia oraz reguły wyzwalającej zdarzenie zliczenia [39].

W starszych rozwiązaniach przemysłowych zadanie liczenia obiektów na przenośnikach realizowano często przy użyciu barier fotoelektrycznych, czujników zbliżeniowych lub prostych układów pomiarowych. Metody te są stosunkowo łatwe do wdrożenia i mogą działać poprawnie w kontrolowanych warunkach, jednak ich możliwości są ograniczone. Nie pozwalają one bowiem na jednoczesne rozróżnianie klas obiektów, mają problemy w sytuacji stykania się lub częściowego nakładania paczek oraz wymagają dedykowanej infrastruktury sprzętowej dla każdego punktu pomiarowego [12]. Z tego względu w nowoczesnych sortowniach coraz większą rolę odgrywają systemy oparte na analizie obrazu.

Współcześnie dominującym podejściem jest paradygmat *tracking-by-detection* (śledzenie przez detekcję). W takim ujęciu detektor najpierw generuje dla każdej klatki zbiór ramek ograniczających wraz z etykietami klas i wartościami ufności, a następnie algorytm śledzenia próbuje powiązać nowe detekcje z obiektami obserwowanymi w poprzednich klatkach. W efekcie każdej fizycznej przesyłce przypisywany jest identyfikator ścieżki, który powinien być utrzymany możliwie długo, mimo chwilowych zakłóceń, spadków pewności detekcji czy częściowych przesłonięć [28].

Choć środowisko taśmy transportowej jest bardziej uporządkowane niż na przykład ulice, zadanie śledzenia i zliczania nadal pozostaje trudne. W praktyce występują tu takie problemy jak rozmycie ruchu spowodowane prędkością taśmy, nierównomierne oświetlenie, częściowe przesłonięcia, duża zmienność rozmiaru paczek oraz znaczne podobieństwo wizualne wielu przesyłek. Szczególnie problematyczne są sytuacje, w których małe koperty bąbelkowe lub cienkie przesyłki foliowe są częściowo zasłonięte przez większe paczki albo pojawiają się bardzo blisko innych obiektów.

Z punktu widzenia metod śledzenia kluczowym etapem jest skojarzenie nowych detekcji z istniejącymi ścieżkami. W prostszych rozwiązaniach wykorzystuje się model ruchu, najczęściej oparty na filtrze Kalmana[8], oraz miary geometryczne, takie jak Intersection over Union. Podejście to jest wydajne obliczeniowo i dobrze sprawdza się wtedy, gdy ruch obiektów jest względnie regularny, a liczba przesłonięć niewielka. Jego wadą jest jednak podatność na błędy identyfikacji w sytuacjach, gdy obiekty chwilowo znikają, nakładają się na siebie albo są wykrywane niestabilnie [48].

Aby ograniczyć liczbę błędnych przełączeń identyfikatorów, rozwijano także metody wzbogacające śledzenie o cechy wyglądu obiektu. Przykładem takiego podejścia jest BoT-SORT, który łączy informację o ruchu z dodatkowymi wskazówkami ułatwiającymi poprawne dopasowanie detekcji do ścieżek. Rozwiązania tego typu mogą być użyteczne w bardziej zatłoczonych scenach, ale zwykle wiążą się z większym kosztem obliczeniowym i większą złożonością całego *pipeline*'u [1].

W ostatnich latach szczególne znaczenie zyskały metody, które poprawiają ciągłość śledzenia nie tyle przez rozbudowane modelowanie wyglądu, ile przez skuteczniejsze wykorzystanie samych wyników detekcji. Dobrym przykładem jest ByteTrack, który nie odrzuca zbyt wcześnie detekcji o niższej pewności, lecz próbuje wykorzystać je do podtrzymania ścieżek obiektów chwilowo przesłoniętych lub słabiej widocznych. W zastosowaniach logistycznych jest to bardzo korzystne, ponieważ małe paczki lub koperty często generują mniej stabilne detekcje niż większe obiekty, a mimo to powinny zachować ten sam identyfikator w kolejnych klatkach.

Skuteczność śledzenia jest przy tym silnie zależna od jakości warstwy detekcyjnej. Jeżeli detektor generuje dokładne i częste aktualizacje ramek ograniczających, *tracker* ma większą szansę na utrzymanie spójności śledzenia. Jeżeli natomiast detekcje są niestabilne, opóźnione lub znacząco zmieniają swoje położenie i rozmiar między kolejnymi klatkami, rośnie ryzyko fragmentacji ścieżek oraz błędów zliczania. W praktyce oznacza to, że dobór metody śledzenia powinien być rozpatrywany łącznie z doborem modelu detekcji oraz docelowej platformy sprzętowej.

W praktycznych zastosowaniach przemysłowych analiza obrazu jest często ograniczana do wybranego obszaru, istotnego z punktu widzenia realizowanego zadania. Takie podejście pozwala skoncentrować przetwarzanie na fragmencie obrazu, w którym rzeczywiście obserwowany jest przepływ obiektów, a jednocześnie ogranicza wpływ elementów tła i

[8] Filtr Kalmana — rekurencyjny algorytm estymacji stanu układu dynamicznego, opracowany przez Rudolfa E. Kálmána w 1960 roku. Łączy predykcję stanu na podstawie modelu ruchu z korekcją opartą na pomiarach, minimalizując błąd średniokwadratowy estymacji.

obszarów niemających znaczenia dla procesu analizy. W rezultacie możliwe jest bardziej uporządkowane i stabilne prowadzenie obserwacji obiektów w strumieniu wideo [20].

Istotną rolę w takim układzie pełni śledzenie wieloobiektowe, którego zadaniem jest przypisywanie kolejnym detekcjom tych samych identyfikatorów w następnych klatkach obrazu. Dzięki temu obiekt obserwowany przez pewien czas może być traktowany jako ten sam obiekt, a nie jako zbiór niezależnych wykryć. Ma to bezpośrednie znaczenie dla poprawności zliczania, ponieważ pozwala powiązać końcowy wynik nie z pojedynczym pojawieniem się obiektu w obrazie, lecz z jego spójną obecnością w analizowanym obszarze.

W konsekwencji zliczanie obiektów na taśmie transportowej należy traktować jako wynik współdziałania trzech powiązanych elementów: detekcji, śledzenia oraz reguły zliczania. Detekcja odpowiada za lokalizację obiektów w pojedynczych klatkach, śledzenie za utrzymanie ich tożsamości w czasie, natomiast mechanizm zliczający przekształca informacje o obserwowanych obiektach w końcowy wynik liczbowy.

2.3. Przegląd algorytmów śledzenia obiektów

Śledzenie wielu obiektów (ang. Multi-Object Tracking, MOT) należy do podstawowych problemów widzenia komputerowego, ponieważ jego celem jest nie tylko lokalizacja obiektów w kolejnych klatkach nagrania, lecz także utrzymanie ich spójnej tożsamości w czasie [4]. W odróżnieniu od śledzenia pojedynczego obiektu, w zadaniu MOT liczba obserwowanych obiektów może zmieniać się dynamicznie, co istotnie zwiększa złożoność problemu. Z punktu widzenia systemów analizy taśm transportowych ma to szczególne znaczenie, ponieważ poprawne zliczanie paczek wymaga traktowania każdej przesyłki jako jednego fizycznego obiektu obserwowanego w wielu kolejnych klatkach, a nie jako serii niezależnych detekcji.

We współczesnych systemach MOT dominuje paradygmat *tracking-by-detection*. W podejściu tym detektor obiektów generuje dla każdej klatki ramki ograniczające, etykiety klas oraz wartości ufności, natomiast osobny moduł śledzenia odpowiada za powiązanie bieżących detekcji z trajektoriami utworzonymi w poprzednich klatkach. Kluczowym zagadnieniem jest tu asocjacja danych — dopasowanie nowych detekcji do istniejących ścieżek — realizowana najczęściej przy użyciu filtru Kalmana, miary IoU oraz algorytmu węgierskiego [5]. Takie rozdzielenie warstwy detekcji i asocjacji jest praktyczne z inżynierskiego punktu widzenia, ponieważ umożliwia łączenie różnych detektorów z tym samym *tracker*em oraz niezależną ocenę wpływu obu komponentów na końcową jakość systemu. W środowisku logistycznym dodatkowym wyzwaniem jest to, że paczki mogą znacząco różnić się rozmiarem, poruszać się blisko siebie i okresowo zasłaniać się wzajemnie, co sprawia, że wybór algorytmu śledzenia ma bezpośrednie przełożenie na dokładność zliczania.

Spośród algorytmów śledzenia szczególną rolę w zastosowaniach czasu rzeczywistego odgrywają metody online, do których należą m.in. ByteTrack, BoT-SORT i SORT.

2.3.1. ByteTrack

ByteTrack jest algorytmem wieloobiektowego śledzenia należącym do rodziny *tracking-by-detection*, zaproponowanym jako proste, a zarazem bardzo skuteczne rozwinięcie klasycznych metod asocjacji. Jego główna idea polega na tym, że podczas śledzenia nie należy bezwarunkowo odrzucać wszystkich detekcji o niższej ufności, ponieważ część z nich nadal może odpowiadać rzeczywistym obiektom chwilowo gorzej widocznym w obrazie, na przykład z powodu rozmycia ruchu, częściowej okluzji, zmiany skali lub niekorzystnych warunków rejestracji [48]. W praktyce oznacza to, że ByteTrack poprawia ciągłość identyfikacji obiektów nie przez komplikowanie samego detektora, lecz przez bardziej efektywne wykorzystanie wygenerowanych przez niego wyników.

Mechanizm działania ByteTrack opiera się na dwuetapowej asocjacji detekcji. W pierwszym etapie istniejące ścieżki są dopasowywane do detekcji o wysokiej ufności, które stanowią najbardziej wiarygodne hipotezy obecności obiektu w bieżącej klatce. Następnie ścieżki, które nie zostały sparowane w pierwszym kroku, są porównywane z detekcjami o niższej ufności, o ile nadal mieszczą się one w dopuszczalnym zakresie progowym. Dzięki temu ramki ograniczające o słabszym wyniku detekcyjnym nie są automatycznie traktowane jako bezużyteczny szum, lecz stają się dodatkowym źródłem informacji pozwalającym podtrzymać trajektorię rzeczywistego obiektu. Jednocześnie nowe ścieżki inicjalizowane są wyłącznie na podstawie nieprzypisanych detekcji o wysokiej ufności, co ogranicza ryzyko tworzenia fałszywych identyfikatorów z tła lub artefaktów detektora.

Pod względem modelu ruchu ByteTrack zachowuje lekkość znaną z klasycznych *trackerów* online. Wykorzystuje filtr Kalmana do przewidywania położenia obiektu w kolejnej klatce oraz dopasowanie oparte na miarze IoU, co pozwala utrzymać wysoką wydajność obliczeniową i pracę bliską czasowi rzeczywistemu. Ścieżki, które nie uzyskają dopasowania, nie są od razu usuwane, lecz przechodzą do stanu utraconego i przez pewien czas mogą zostać odzyskane, jeśli odpowiadająca im detekcja pojawi się ponownie w kolejnych klatkach. Takie rozwiązanie jest szczególnie użyteczne w scenach, w których obiekty znikają tylko na krótko, na przykład wskutek chwilowego przesłonięcia przez sąsiednią paczkę.

Ruch paczek ma zwykle względnie uporządkowany charakter, co sprzyja metodom bazującym na predykcji ruchu, ale jednocześnie małe lub częściowo zasłonięte przesyłki często generują mniej stabilne detekcje niż duże kartony. W takich warunkach dwuetapowa asocjacja ogranicza fragmentację trajektorii oraz zmniejsza prawdopodobieństwo wielokrotnego zliczenia tej samej paczki po ponownym nadaniu jej nowego identyfikatora [48]. ByteTrack jest niezależny od konkretnej architektury detektora i może być łączony z różnymi modelami wykrywania obiektów.

Mimo swoich zalet ByteTrack nie jest rozwiązaniem pozbawionym ograniczeń. Nadal pozostaje on metodą *tracking-by-detection*, a więc jego skuteczność w dużym stopniu zależy od jakości i stabilności wyników detektora; długotrwałe braki detekcji mogą nadal prowadzić do zerwania ciągłości śledzenia. Ponadto w scenach o bardzo dużym zagęszczeniu obiektów oraz długich przesłonięciach metody wykorzystujące dodatkowe cechy wyglądu, takie jak BoT-SORT, mogą zapewnić większą odporność na przełączenia tożsamości [1]. Pomimo tych ograniczeń ByteTrack stanowi atrakcyjny kompromis pomiędzy prostotą, szybkością działania oraz odpornością na chwilowe spadki pewności detekcji, dlatego jest

uzasadnionym wyborem jako jeden z głównych algorytmów śledzenia analizowanych w niniejszej pracy.

2.3.2. BoT-SORT

BoT-SORT jest algorytmem śledzenia wielu obiektów rozwijanym w paradygmacie *tracking-by-detection*, w którym detekcje uzyskane w kolejnych klatkach obrazu są łączone w trajektorie zachowujące tożsamość obiektów w czasie. Metoda ta została zaproponowana jako rozbudowane i praktyczne rozwinięcie klasycznych algorytmów śledzenia czasu rzeczywistego, mające ograniczyć typowe problemy prostszych *tracker*ów, takie jak podatność na ruch kamery, niedokładne przewidywanie położenia ramek ograniczających oraz niewystarczające wykorzystanie informacji o wyglądzie obiektu podczas asocjacji. Zamiast wprowadzać całkowicie nowy paradygmat śledzenia, BoT-SORT łączy w jednym spójnym *pipeline* kilka usprawnień znanych wcześniej z literatury, integrując je w jednolitą i praktyczną metodę śledzenia [5].

Jednym z kluczowych elementów BoT-SORT jest ulepszony model ruchu oparty na filtrze Kalmana. W odróżnieniu od klasycznego SORT, w którym reprezentacja stanu obiektu jest uproszczona i opiera się między innymi na środku ramki oraz proporcji boków, BoT-SORT stosuje bardziej adekwatny opis geometrii obiektu, co prowadzi do dokładniejszego przewidywania jego położenia pomiędzy kolejnymi klatkami. Ma to istotne znaczenie w sytuacjach, gdy rozmiar lub kształt obiektu zmienia się w obrazie, na przykład wskutek zmiany perspektywy, obrotu, częściowego przesłonięcia albo niestabilności detekcji [1].

Drugim ważnym komponentem algorytmu jest kompensacja ruchu kamery (ang. Camera Motion Compensation, CMC). Jej zadaniem jest oszacowanie globalnego przesunięcia tła pomiędzy kolejnymi klatkami i skorygowanie przewidywanych pozycji śledzonych obiektów jeszcze przed etapem dopasowywania detekcji do istniejących trajektorii. W typowych zastosowaniach monitoringu mobilnego mechanizm ten ma szczególnie duże znaczenie, ale także w systemach przemysłowych z pozornie nieruchomą kamerą może poprawiać stabilność śledzenia w obecności drgań konstrukcji, wibracji przenośnika lub niewielkich poruszeń układu rejestracji obrazu [22].

Z punktu widzenia klasyfikacji metod śledzenia BoT-SORT należy do grupy algorytmów online, a więc takich, które mogą działać na strumieniu wideo bez konieczności analizy całej sekwencji z wyprzedzeniem. Cecha ta ma duże znaczenie praktyczne, ponieważ umożliwia bezpośrednią integrację *tracker*a z detektorem obiektów w systemach działających w czasie rzeczywistym lub bliskim rzeczywistemu, bez potrzeby stosowania kosztownej optymalizacji offline[9]. W implementacjach stosowanych razem z modelami z rodziny YOLO, BoT-SORT jest często udostępniany jako rozszerzony *tracker* bazujący na koncepcjach ByteTrack, wzbogacony o moduł kompensacji ruchu kamery i opcjonalną reidentyfikację obiektów.

[9] Termin ewaluacja offline odnosi się w niniejszej pracy do oceny jakości modelu przeprowadzanej na statycznym zbiorze walidacyjnym po zakończeniu procesu treningu, bez ograniczeń czasowych. Wyniki tej ewaluacji (mAP, precyzja, czułość) charakteryzują zdolności predykcyjne modelu niezależnie od warunków przetwarzania strumieniowego.

Skuteczność BoT-SORT została potwierdzona na popularnych benchmarkach MOTChallenge[10], gdzie w momencie publikacji należał on do najlepszych metod śledzenia online [1]. W literaturze podawane są bardzo wysokie wyniki dla zbiorów MOT17[11] i MOT20[12], co wskazuje, że metoda jest nie tylko koncepcyjnie interesująca, ale również empirycznie dojrzała i konkurencyjna względem innych *tracker*ów swojej klasy.

W kontekście niniejszej pracy BoT-SORT stanowi wartościowy punkt odniesienia wobec lżejszych metod, takich jak ByteTrack czy SORT, ponieważ oferuje bardziej zaawansowaną asocjację opartą nie tylko na modelu ruchu, ale również na dodatkowych wskazówkach wizualnych i mechanizmie kompensacji ruchu kamery [9]. Jego użycie ma umożliwić ocenę, czy bardziej rozbudowany model asocjacji przynosi wymierne korzyści w zadaniu detekcji i zliczania małych oraz dużych paczek na taśmie transportowej, co jest bezpośrednio powiązane z częścią eksperymentalną pracy.

2.3.3. SORT

SORT (Simple Online and Realtime Tracking) należy do najbardziej wpływowych i najczęściej przywoływanych algorytmów śledzenia wieloobiektowego rozwijanych w paradygmacie *tracking-by-detection*. Metoda została zaproponowana jako rozwiązanie celowo lekkie, którego celem nie było modelowanie wszystkich trudności występujących w śledzeniu, lecz pokazanie, że połączenie silnego detektora obiektów z prostym modelem ruchu i wydajną procedurą asocjacji danych może już zapewniać konkurencyjne wyniki w scenariuszach online czasu rzeczywistego. Z tego względu SORT jest często traktowany jako klasyczny punkt odniesienia dla późniejszych *tracker*ów, ponieważ priorytetowo traktuje szybkość działania, prostotę implementacji oraz modularność całego *pipeline*'u [4].

Z metodologicznego punktu widzenia SORT wyraźnie oddziela etap detekcji od etapu śledzenia. W każdej klatce obrazu zewnętrzny detektor generuje zbiór ramek ograniczających, natomiast zadaniem *tracker*a jest skojarzenie bieżących detekcji z trajektoriami aktywnymi w poprzednich klatkach. Taka konstrukcja ułatwia łączenie algorytmu z różnymi detektorami i jest szczególnie użyteczna w badaniach porównawczych, w których istotne jest osobne analizowanie wpływu jakości detekcji oraz jakości śledzenia na wynik końcowy.

Podstawę działania SORT stanowią trzy klasyczne komponenty: filtr Kalmana, miara Intersection over Union (IoU) oraz algorytm węgierski. Każdy śledzony obiekt jest reprezentowany przez wektor stanu obejmujący położenie środka ramki ograniczającej, jej skalę, proporcje boków oraz składowe prędkości, a jego stan jest propagowany pomiędzy kolejnymi klatkami z wykorzystaniem liniowego modelu ruchu o stałej prędkości. Następnie podobieństwo pomiędzy predykcjami ścieżek śledzenia a nowymi detekcjami wyznaczane jest na podstawie IoU, a końcowe dopasowanie jeden do jednego realizowane

[10] „MOTChallenge to benchmark dla śledzenia wielu obiektów za pomocą pojedynczej kamery, uruchomiony pod koniec 2014 roku w celu zebrania istniejących i nowych danych oraz stworzenia ujednoliconych ram dla standaryzowanej ewaluacji metod śledzenia wielu obiektów" [10].

[11] „MOT17 rozszerza sekwencje MOT16 o bardziej precyzyjne etykiety i ocenia skuteczność śledzenia dla trzech różnych detektorów obiektów" [10].

[12] „MOT20 to zbiór danych składający się z 8 nowych sekwencji przedstawiających bardzo zatłoczone, wymagające sceny" [10].

jest za pomocą algorytmu węgierskiego. Po zakończeniu asocjacji dopasowane ścieżki są aktualizowane, niedopasowane detekcje mogą inicjalizować nowe trajektorie, a ścieżki pozostające bez dopasowania przez określoną liczbę klatek są usuwane z aktywnego zbioru [5].

Istotną zaletą SORT jest jego charakter online oraz bardzo wysoka wydajność — w oryginalnej publikacji wskazano szybkość działania rzędu około 260 Hz, czyli ponad dwudziestokrotnie wyższą niż w przypadku innych *tracker*ów porównywanych przez autorów w czasie publikacji pracy [5]. Z praktycznego punktu widzenia oznacza to, że SORT może pełnić rolę bardzo lekkiej warstwy śledzenia, która dodaje jedynie niewielki narzut obliczeniowy do całego systemu detekcji i analizy wideo [4].

Minimalistyczna konstrukcja SORT wiąże się jednak z istotnymi ograniczeniami. Algorytm opiera się wyłącznie na informacjach geometrycznych, nie wykorzystując deskryptorów wyglądu ani jawnej reidentyfikacji obiektów. W efekcie jego skuteczność jest silnie zależna od jakości i stabilności detektora, a chwilowe zaniki detekcji, częściowe przesłonięcia lub duże podobieństwo obiektów mogą prowadzić do fragmentacji trajektorii oraz przełączeń identyfikatorów. To właśnie te ograniczenia stały się impulsem do opracowania rozszerzeń, takich jak DeepSORT, Observation-Centric SORT czy Hybrid-SORT.

W kontekście niniejszej pracy SORT pełni rolę lekkiej metody referencyjnej, pozwalającej ocenić, na ile poprawność zliczania paczek może zostać osiągnięta dzięki prostemu modelowi ruchu i geometrycznej asocjacji, a na ile konieczne jest zastosowanie bardziej zaawansowanych *tracker*ów.

2.4. Porównanie podejść — wady, zalety, ograniczenia

Analizowane w niniejszej pracy podejścia do detekcji i śledzenia obiektów reprezentują odmienne filozofie projektowania systemów wizyjnych, mimo że wszystkie prowadzą do realizacji tego samego celu praktycznego, jakim jest wykrywanie i zliczanie przesyłek poruszających się po taśmie transportowej. Ich porównanie nie powinno być jednak sprowadzane wyłącznie do pojedynczej miary, takiej jak mAP lub FPS, ponieważ w zastosowaniach logistycznych równie istotne są stabilność działania, odporność na okluzje, koszt obliczeniowy oraz łatwość wdrożenia w docelowym środowisku sprzętowym.

W przypadku modeli detekcji pierwszą istotną osią porównania jest relacja pomiędzy szybkością inferencji a jakością lokalizacji obiektów. Modele z rodziny YOLO, w tym YOLOv11, należą do architektur jednoetapowych, w których lokalizacja i klasyfikacja obiektu realizowane są w jednym przebiegu sieci. Takie podejście sprzyja niskim opóźnieniom, wysokiej przepustowości i relatywnie prostej integracji z systemami działającymi w czasie rzeczywistym, dlatego YOLOv11 stanowi naturalnego kandydata do zastosowań opartych na analizie strumienia wideo z przenośnika taśmowego [14].

Dodatkową zaletą YOLOv11 jest skalowalność całej rodziny modeli, obejmującej warianty lekkie i bardziej rozbudowane. Ma to duże znaczenie praktyczne, ponieważ pozwala dostosować tę samą architekturę do różnych środowisk obliczeniowych, od wydajnych kart GPU po słabsze konfiguracje sprzętowe. W kontekście projektowanej aplikacji jest

to szczególnie istotne, gdyż umożliwia porównanie kompromisu pomiędzy dokładnością a wydajnością bez konieczności całkowitej zmiany paradygmatu detekcji [42].

Jednocześnie zalety YOLOv11 wiążą się z pewnymi ograniczeniami typowymi dla detektorów jednoetapowych. W trudniejszych scenach, obejmujących małe obiekty, częściowe przesłonięcia lub bliskie sąsiedztwo wielu paczek o zróżnicowanych rozmiarach, model może wykazywać większą wrażliwość na ustawienia progów ufności oraz procedurę non-maximum suppression. Oznacza to, że osiągnięcie stabilnej równowagi pomiędzy fałszywymi alarmami a pominiętymi detekcjami może wymagać starannego strojenia parametrów i doboru odpowiedniego wariantu wielkości modelu.

Odmienne podejście reprezentuje RF-DETR, należący do grupy nowoczesnych detektorów transformatorowych. Jego najważniejszą cechą jest potraktowanie detekcji jako problemu bezpośredniej predykcji zbioru obiektów w architekturze end-to-end, co ogranicza zależność od klasycznych heurystyk, takich jak anchor boxes czy non-maximum suppression. Dzięki bardziej globalnemu modelowaniu relacji przestrzennych RF-DETR może lepiej radzić sobie w scenach gęstych, z częściowym nakładaniem się obiektów oraz dużą zmiennością skali przesyłek [33].

Z perspektywy praktycznej RF-DETR jest więc rozwiązaniem atrakcyjnym tam, gdzie najważniejsza jest odporność na trudniejsze przypadki i jakość detekcji w bardziej złożonym obrazie. Jednocześnie modele transformatorowe są zwykle bardziej wymagające pod względem pamięci, optymalizacji i efektywności sprzętowej niż dojrzałe architektury CNN przeznaczone do pracy w czasie rzeczywistym. W rezultacie RF-DETR może oferować wysoką jakość predykcji, lecz za cenę cięższego profilu wdrożeniowego, co nabiera znaczenia szczególnie w środowiskach CPU lub przy restrykcyjnych wymaganiach dotyczących opóźnienia inferencji.

Trzecie z analizowanych rozwiązań, Faster R-CNN, reprezentuje klasyczny paradygmat dwuetapowy. Jego główną zaletą jest rozdzielenie etapu generowania propozycji regionów od etapu ich klasyfikacji i doprecyzowania ramek ograniczających, co historycznie przekładało się na wysoką jakość lokalizacji i dużą precyzję detekcji. Z tego względu Faster R-CNN pozostaje ważnym punktem odniesienia metodologicznego oraz wartościowym modelem referencyjnym w badaniach porównawczych [30].

Największą wadą Faster R-CNN jest jednak wysoki koszt obliczeniowy. Dwuetapowy charakter przetwarzania oznacza zwykle większe opóźnienie i niższą przepustowość niż w przypadku współczesnych modeli jednoetapowych, co ogranicza jego użyteczność w systemach analizujących szybki strumień przesyłek w czasie rzeczywistym. W praktyce oznacza to, że Faster R-CNN lepiej sprawdza się jako silny punkt odniesienia jakościowego niż jako najbardziej naturalny wybór do wdrożenia w produkcyjnym systemie liczenia paczek na taśmie transportowej.

Porównanie modeli detekcji nabiera pełnego znaczenia dopiero po uwzględnieniu warstwy śledzenia wieloobiektowego, ponieważ poprawność zliczania zależy nie tylko od wykrycia paczki w pojedynczej klatce, lecz także od utrzymania jej spójnej tożsamości w czasie. W tym kontekście SORT stanowi rozwiązanie bazowe, bardzo lekkie obliczeniowo i proste implementacyjnie. Jego zaletą jest niewielki narzut wydajnościowy, natomiast podstawową słabością pozostaje silna zależność od jakości detektora oraz podatność na

fragmentację trajektorii w przypadku chwilowych zaników detekcji, przesłonięć lub bliskiego sąsiedztwa obiektów.

ByteTrack rozwija tę ideę w sposób szczególnie użyteczny dla zastosowań logistycznych. Zamiast odrzucać detekcje o niższej ufności, próbuje wykorzystać je w procesie asocjacji, co pozwala lepiej podtrzymywać trajektorie obiektów chwilowo gorzej widocznych. W zadaniu zliczania paczek ma to duże znaczenie, ponieważ małe przesyłki, koperty bąbelkowe lub częściowo zasłonięte obiekty mogą generować mniej stabilne detekcje, mimo że nadal są rzeczywistymi elementami sceny. Dzięki temu ByteTrack często oferuje bardzo korzystny kompromis pomiędzy szybkością działania, prostotą *pipeline*'u oraz stabilnością zliczania.

BoT-SORT reprezentuje bardziej zaawansowane podejście do śledzenia. Łączy on informacje o ruchu z silniejszym mechanizmem asocjacji, kompensacją ruchu kamery oraz opcjonalnym wykorzystaniem cech wyglądu obiektów. Takie rozwiązanie może skuteczniej ograniczać liczbę przełączeń tożsamości w trudniejszych scenach, zwłaszcza wtedy, gdy obiekty są do siebie podobne, częściowo się zasłaniają lub poruszają się blisko siebie.

Zwiększona odporność BoT-SORT nie jest jednak darmowa. Wiąże się ona z większą złożonością implementacyjną oraz wyższym kosztem obliczeniowym niż w przypadku SORT czy ByteTrack. W rezultacie BoT-SORT może poprawiać stabilność śledzenia w bardziej wymagających warunkach, ale nie w każdym scenariuszu zysk ten będzie uzasadniał dodatkowe obciążenie, szczególnie wtedy, gdy system ma działać na słabszym sprzęcie lub jednocześnie realizować także inne zadania, takie jak wizualizacja wyników czy obsługa aplikacji webowej [1].

Z punktu widzenia całego systemu nie istnieje jedno rozwiązanie uniwersalnie najlepsze. YOLOv11 wydaje się najsilniejszym kandydatem w scenariuszach, w których kluczowe są niskie opóźnienia, wysoka przepustowość i względnie proste wdrożenie. RF-DETR staje się szczególnie interesujący wtedy, gdy priorytetem jest jakość detekcji w trudnych scenach i większa odporność na złożone relacje przestrzenne pomiędzy obiektami. Faster R-CNN zachowuje wartość przede wszystkim jako solidny model referencyjny, pozwalający lepiej zrozumieć kompromis między dokładnością a wydajnością.

ByteTrack stanowi rozwiązanie bardzo praktyczne dla systemów zliczających opartych na detekcji, natomiast BoT-SORT jest opcją bardziej zaawansowaną, potencjalnie korzystną w trudniejszych przypadkach asocjacji. Ostateczna ocena tych metod nie powinna więc opierać się wyłącznie na benchmarkowych miarach dokładności, lecz także na dopasowaniu do domeny, ograniczeniach sprzętowych oraz stabilności całego łańcucha detekcja–śledzenie–zliczanie.

2.5. Uzasadnienie wyboru metod zastosowanych w pracy

Dobór metod zastosowanych w niniejszej pracy wynika bezpośrednio z celu badawczego, którym jest opracowanie, implementacja i ewaluacja demonstracyjnego systemu wizyjnego do detekcji oraz zliczania małych i dużych przesyłek pocztowych przemieszczających się po taśmie transportowej w czasie rzeczywistym. W takim scenariuszu nie wystarcza analiza pojedynczego modelu detekcji w oderwaniu od reszty *pipeline*'u, ponieważ

praktyczna użyteczność rozwiązania zależy jednocześnie od jakości lokalizacji obiektów, stabilności ich śledzenia, poprawności końcowego zliczania oraz kosztu obliczeniowego całego systemu [28]. Z tego względu w pracy przyjęto metodykę porównawczą obejmującą kilka komplementarnych paradygmatów detekcji i śledzenia, tak aby odpowiedzieć nie tylko na pytanie, który model osiąga najlepszy wynik liczbowy, lecz także, które założenia architektoniczne najlepiej odpowiadają wymaganiom analizowanego problemu.

Wybór trzech detektorów — YOLOv11, RF-DETR oraz Faster R-CNN — jest uzasadniony metodologicznie, ponieważ modele te reprezentują trzy odmienne podejścia do detekcji obiektów: odpowiednio architekturę jednoetapową, transformatorową architekturę *end-to-end* oraz klasyczne podejście dwuetapowe. Taki dobór pozwala przeanalizować kompromis pomiędzy szybkością inferencji, precyzją lokalizacji, odpornością na częściowe okluzje oraz złożonością obliczeniową w warunkach typowych dla taśmy transportowej, gdzie obiekty istotnie różnią się gabarytami, mogą się częściowo zasłaniać i muszą być analizowane w strumieniu wideo. Porównanie tych modeli ma zatem wartość nie tylko aplikacyjną, ale również poznawczą, ponieważ umożliwia zestawienie różnych filozofii projektowania nowoczesnych detektorów w jednym, spójnym scenariuszu badawczym.

YOLOv11 wybrano jako podstawowego reprezentanta współczesnych detektorów jednoetapowych, ponieważ rodzina YOLO od lat stanowi jeden z najważniejszych punktów odniesienia w zastosowaniach wymagających wysokiej szybkości działania. W kontekście niniejszej pracy jego najważniejsze zalety to bardzo dobra wydajność inferencji, skalowalność do wielu wariantów rozmiarowych oraz praktyczna przydatność w zadaniach analizy strumienia wideo, w tym w systemach opartych na przenośnikach i automatycznym sortowaniu obiektów [12]. Jest to szczególnie istotne, ponieważ jedna z osi badania dotyczy porównania małych i dużych wariantów modeli, a druga praktycznej wykonalności wnioskowania na CPU i GPU; do takiej analizy potrzebna była architektura dostępna w kilku skalach i projektowana z myślą o efektywnym wdrożeniu. Dodatkowo właściwości wieloskalowe YOLOv11 są ważne z punktu widzenia analizowanego problemu, ponieważ system ma wykrywać zarówno niewielkie koperty bąbelkowe i foliopaki, jak i duże paczki kartonowe w tym samym kadrze.

RF-DETR włączono do badań jako reprezentanta współczesnego nurtu transformatorowego, który rozwija detekcję obiektów w kierunku predykcji *end-to-end* i ograniczenia zależności od klasycznych heurystyk postprocessingu. Jego obecność poszerza analizę poza architektury zdominowane przez konwolucje i pozwala zweryfikować, czy bardziej globalne modelowanie relacji przestrzennych przynosi korzyści w scenach z częściowymi okluzjami, zróżnicowaną skalą obiektów oraz nieregularnym układem przesyłek na taśmie [31]. W praktyce RF-DETR pełni więc nie tylko rolę dodatkowego benchmarku, ale także testu aktualnego kierunku rozwoju detekcji, którego przydatność w zadaniach logistycznych wymaga empirycznej weryfikacji.

Faster R-CNN został wybrany jako klasyczny model referencyjny reprezentujący podejście dwuetapowe. Choć architektura ta zwykle ustępuje nowoczesnym modelom czasu rzeczywistego pod względem opóźnienia inferencji, nadal pozostaje ważnym punktem odniesienia dzięki wysokiej jakości lokalizacji i dużemu znaczeniu historycznemu w rozwoju detekcji obiektów. Uwzględnienie Faster R-CNN wzmacnia projekt eksperymentu, ponie-

waż pozwala ocenić, czy wzrost szybkości uzyskiwany przez YOLOv11 i RF-DETR nie odbywa się kosztem stabilności predykcji oraz dokładności lokalizacji. W konsekwencji model ten pełni w pracy funkcję jakościowego punktu odniesienia, a nie wyłącznie potencjalnego kandydata wdrożeniowego.

Zestawienie YOLOv11, RF-DETR i Faster R-CNN jest metodologicznie uzasadnione, ponieważ obejmuje trzy różne sposoby rozwiązywania tego samego problemu — podejście jednoetapowe, transformatorowe end-to-end oraz klasyczne podejście dwuetapowe. Dzięki temu porównanie tych modeli nie sprowadza się wyłącznie do wskazania, który z nich osiąga najlepszy wynik liczbowy, ale pozwala również wyjaśnić, z jakich założeń architektonicznych wynikają różnice w skuteczności, szybkości działania i kosztach obliczeniowych.

Sama warstwa detekcji nie jest jednak wystarczająca do poprawnego zliczania przesyłek w materiale wideo, ponieważ ten sam obiekt występuje w wielu kolejnych klatkach i bez zachowania tożsamości mógłby zostać policzony wielokrotnie. Z tego względu w pracy przyjęto paradygmat *tracking-by-detection*, który umożliwia osobną analizę jakości detekcji i jakości śledzenia, a jednocześnie dobrze odpowiada praktyce współczesnych systemów wieloobiektowego śledzenia. Wybór tego paradygmatu jest szczególnie uzasadniony w analizowanym zadaniu, gdzie występują chwilowe zaniki detekcji, częściowe przesłonięcia, bliskie sąsiedztwo obiektów oraz ryzyko fragmentacji trajektorii prowadzącej bezpośrednio do błędów zliczania.

W warstwie śledzenia wybrano trzy algorytmy: SORT, ByteTrack oraz BoT-SORT. Taki zestaw tworzy przekrój metod o różnym stopniu złożoności — od lekkiego baseline'u (metoda bazowa), przez rozwiązanie zoptymalizowane pod kątem ciągłości śledzenia przy niestabilnych detekcjach, aż po bardziej rozbudowany *tracker* oferujący silniejszą strategię asocjacji. Dzięki temu możliwe jest zbadanie nie tylko tego, który tracker daje najlepszy wynik, ale również tego, jak bardzo poprawa jakości śledzenia uzasadnia dodatkowy koszt obliczeniowy w systemie działającym blisko czasu rzeczywistego.

ByteTrack uznano za szczególnie ważny element badań, ponieważ jego strategia wykorzystania również detekcji o niższej pewności dobrze odpowiada charakterowi analizowanego problemu. W scenach z małymi przesyłkami, cienkimi opakowaniami foliowymi lub częściowymi okluzjami słabsze detekcje nie muszą oznaczać fałszywych alarmów, lecz mogą być jedynym sygnałem podtrzymującym tożsamość obiektu w kolejnych klatkach. Z tego powodu ByteTrack ogranicza ryzyko fragmentacji trajektorii i wtórnych błędów zliczania, zachowując przy tym wysoką efektywność oraz niezależność od konkretnej rodziny detektorów; w literaturze porównawczej jest też wskazywany jako rozwiązanie wyraźnie skuteczniejsze od prostych baseline'ów śledzenia przy zachowaniu korzystnej wydajności [38].

Z kolei SORT został pozostawiony jako lekki punkt odniesienia, pozwalający sprawdzić, ile można uzyskać dzięki samemu modelowi ruchu i asocjacji geometrycznej przy minimalnym narzucie obliczeniowym.

BoT-SORT włączono natomiast dlatego, że oferuje bardziej zaawansowaną asocjację opartą nie tylko na modelu ruchu, ale również na mechanizmie kompensacji ruchu kamery. Jego obecność w zestawie trackerów pozwala zweryfikować, czy bardziej rozbudowany

model asocjacji przynosi wymierne korzyści w zadaniu detekcji i zliczania małych oraz dużych paczek na taśmie transportowej — lub czy wzrost złożoności obliczeniowej nie jest uzasadniony w tym konkretnym scenariuszu.

Dodatkowym argumentem za wyborem ByteTrack jest jego niezależność od konkretnej architektury detektora — może być łączony z YOLOv11, RF-DETR i Faster R-CNN bez modyfikacji pozostałych etapów *pipeline*'u, co jest niezbędnym warunkiem przeprowadzenia rzetelnego porównania wpływu warstwy śledzenia na końcową dokładność zliczania.

Osobnego uzasadnienia wymaga porównanie małych i dużych wariantów modeli oraz inferencji na CPU i GPU. Eksperymenty te nie stanowią jedynie technicznego dodatku, ale odpowiadają na praktyczne pytanie, czy bardziej rozbudowane modele rzeczywiście dają istotny przyrost jakości oraz czy system może zachować użyteczność również w środowisku o ograniczonych zasobach sprzętowych [16]. Z perspektywy zastosowań logistycznych najlepszym rozwiązaniem nie jest bowiem model o najwyższej izolowanej skuteczności, lecz taki, który zapewnia najkorzystniejszy kompromis między dokładnością detekcji, poprawnością zliczania, przepustowością oraz dostępnością infrastruktury obliczeniowej. Taka perspektywa dobrze odpowiada rzeczywistym warunkom wdrożeniowym, w których ograniczenia sprzętowe i kosztowe są równie ważne jak same wyniki jakościowe. Dodatkowym argumentem za takim zestawem metod jest demonstracyjny, *proof-of-concept*owy charakter całego projektu. Połączenie kilku rodzin detektorów, kilku trackerów oraz porównań wariantów modelowych i platform sprzętowych pozwala sformułować wnioski nie tylko o jakości algorytmicznej, lecz także o wykonalności implementacyjnej w realnym systemie, czego dopełnieniem jest przygotowana aplikacja webowa udostępniająca opracowany *pipeline* użytkownikowi końcowemu. W rezultacie wybrany zestaw metod tworzy spójny układ eksperymentalny: YOLOv11, RF-DETR i Faster R-CNN reprezentują główne paradygmaty detekcji, natomiast SORT, ByteTrack i BoT-SORT reprezentują komplementarne strategie śledzenia. Dzięki temu możliwa jest rzetelna weryfikacja hipotezy pracy oraz udzielenie odpowiedzi na pytania badawcze dotyczące kompromisu między skutecznością detekcji, stabilnością śledzenia, poprawnością zliczania i wydajnością obliczeniową.

Rozdział 3

Dane i ich przygotowanie

Budowa wiarygodnego systemu detekcji i zliczania obiektów nie rozpoczyna się od wyboru modelu, lecz od zgromadzenia i odpowiedniego przygotowania danych. Jakość, różnorodność i reprezentatywność zbioru danych wyznaczają praktyczny pułap skuteczności systemu, ponieważ nawet zaawansowane architektury głębokiego uczenia nie są w stanie w pełni skompensować braków wynikających z niejednorodnych, błędnie oznaczonych lub niereprezentatywnych danych. W przypadku analizy paczek poruszających się na taśmie transportowej znaczenie tego etapu jest szczególnie duże, gdyż dane muszą możliwie wiernie odzwierciedlać rzeczywiste warunki pracy systemu, w tym zmienną skalę obiektów, częściowe przesłonięcia, rozmycie ruchu, zróżnicowanie materiałów opakowaniowych oraz gęste ułożenie przesyłek.

W niniejszej pracy zbiór danych pełni podwójną funkcję. Z jednej strony stanowi podstawę do treningu i porównania modeli detekcyjnych, a z drugiej umożliwia ocenę skuteczności śledzenia i zliczania paczek w sekwencjach wideo, gdzie ten sam obiekt pozostaje widoczny w wielu kolejnych klatkach. Z tego względu przygotowanie danych obejmuje nie tylko ich pozyskanie i adnotację, lecz także czyszczenie zbioru, doprecyzowanie definicji klas, wstępne przetwarzanie, augmentację oraz podział materiału na zbiór treningowy i walidacyjny w proporcji 96%/4%, co pozwoliło przeznaczyć maksymalną liczbę przykładów do procesu uczenia.

3.1. Charakterystyka problemu: małe i duże paczki pocztowe

Rozróżnienie między małymi a dużymi paczkami pocztowymi stanowi jeden z kluczowych aspektów praktycznych i metodologicznych problemu analizowanego w tej pracy. Z perspektywy człowieka podział ten wydaje się intuicyjny, jednak w systemie wizyjnym nie opiera się on na semantycznej intuicji, lecz na wzorcach wizualnych wydobywanych z obrazu, map cech i zależności czasowych pomiędzy kolejnymi klatkami [4, 18].

W kontekście analizy obrazu pojęć „mała” i „duża” paczka nie należy rozumieć wyłącznie przez pryzmat rzeczywistych wymiarów fizycznych. Znacznie ważniejszy jest wizualny ślad obiektu w kadrze, czyli liczba pikseli zajmowanych przez przesyłkę przy danej konfiguracji kamery, rozdzielczości obrazu, perspektywie oraz geometrii taśmy transportowej. Ten sam obiekt może być zatem relatywnie łatwy do detekcji w jednym ustawieniu stanowiska i wyraźnie trudniejszy w innym, jeśli jego obraz będzie mniejszy, mniej kontrastowy lub częściowo zasłonięty.

Szczególnie problematyczne są małe przesyłki, takie jak koperty bąbelkowe, foliopaki czy niewielkie pudełka kartonowe. Obiekty tego typu zwykle zajmują niewielki obszar obrazu, przez co zawierają mniej informacji teksturalnej, mniej wyraźnych krawędzi i słabszy kontekst wizualny, co utrudnia ich poprawną lokalizację i klasyfikację. Dodatkowo małe paczki są bardziej podatne na wpływ rozmycia ruchu, artefaktów kompresji, zmian oświetlenia oraz błędów adnotacji, ponieważ nawet niewielka pomyłka lokalizacyjna stanowi w ich przypadku istotny procent pola obiektu [41].

Istotnym utrudnieniem jest także deformowalność części małych przesyłek. Foliopaki i częściowo wypełnione koperty bąbelkowe nie zachowują stałego kształtu, lecz dopasowują się do zawartości i sposobu ułożenia na taśmie, przez co ich kontur może się wyraźnie zmieniać pomiędzy przykładami, a nawet pomiędzy kolejnymi klatkami tego samego nagrania. Problem ten może dodatkowo nasilać się w warunkach oświetlenia przemysłowego, gdy powierzchnie z tworzyw sztucznych generują refleksy świetlne utrudniające ekstrakcję cech wizualnych.

Duże paczki tworzą odmienną klasę trudności. Zwykle są one łatwiejsze do wykrycia na poziomie pojedynczej klatki, ponieważ zajmują większy obszar obrazu i dostarczają bogatszej informacji wizualnej, ale równocześnie częściej przesłaniają inne obiekty w gęstych scenach logistycznych. W praktyce oznacza to, że duża paczka może częściowo lub niemal całkowicie zasłonić mniejszą przesyłkę, co zwiększa ryzyko pominiętych detekcji, fragmentacji trajektorii oraz podwójnego zliczenia po ponownej identyfikacji przez tracker.

Współwystępowanie małych i dużych paczek w jednym strumieniu wideo prowadzi zatem do asymetrycznego środowiska detekcji. Małe paczki są trudne dlatego, że bywają wizualnie słabe i zajmują niewielki obszar obrazu, natomiast duże są trudne dlatego, że wpływają na widoczność i rozmieszczenie innych obiektów w scenie. W systemie zliczania problem ten nie kończy się na jednorazowym wykryciu obiektu, ponieważ konieczne jest utrzymanie jego tożsamości w czasie, tak aby ta sama paczka nie była liczona wielokrotnie w kolejnych klatkach.

Dodatkową trudność stanowi duża zmienność wewnątrz obu klas. Do klasy małych paczek mogą należeć zarówno cienkie koperty bąbelkowe, lekkie foliopaki, jak i małe pudełka kartonowe, natomiast klasa dużych paczek obejmuje wszystkie duże obiekty. Oznacza to, że zadanie nie polega na rozdzieleniu dwóch idealnie jednorodnych kategorii geometrycznych, lecz na nauczeniu modelu uogólniania w warunkach szerokiej zmienności wewnątrzklasowej przy jednoczesnym zachowaniu czytelnej granicy między klasami [45].

Trudności te są wzmacniane przez dynamikę sceny. Nawet przy nieruchomej kamerze paczki poruszają się z określoną prędkością po taśmie, mogą rotować, stykać się, częściowo nachodzić na siebie i okresowo znikać z pełnej widoczności, co pogarsza stabilność zarówno detekcji, jak i śledzenia. W konsekwencji jakość końcowego systemu zależy nie tylko od samego detektora, lecz także od stabilności warstwy śledzenia, która musi przekształcić sekwencję pojedynczych detekcji w spójne trajektorie obiektów.

Z punktu widzenia przygotowania danych oznacza to konieczność zachowania wysokiej spójności adnotacji. Jeżeli granica między klasą „mała paczka” a klasą „duża paczka” zostanie zdefiniowana nieprecyzyjnie lub będzie stosowana niespójnie, zbiór danych wpro-

wadzi niejednoznaczność, która później ujawni się w niestabilnych predykcjach i mniej wiarygodnej ewaluacji ilościowej.

Rozróżnienie małych i dużych paczek ma również bezpośrednie znaczenie dla porównania modeli, ich wariantów rozmiarowych oraz środowisk sprzętowych. Lżejsze modele mogą oferować szybszą inferencję, ale jednocześnie gorzej radzić sobie z małymi, trudnymi wizualnie obiektami, natomiast większe warianty mogą poprawiać jakość detekcji kosztem wyższego obciążenia obliczeniowego, co staje się szczególnie ważne przy porównaniu inferencji na CPU i GPU oraz przy zadaniach bliskich czasu rzeczywistego. Tym samym podział na małe i duże paczki nie jest wyłącznie prostym problemem klasyfikacyjnym, lecz stanowi praktyczny test kompromisu między dokładnością, odpornością na zakłócenia i wydajnością obliczeniową całego systemu.

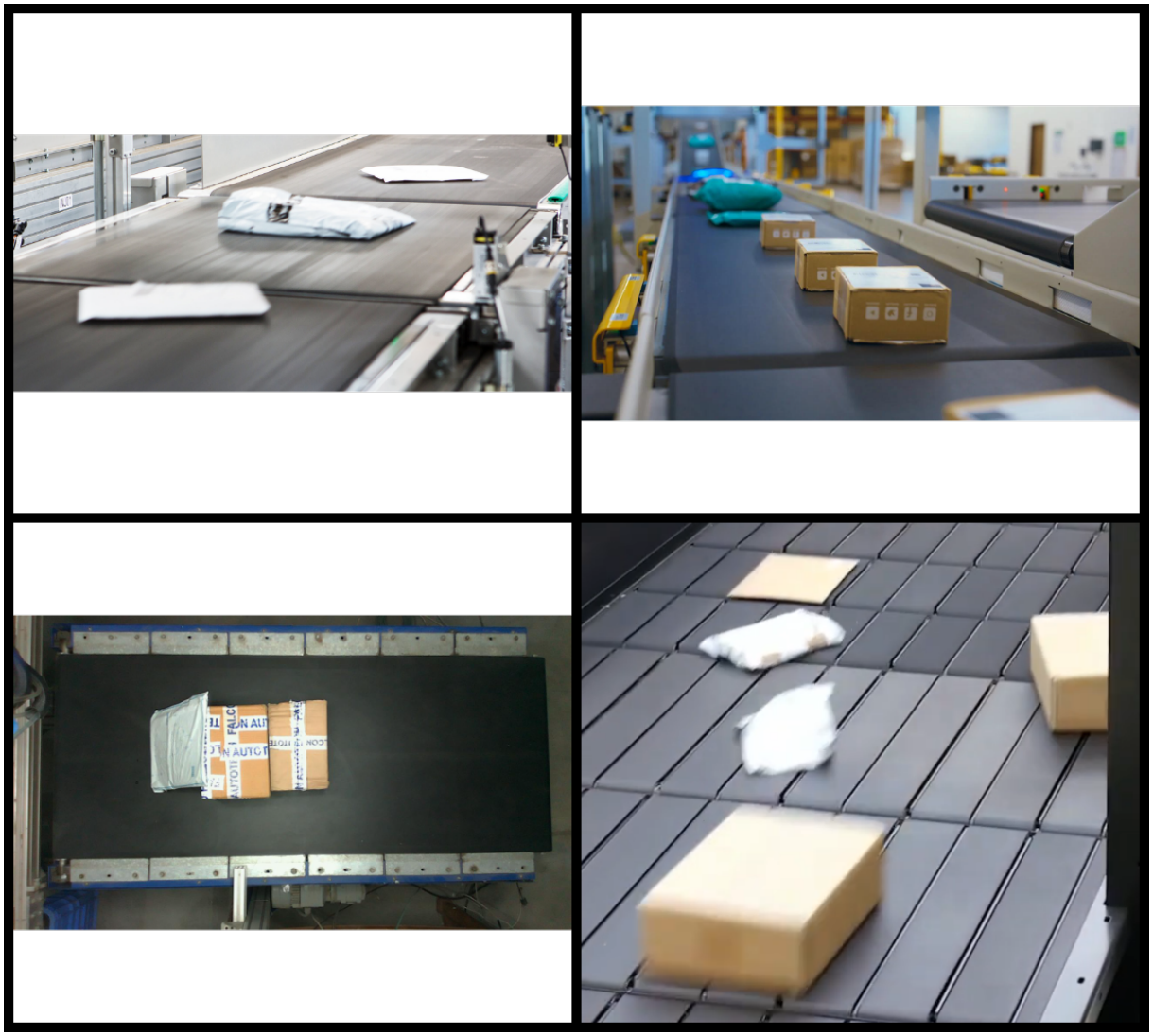
Z perspektywy zastosowań logistycznych znaczenie tego problemu jest jednocześnie naukowe i praktyczne. Błędy dotyczące małych przesyłek sprzyjają niedoliczeniom i utracie trajektorii, natomiast błędy dotyczące dużych paczek mogą zaburzać interpretację całej sceny poprzez zasłanianie innych obiektów i generowanie wtórnych błędów zliczania. Z tego względu precyzyjna charakterystyka obu klas stanowi punkt wyjścia dla dalszych etapów opisanych w niniejszym rozdziale — doboru i opisu źródeł danych, procesu adnotacji oraz analizy statystycznej zbioru [15].

3.2. Źródła i charakterystyka zbioru danych

Zbiór danych wykorzystany w niniejszej pracy nie został przejęty z jednego gotowego benchmarku, lecz został zbudowany samodzielnie jako korpus zadaniowy dostosowany do problemu detekcji dwóch klas przesyłek: małych i dużych paczek pocztowych. Przyjęcie takiego podejścia było konieczne, ponieważ dostępne w literaturze zbiory związane z logistyką koncentrują się zazwyczaj na węższych zadaniach, takich jak wykrywanie naruszeń opakowań, depaletyzacja pudeł o względnie jednorodnych kształtach albo analiza obiektów na taśmach sortowniczych w domenie innej niż przesyłki pocztowe [24]. W konsekwencji nie zidentyfikowano publicznie dostępnego zbioru, który wprost odpowiadałby potrzebom niniejszej pracy, tj. detekcji małych i dużych przesyłek w zróżnicowanych nagraniach z linii transportowej.

Przygotowany zbiór został zbudowany z dwóch komplementarnych rodzajów materiału: pojedynczych obrazów oraz klatek wyodrębnionych z publicznie dostępnych nagrań wideo przedstawiających pracę sortowni, linii taśmowych i środowisk magazynowych. Takie połączenie materiału statycznego i dynamicznego pozwala lepiej odwzorować rzeczywiste warunki pracy systemu wizyjnego, w których oprócz zróżnicowania samych obiektów występują także rozmycie ruchu, artefakty kompresji, chwilowe przesłonięcia oraz zmiany położenia paczek między kolejnymi klatkami. W przypadku materiału filmowego istotne było również ograniczenie nadmiernej podobności kolejnych ujęć, dlatego do zbioru wybierano klatki oddalone od siebie czasowo, co zwiększało różnorodność wizualną i zmniejszało ryzyko przecieku danych pomiędzy podzbiorami uczącymi i walidacyjnymi. Przykładowe obrazy wchodzące w skład przygotowanego zbioru, obejmujące zarówno pojedyncze foto-

grafie, jak i klatki wyodrębnione z publicznie dostępnych nagrań wideo, przedstawiono na rysunku 3.1.



Rysunek 3.1: Przykładowe obrazy wchodzące w skład zbioru danych: pojedyncze fotografie oraz klatki wyodrębnione z publicznie dostępnych nagrań przedstawiających pracę sortowni i linii transportowych.

Zgromadzony zbiór ma charakter celowo heterogeniczny. Obejmuje przykłady zróżnicowane pod względem kąta obserwacji, warunków oświetleniowych, jakości obrazu, rodzaju tła, tekstury taśmy, materiału opakowania, stopnia deformacji przesyłki oraz skali obiektu w kadrze. Z punktu widzenia uczenia modeli detekcyjnych taka różnorodność jest pożądana, ponieważ zwiększa szansę na uzyskanie lepszej zdolności uogólniania i większej odporności modelu na zmianę warunków obrazowania. Ma to szczególne znaczenie w analizowanym zadaniu, gdyż zarówno klasa małych paczek, jak i klasa dużych paczek obejmuje obiekty o szerokiej zmienności wewnętrznej, takie jak koperty bąbelkowe, foliopaki, niewielkie pudełka kartonowe oraz większe kartony o odmiennych proporcjach, kształtach i fakturach powierzchni. Zróżnicowanie wizualne zgromadzonego materiału, obejmujące

odmienne warunki oświetleniowe, geometrię sceny oraz skalę obiektów w kadrze, zilustrowano na rysunku 3.2.

Rysunek 3.2: Przykłady zróżnicowania wizualnego zbioru danych: odmienne warunki oświetleniowe, geometria sceny.

Istotną cechą zbioru jest również obecność scen trudnych z perspektywy detekcji i śledzenia obiektów. W wielu przykładach paczki nie występują w izolacji, lecz częściowo na siebie nachodzą, zasłaniają się wzajemnie lub poruszają się w niewielkich odstępach, co utrudnia poprawną lokalizację każdego obiektu i utrzymanie jego tożsamości w czasie. Dodatkowo małe przesyłki zajmują relatywnie niewielki obszar obrazu, przez co są bardziej podatne na wpływ rozmycia ruchu, kompresji i błędów adnotacji, natomiast większe paczki częściej przesłaniają inne obiekty, zaburzając zarówno detekcję, jak i późniejsze zliczanie. Przykłady scen szczególnie trudnych z perspektywy detekcji i śledzenia, takich jak częściowe przesłonięcia oraz niewielkie odstępy między obiektami, pokazano na rysunku 3.3.

Przyjęta strategia pozyskiwania danych ma jednak także istotne ograniczenia. Ponieważ materiał został zebrany z publicznych źródeł internetowych, zbiór nie spełnia warunków w pełni kontrolowanego planu próbkowania i może zawierać nierównomierną reprezentację niektórych typów przesyłek, powtarzalne wzorce wizualne oraz artefakty charakterystyczne dla konkretnych źródeł. Oznacza to, że odchylenia obecne na etapie gromadzenia danych mogą zostać częściowo przeniesione do wytrenowanych modeli i ujawnić się spadkiem skuteczności w środowiskach odmiennych od tych nadreprezentowanych w zbiorze treningowym. Z tego względu przygotowany zbiór należy traktować jako zbiór zadaniowy, wystarczający do przeprowadzenia eksperymentów porównawczych w ramach niniejszej pracy, lecz nie jako kompletną reprezentację wszystkich możliwych warunków przemysłowych spotykanych w sortowniach [11].

Rysunek 3.3: Przykłady scen trudnych w zbiorze danych: częściowe przesłonięcia, niewielkie odstępy między obiektami.

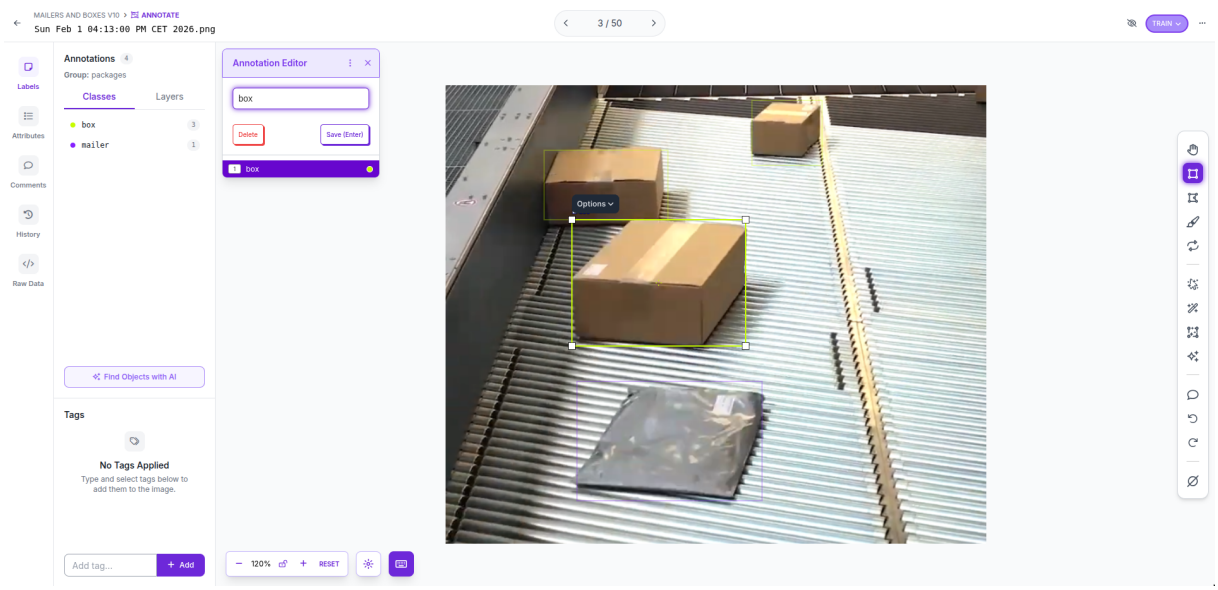
Ostatecznie najważniejszą właściwością przygotowanego zbioru nie jest ścisła standaryzacja, lecz praktyczna zmienność. Celem było zgromadzenie danych, które możliwie dobrze odwzorowują rzeczywiste trudności analizy obrazu w logistyce: zróżnicowaną skalę obiektów, niejednorodne opakowania, nieregularne ułożenie paczek oraz dynamiczny charakter nagrań z taśmy transportowej. Dzięki temu zbiór stanowi uzasadnioną podstawę do trenowania i porównywania modeli detekcji w warunkach bliższych praktyce aplikacyjnej niż laboratoryjnym, silnie kontrolowanym scenariuszom.

3.3. Adnotacje danych — narzędzia i proces etykietowania

Adnotacja danych stanowi jeden z kluczowych etapów przygotowania zbioru do uczenia nadzorowanego w zadaniach detekcji obiektów, ponieważ jakość etykiet wpływa bezpośrednio na zdolność modelu do poprawnej lokalizacji i klasyfikacji obiektów [21]. W kontekście niniejszej pracy etap ten nie był traktowany wyłącznie jako techniczny element poprzedzający trening, lecz jako istotna część całej metodologii badawczej. Wynika to z faktu, że nawet dobrze dobrana architektura detekcyjna może osiągać obniżoną skuteczność, jeżeli zostanie nauczona na zbiorze zawierającym niespójne, nieprecyzyjne lub niejednoznaczne oznaczenia.

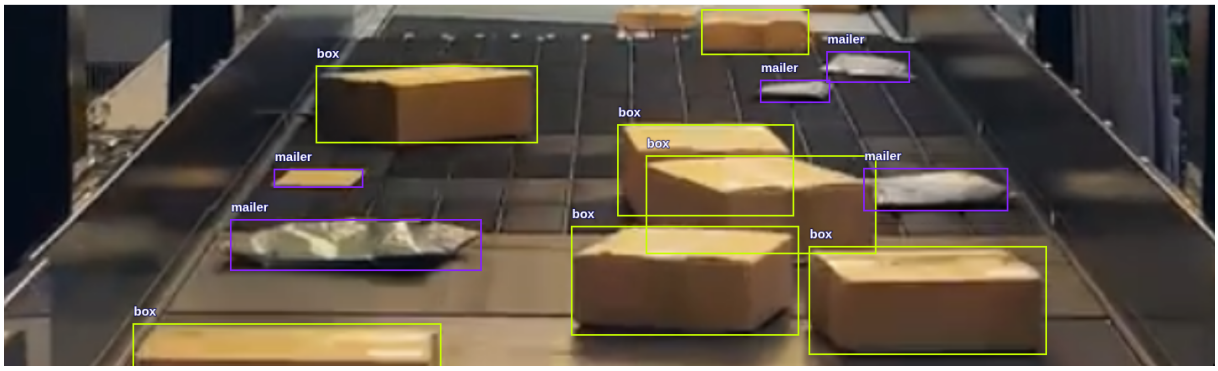
Do przeprowadzenia procesu adnotacji wykorzystano platformę Roboflow [35], która udostępnia przeglądarkowy interfejs do ręcznego oznaczania obrazów oraz wspiera późniejsze wersjonowanie zbioru danych w ramach jednego środowiska pracy. Platforma ta umożliwia tworzenie adnotacji z użyciem ramek ograniczających, edycję już narysowanych oznaczeń oraz przypisywanie im odpowiednich klas obiektów. W niniejszym projekcie, dostępnym w Roboflow pod nazwą mailers-and-boxes-v10[1], zastosowano adnotację typu *bounding box*, ponieważ odpowiada ona bezpośrednio zadaniu detekcji obiektów realizowanemu przez modele analizowane w dalszej części pracy. Każda paczka była oznaczana prostokątną ramką ograniczającą możliwie ciasno dopasowaną do widocznych granic obiektu, tak aby ograniczyć udział tła i jednocześnie zachować spójność przestrzenną etykiet w całym zbiorze. Na rysunku 3.4 przedstawiono interfejs platformy Roboflow, wykorzystanej w niniejszej pracy do ręcznej adnotacji obrazów metodą bounding box.

[1] https://universe.roboflow.com/lukmik-e4qay/mailers-and-boxes-v10



Rysunek 3.4: Interfejs platformy Roboflow wykorzystywany w procesie ręcznej adnotacji obrazów z użyciem ramek bounding box dla klas mailer i box.

W procesie etykietowania przyjęto podział na dwie klasy: mailer oraz box, co pozostaje zgodne z głównym problemem badawczym pracy, czyli rozróżnianiem małych i dużych przesyłek pocztowych na taśmie transportowej. Adnotacja była wykonywana ręcznie, obraz po obrazie, z każdorazową kontrolą poprawności położenia ramki oraz przypisanej klasy. Szczególną uwagę poświęcono przypadkom trudnym, takim jak częściowe zasłonięcia, niewielkie odstępy między sąsiednimi paczkami, deformowalne foliopaki, rozmycie ruchu oraz duża zmienność kształtu i tekstury opakowań, ponieważ to właśnie w takich sytuacjach najłatwiej o błędy adnotacyjne wpływające później na wyniki treningu i ewaluacji. W przypadkach granicznych stosowano tę samą logikę decyzyjną dla całego zbioru, tak aby ograniczyć subiektywność oznaczeń i zapewnić możliwie wysoką spójność *ground truth* pomiędzy wszystkimi próbkami. Przykład klatki ze zbioru treningowego z naniesionymi adnotacjami ground truth, prezentujący ramki ograniczające klas *box* oraz *mailer* wraz z ich etykietami, przedstawiono na rysunku 3.5.



Rysunek 3.5: Przykładowa klatka ze zbioru treningowego z naniesionymi adnotacjami ground truth — widoczne *bounding boxy* klas *box* oraz *mailer* wraz z ich etykietami.

Wybór Roboflow był uzasadniony także z praktycznego punktu widzenia, ponieważ platforma pozwala tworzyć wersje zbioru danych będące zapisanym w czasie stanem obrazów i etykiet, wraz z konfiguracją dalszych etapów przygotowania danych. Takie podejście ułatwia zachowanie powtarzalności eksperymentów, ponieważ kolejne treningi modeli mogą opierać się na tej samej bazie adnotacji, nawet jeśli różnią się ustawieniami preprocessingu, augmentacji lub formatem eksportu danych. W niniejszej pracy miało to szczególne znaczenie, ponieważ przygotowany zbiór był następnie wykorzystywany w eksperymentach porównawczych obejmujących różne architektury detekcyjne oraz różne środowiska uruchomieniowe. Ostatecznie etap adnotacji nie tylko dostarczył danych treningowych, ale również stworzył stabilną i spójną podstawę do dalszej analizy statystycznej zbioru oraz do interpretacji wyników przedstawionych w kolejnych rozdziałach.

3.4. Wstępna analiza statystyczna zbioru

Wstępna analiza statystyczna zbioru danych stanowi istotny etap poprzedzający trening modeli detekcji obiektów, ponieważ pozwala ocenić reprezentatywność zgromadzonego materiału, równowagę klas oraz właściwości obrazów mogące wpływać na późniejsze wyniki eksperymentów [21]. Z tego względu analizie poddano nie tylko łączną liczbę obrazów i adnotacji, lecz także udział przykładów negatywnych, rozkład klas, liczbę obiektów przypadających na pojedynczy obraz oraz zróżnicowanie rozdzielczości i proporcji boków. Wszystkie przedstawione w niniejszej sekcji dane statystyczne pochodzą z panelu analitycznego projektu udostępnionego na platformie Roboflow[2].

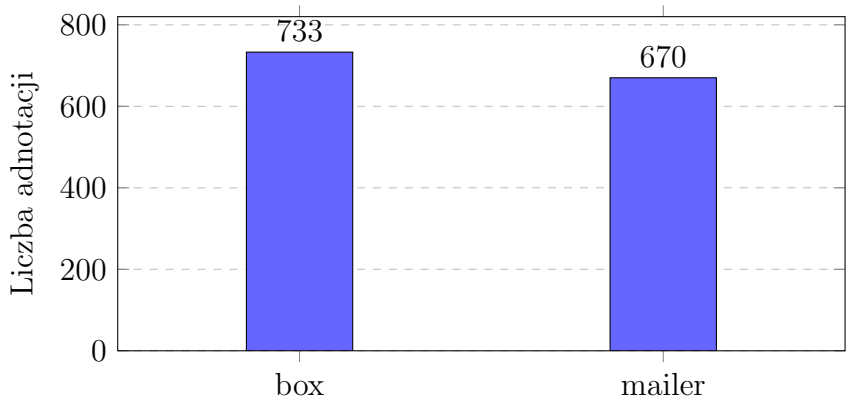
Podstawowe statystyki zbioru zestawiono w tabeli 3.1. Zbiór końcowy obejmuje 395 obrazów oraz 1403 adnotacje typu *bounding box*, co daje średnio 3,6 adnotacji na obraz. Pod względem liczebności jest to zbiór relatywnie niewielki, jednak wystarczający do przeprowadzenia eksperymentów porównawczych w warunkach zadaniowych, zwłaszcza że został przygotowany specjalnie z myślą o detekcji przesyłek na taśmie transportowej.

[2] https://universe.roboflow.com/lukmik-e4qay/mailers-and-boxes-v10/analytics

Tabela 3.1: Podstawowe statystyki zbioru danych oraz podział na zbiory treningowy, walidacyjny i testowy.

Element	**Wartość**	**Komentarz**
Liczba obrazów	395	Całkowita liczebność zbioru.
Liczba adnotacji	1403	Wszystkie adnotacje *bounding box*.
Średnia liczba adnotacji na obraz	3,6	Wskazuje przeciętne zagęszczenie obiektów.
Null examples	20 (5,1%)	Obrazy bez adnotacji, przydatne jako negatywny kontekst.
Liczba klas	2	box, mailer.
Podział na zbiory (obrazy)	96% / 4% / 0%	Trening / walidacja / test.
Adnotacje box w splitach	640 / 90 / 3	Trening / walidacja / test.
Adnotacje mailer w splitach	559 / 107 / 4	Trening / walidacja / test.

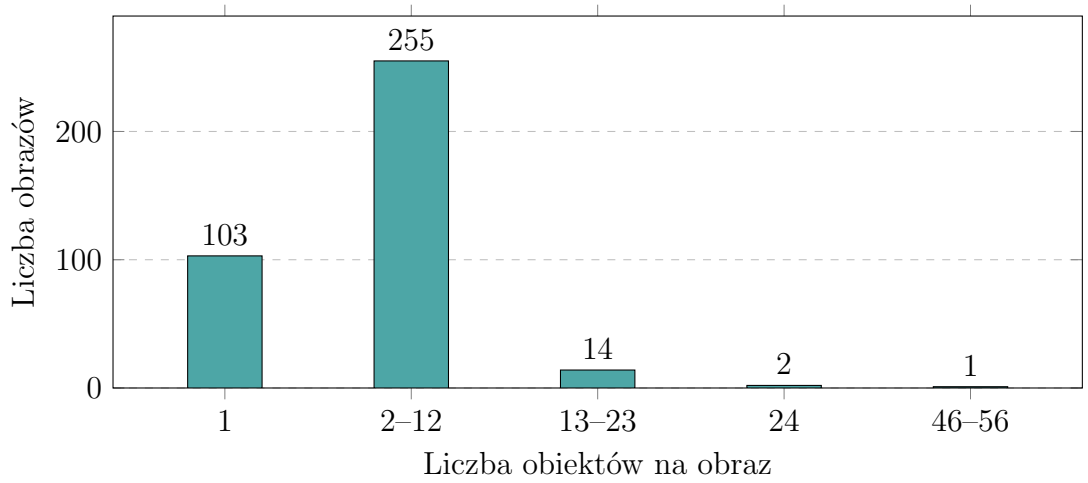
W zbiorze występują dwie klasy docelowe: *box* oraz *mailer*. Ich rozkład, przedstawiony na rysunku 3.6, jest zbliżony, co należy uznać za cechę korzystną z punktu widzenia treningu modeli detekcyjnych, ponieważ ogranicza ryzyko uprzywilejowania klasy większościowej i obniżenia skuteczności dla klasy rzadszej [7]. Istotną obserwacją jest również obecność 20 tzw. null examples, czyli obrazów bez adnotacji, stanowiących około 5,1% całego zbioru. Tego rodzaju próbki mogą poprawiać odporność modelu na fałszywe detekcje, ponieważ wprowadzają do procesu uczenia negatywny kontekst wizualny niezawierający przesyłek.



Rysunek 3.6: Rozkład adnotacji pomiędzy klasami box i mailer.

Na poziomie obrazów zastosowano podział 96% / 4% / 0% na zbiory treningowy, walidacyjny i testowy. Decyzja ta wynikała z ograniczonej liczebności całego korpusu i miała na celu maksymalizację liczby przykładów dostępnych podczas uczenia. W praktyce zbiory walidacyjny i testowy pełnią przede wszystkim funkcję kontrolną, a większą wartość interpretacyjną w dalszej części pracy mają wyniki walidacyjne oraz eksperymenty przeprowadzone na sekwencjach wideo.

Kolejną ważną cechą zbioru jest liczba obiektów oznaczonych na pojedynczym obrazie, której rozkład przedstawiono na rysunku 3.7. W zbiorze dominują sceny o małym i średnim zagęszczeniu obiektów, jednak występują również przypadki silnie zatłoczone. Z perspektywy analizowanego zastosowania jest to cecha pożądana, ponieważ system powinien działać poprawnie zarówno dla prostych scen z pojedynczymi przesyłkami, jak i dla bardziej złożonych ujęć, w których obiekty znajdują się blisko siebie lub częściowo się przesłaniają.



Rysunek 3.7: Rozkład liczby obiektów przypadających na pojedynczy obraz.

Charakterystykę geometryczną obrazów zestawiono w tabeli 3.2. Średni rozmiar obrazu wynosi 0,30 MP, przy zakresie od 0,01 MP do 12,19 MP, natomiast mediana rozmiaru obrazu to 641 × 442 piksele. Oznacza to znaczną niejednorodność materiału wejściowego, a więc sytuację, w której ten sam typ obiektu może zajmować bardzo różną liczbę pikseli w zależności od ujęcia [2].

Tabela 3.2: Kategorie rozmiaru i proporcji obrazów w analizowanym zbiorze danych.

Obszar	**Kategoria**	**Znaczenie**	**Liczba obrazów**
Rozmiar	small	maksymalny wymiar 224 px	19
Rozmiar	medium	maksymalny wymiar 512 px	107
Rozmiar	large	maksymalny wymiar 1024 px	190
Rozmiar	jumbo	maksymalny wymiar 8064 px	79
Proporcje	very tall	0,33:1	2
Proporcje	tall	0,5:1	53
Proporcje	square	1:1	38
Proporcje	wide	2:1	119
Proporcje	very wide	3:1	147
Proporcje	extra wide	4:1	36

W kategoriach rozmiaru dominują obrazy large i medium, natomiast obrazy small stanowią niewielki odsetek zbioru. Sugeruje to, że warunki bardzo niskiej rozdzielczości są reprezentowane słabiej niż obrazy średniej i wysokiej jakości. Pod względem proporcji boków przeważają obrazy poziome, zwłaszcza very wide oraz wide, co dobrze odpowiada geometrii scen rejestrowanych nad taśmą transportową.

Całościowo profil statystyczny analizowanego zbioru można ocenić jako korzystny z punktu widzenia celów niniejszej pracy. Zbiór jest wprawdzie niewielki, ale pozostaje dobrze zrównoważony klasowo, obejmuje zarówno sceny proste, jak i zatłoczone, a także charakteryzuje się znaczną zmiennością skali, rozdzielczości i proporcji obrazów. Jednocześnie interpretacja wyników powinna uwzględniać ograniczenia zbioru, przede wszystkim jego niewielką liczebność oraz bardzo mały zbiór testowy.

3.5. Augmentacja i preprocessing danych

Przygotowanie zbioru danych do treningu modeli detekcji obiektów nie ogranicza się wyłącznie do zgromadzenia i zaadnotowania materiału wizyjnego. Istotnym etapem jest również odpowiednie przetwarzanie wstępne obrazów (preprocessing) oraz rozszerzenie zbioru za pomocą technik augmentacji. Oba procesy pełnią odmienne, ale komplementarne funkcje: preprocessing służy ujednoliceniu właściwości obrazów wejściowych, natomiast augmentacja zwiększa różnorodność danych treningowych, co sprzyja poprawie zdolności uogólniania modelu i ogranicza ryzyko nadmiernego dopasowania do zbioru uczącego [44].

W niniejszej pracy operacje te zostały przeprowadzone z wykorzystaniem platformy Roboflow w ramach projektu obejmującego dwie klasy obiektów: mailer oraz box. Przyjęty potok przygotowania danych miał charakter celowo zachowawczy. Jego zadaniem nie było tworzenie silnie przekształconych, sztucznych wariantów obrazów, lecz standaryzacja danych oraz wprowadzenie jedynie takich modyfikacji, które odpowiadają realistycznym warunkom pracy systemu na taśmie transportowej.

Pierwszym etapem przygotowania danych było ręczne docięcie zgromadzonych zdjęć oraz zrzutów ekranu. Celem tego kroku było usunięcie zbędnych marginesów, elementów interfejsu oraz fragmentów tła, które nie należały do właściwej sceny obserwowanej przez system. Dzięki temu zwiększono udział obszaru istotnego z punktu widzenia detekcji przesyłek w końcowym obrazie oraz ograniczono wpływ szumu wizualnego, co ma znaczenie zwłaszcza w przypadku niewielkich zbiorów danych i obiektów zajmujących jedynie część kadru.

Po ręcznym przygotowaniu materiału zastosowano dwa kroki preprocessingu dostępne na platformie Roboflow. Pierwszym z nich było Auto-Orient. Operacja ta służy do ujednolicenia orientacji obrazów zapisanych przez różne urządzenia rejestrujące, które mogą przechowywać informację o obrocie zdjęcia wyłącznie w metadanych EXIF. Zastosowanie Auto-Orient pozwala uniknąć sytuacji, w której obraz byłby interpretowany w innej orientacji niż ta, dla której przygotowano adnotacje, co mogłoby prowadzić do niespójności między danymi wejściowymi a położeniem ramek ograniczających.

Drugim krokiem było przeskalowanie wszystkich obrazów do rozdzielczości 640×640 pikseli z użyciem metody Fit (black edges). Podejście to odpowiada technice letterboxingu,

czyli dopasowaniu obrazu do docelowego formatu przy zachowaniu oryginalnych proporcji i uzupełnieniu brakujących obszarów czarnym wypełnieniem. W analizowanym zadaniu było to rozwiązanie korzystniejsze niż proste rozciąganie obrazu, ponieważ ograniczało ryzyko deformacji geometrycznych przesyłek, a tym samym pozwalało zachować cechy istotne dla rozróżniania klas mailer i box.

Zastosowany preprocessing miał zatem dwa główne cele: zapewnienie spójnej orientacji danych wejściowych oraz ujednolicenie ich rozmiaru bez naruszania rzeczywistych proporcji obiektów. Jest to szczególnie ważne w przypadku przesyłek o zróżnicowanych kształtach, takich jak płaskie koperty foliowe, koperty bąbelkowe oraz pudełka o bardziej regularnej geometrii. Zachowanie proporcji obrazu można uznać za istotne z punktu widzenia poprawnej nauki cech wizualnych, ponieważ ogranicza ono ryzyko, że model będzie reagował na zniekształcenia wprowadzone przez preprocessing zamiast na rzeczywiste właściwości obiektów.

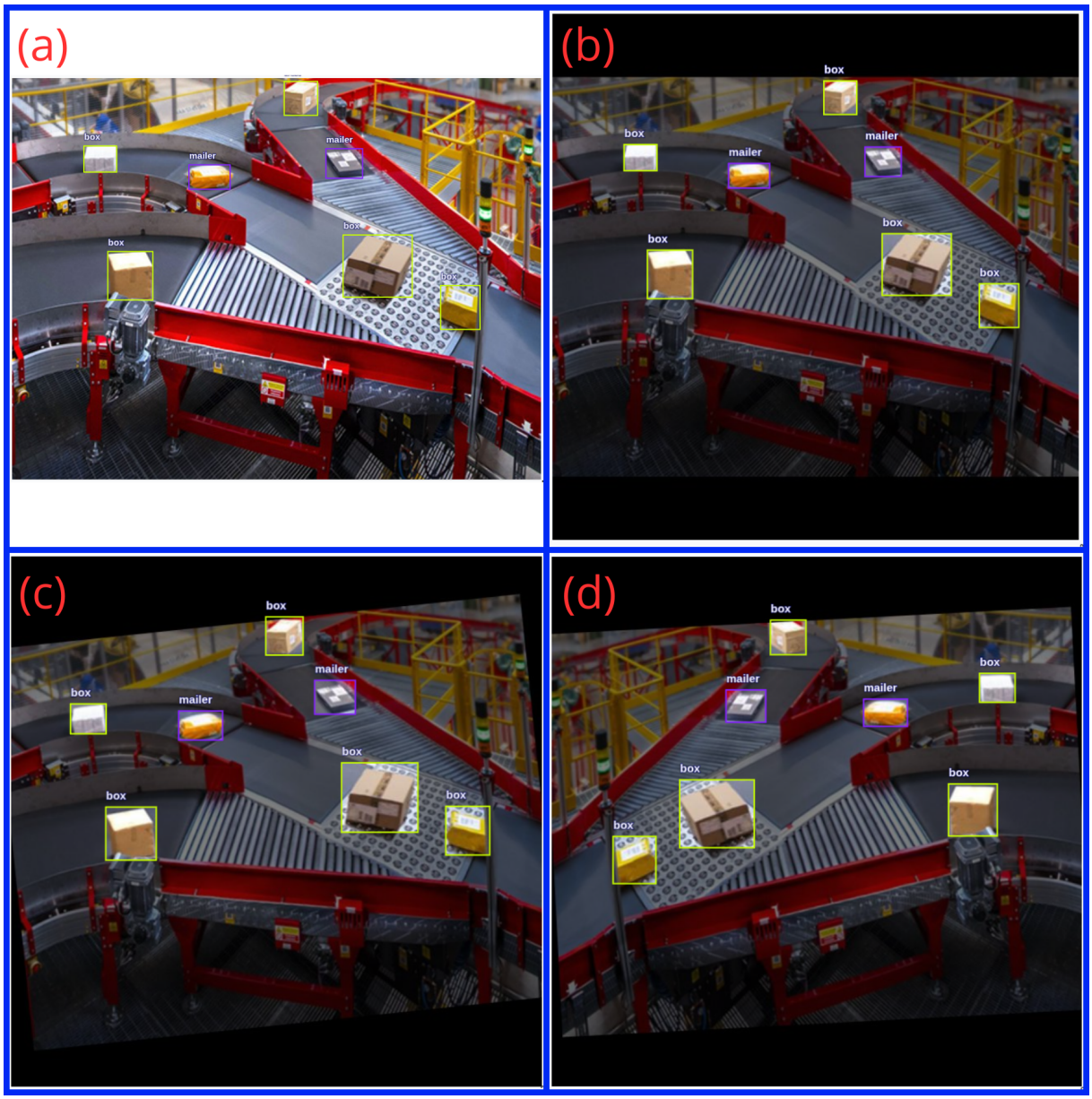
Równolegle z preprocessingiem zastosowano również augmentację danych. Jej celem było zwiększenie różnorodności zbioru treningowego bez konieczności pozyskiwania nowych obrazów. W konfiguracji Roboflow ustawiono generowanie trzech zaugmentowanych wariantów dla każdego przykładu treningowego, co zwiększało liczbę próbek wykorzystywanych podczas uczenia i poprawiało warunki generalizacji modeli na nowe sceny.

Pierwszą zastosowaną transformacją było poziome odbicie lustrzane (Horizontal Flip). W kontekście analizowanego problemu jest to augmentacja uzasadniona, ponieważ to, czy przesyłka znajduje się po lewej, czy po prawej stronie kadru, nie zmienia jej klasy semantycznej. Odbicie poziome ogranicza więc ryzyko, że model nauczy się przypadkowych zależności między klasą obiektu a układem sceny, kierunkiem ruchu na taśmie lub dominującą kompozycją obrazów w zbiorze treningowym [37].

Drugą techniką była operacja Crop, skonfigurowana w zakresie od 0% do 20% maksymalnego powiększenia. Transformacja ta symuluje umiarkowane zmiany kadru i skali obiektu, które mogą występować w praktyce w zależności od ułożenia przesyłki, jej położenia względem kamery oraz częściowej widoczności przy krawędziach obrazu. Zakres kadrowania został celowo ograniczony do wartości umiarkowanych, aby zwiększyć zróżnicowanie danych bez nadmiernego usuwania cech wizualnych potrzebnych do poprawnej detekcji, szczególnie w przypadku mniejszych i bardziej płaskich przesyłek.

Trzecią techniką była niewielka rotacja w zakresie od $-7°$ do $+7°$. Jej zadaniem było zwiększenie odporności modelu na drobne odchylenia orientacji paczek, które mogą pojawiać się na taśmie transportowej wskutek nieregularnego ułożenia obiektów, ich lekkiego obrotu w ruchu lub niewielkich różnic w ustawieniu kamery. Przyjęty zakres rotacji był świadomie konserwatywny, ponieważ zbyt duże obroty prowadziłyby do generowania mało realistycznych przykładów treningowych i mogłyby pogarszać spójność geometryczną pomiędzy obrazem a adnotacjami.

Przykładowy efekt zastosowanego preprocessingu oraz wybranych operacji augmentacyjnych przedstawiono na rysunku 3.8. Zaprezentowane warianty mają charakter ilustracyjny i pokazują, w jaki sposób pojedynczy obraz może zostać przekształcony w ramach przyjętego potoku przygotowania danych.



Rysunek 3.8: Przykład wpływu preprocessingu i augmentacji na pojedynczy obraz ze zbioru danych: (a) obraz oryginalny, (b) obraz po preprocessingu obejmującym standaryzację orientacji oraz przeskalowanie do rozdzielczości 640 × 640 pikseli z zachowaniem proporcji i czarnym wypełnieniem, (c) obraz po augmentacji w postaci niewielkiej rotacji, (d) obraz po połączonych transformacjach odbicia poziomego i rotacji.

Warto podkreślić również, że nie zastosowano bardziej agresywnych transformacji, takich jak duże rotacje, silne deformacje perspektywiczne czy złożone augmentacje kompozytowe. Taka decyzja była uzasadniona specyfiką badanego problemu, ponieważ w zadaniu detekcji przesyłek na taśmie transportowej bardziej pożądane jest zachowanie realizmu scen niż maksymalne zwiększanie wariancji danych za wszelką cenę. Przyjęta strategia opierała się więc na zasadzie, że augmentacja powinna poszerzać zbiór o warianty prawdopodobne z punktu widzenia rzeczywistej pracy systemu, a nie tworzyć przypadki mało reprezentatywne dla docelowego środowiska.

Podsumowując, zastosowany potok przygotowania danych został dobrany bezpośrednio do charakteru analizowanego zadania. Ręczne kadrowanie usuwało nieistotne fragmenty obrazu, Auto-Orient standaryzował orientację wejścia, a resize do 640 × 640 z czarnym wypełnieniem zapewniał jednolity format bez zniekształcania obiektów. Z kolei augmentacje w postaci odbicia poziomego, umiarkowanego cropu i niewielkiej rotacji wprowadzały kontrolowaną zmienność, która poprawiała odporność modeli na realistyczne różnice występujące w nagraniach z linii transportowej.

Rozdział 4

Implementacja systemu

Niniejszy rozdział przedstawia praktyczną realizację systemu opracowanego na potrzeby pracy i koncentruje się na tych elementach implementacyjnych, które stanowią podstawę późniejszych eksperymentów porównawczych. Kompletny kod źródłowy opisanej poniżej aplikacji demonstracyjnej jest dostępny w otwartym repozytorium GitHub (https://github.com/lukmiik/masters_thesis_parcels_object_detection_web_app). Zaimplementowane rozwiązanie ma postać aplikacji demonstracyjnej typu *proof-of-concept*, której głównym zadaniem jest detekcja, śledzenie oraz zliczanie dwóch klas przesyłek pocztowych — małych przesyłek typu mailer oraz większych paczek typu box — poruszających się po taśmie transportowej. Przyjęta forma implementacji nie odpowiada pełnemu systemowi przemysłowemu, lecz kontrolowanemu środowisku badawczemu, które umożliwia ocenę skuteczności różnych konfiguracji modeli detekcyjnych, metod śledzenia oraz środowisk wykonawczych przy zachowaniu tego samego schematu przetwarzania.

Architektura rozwiązania została zaprojektowana jako modularna platforma eksperymentalna, a nie pojedynczy, monolityczny skrypt. Takie podejście ułatwia porównywanie różnych rodzin detektorów, wariantów rozmiarowych modeli, konfiguracji trackerów oraz uruchomień na CPU i GPU bez konieczności przebudowy całej aplikacji. Z perspektywy inżynierii oprogramowania pozwala to zachować rozdział odpowiedzialności pomiędzy warstwą prezentacji, warstwą inferencji, modułem śledzenia, logiką zliczania oraz komponentami odpowiedzialnymi za zapis wyników i metryk.

Całość implementacji została oparta na języku Python, który pełni rolę wspólnego środowiska zarówno dla modułów inferencyjnych, jak i narzędzi pomocniczych wykorzystywanych podczas analizy eksperymentów. Warstwa interfejsu użytkownika została zrealizowana jako aplikacja webowa w frameworku Streamlit, dzięki czemu możliwe jest wygodne wybieranie nagrań wejściowych, modeli, trackerów, progów ufności oraz przeglądanie wyników bez konieczności ręcznej modyfikacji kodu. Jednocześnie logika obliczeniowa została odseparowana od interfejsu, co umożliwia ponowne wykorzystanie tych samych komponentów także poza aplikacją webową, na przykład w notatnikach badawczych używanych podczas treningu i analizy modeli.

4.1. Architektura systemu — schemat ogólny *pipeline*'u

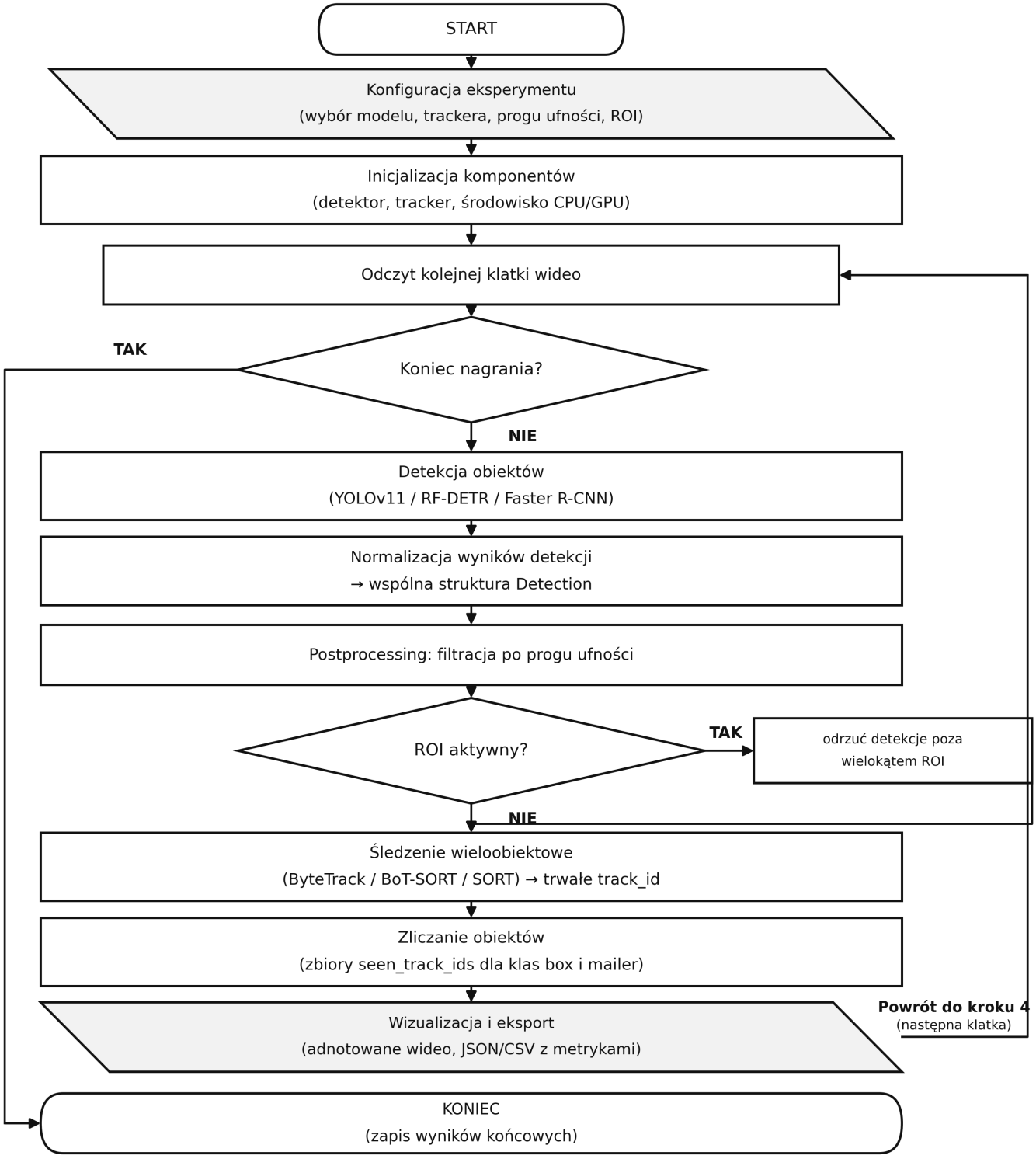
Ogólną architekturę systemu można opisać jako sekwencyjny *pipeline* przetwarzania wideo, który przekształca surowe nagranie wejściowe w dwa rodzaje rezultatów: adnotowany materiał wideo oraz zestaw metryk liczbowych służących do porównania kon-

figuracji eksperymentalnych. Centralnym założeniem tej architektury jest paradygmat *tracking-by-detection*, w którym detekcja obiektów i śledzenie wieloobiektowe są traktowane jako odrębne, wymienne komponenty połączone wspólnym formatem danych. Rozwiązanie to dobrze odpowiada charakterowi pracy badawczej, ponieważ pozwala osobno analizować wpływ detektora i trackera na końcową jakość zliczania.

Przepływ danych w systemie można przedstawić w postaci następujących etapów:

1. **Konfiguracja eksperymentu.** Użytkownik w interfejsie aplikacji wybiera nagranie wejściowe, zestaw model–tracker, próg ufności detekcji oraz opcjonalny obszar zainteresowania ROI zapisany dla danego materiału.
2. **Inicjalizacja komponentów.** Na podstawie wybranej konfiguracji system tworzy instancję odpowiedniego detektora oraz — jeśli dany scenariusz tego wymaga — instancję trackera, a także przypisuje środowisko wykonawcze CPU lub GPU.
3. **Odczyt strumienia wideo.** Moduł przetwarzania otwiera nagranie i iteruje po kolejnych klatkach, traktując każdą z nich jako podstawową jednostkę obliczeniową całego *pipeline*'u.
4. **Detekcja obiektów.** Dla każdej klatki aktywny model wyznacza ramki ograniczające, wartości ufności oraz etykiety klas, przy czym system obsługuje architektury reprezentujące różne rodziny współczesnych detektorów: YOLOv11, RF-DETR oraz Faster R-CNN.
5. **Normalizacja wyników detekcji.** Niezależnie od wewnętrznego formatu wyjściowego konkretnej biblioteki, predykcje są przekształcane do wspólnej struktury danych zawierającej współrzędne ramek, wartości confidence, identyfikatory klas oraz nazwy klas, dzięki czemu dalsze etapy *pipeline*'u pozostają niezależne od konkretnego modelu.
6. **Postprocessing detekcji.** Przed przekazaniem wyników do modułu śledzenia wykonywana jest filtracja po progu ufności, a następnie — jeśli aktywowano ROI — odrzucane są detekcje znajdujące się poza zdefiniowanym wielokątem obszaru roboczego taśmy.
7. **Śledzenie wieloobiektowe.** Odfiltrowane detekcje są przekazywane do trackera, który przypisuje obiektom trwałe identyfikatory i wiąże obserwacje z kolejnych klatek w spójne trajektorie; w zależności od konfiguracji wykorzystywane są algorytmy ByteTrack, BoT-SORT albo SORT.
8. **Zliczanie obiektów.** Logika zliczania rozróżnia dwa typy wyników: bieżącą liczbę obiektów widocznych w aktualnej klatce oraz łączną liczbę unikalnych przesyłek zaobserwowanych w całym nagraniu, wyznaczaną na podstawie zbiorów zapamiętanych identyfikatorów śledzenia.
9. **Wizualizacja i eksport wyników.** Na końcu *pipeline*'u system rysuje adnotacje na klatce, zapisuje wideo wynikowe oraz agreguje metryki, takie jak czas inferencji, FPS i statystyki predykcji do plików JSON i CSV wykorzystywanych później w analizie eksperymentalnej.

Ogólną architekturę systemu oraz zależności pomiędzy jego głównymi komponentami zilustrowano na rysunku 4.1.



Rysunek 4.1: Schemat blokowy pipeline'u przetwarzania wideo systemu detekcji i zliczania przesyłek pocztowych.

Sercem wykonawczym systemu jest moduł odpowiedzialny za przetwarzanie wideo, który scala wszystkie opisane etapy w jedną, wysokopoziomową pętlę operującą na kolejnych klatkach nagrania. Wokół tego modułu rozmieszczono wyspecjalizowane kompo-

nenty odpowiedzialne odpowiednio za inferencję modeli, śledzenie obiektów, obsługę ROI, eksport metryk i prezentację wyników w interfejsie użytkownika. Taki podział nie tylko porządkuje kod źródłowy, ale również zmniejsza koszt rozbudowy systemu, ponieważ dodanie nowego modelu detekcyjnego lub nowego trackera wymaga implementacji wyłącznie odpowiedniego adaptera, bez ingerencji w pozostałe warstwy aplikacji.

Istotnym elementem architektury jest także rozdzielenie warstwy wizualnej od warstwy analitycznej. Aplikacja webowa pełni funkcję orkiestratora eksperymentów i panelu inspekcyjnego, natomiast właściwe obliczenia wykonywane są w modułach warstwy logiki biznesowej, które mogą być wywoływane niezależnie od interfejsu graficznego. Dzięki temu system łączy dwie role: jest jednocześnie demonstratorem działania opracowanej metody oraz narzędziem badawczym umożliwiającym systematyczne porównania modeli, trackerów i środowisk uruchomieniowych.

Z punktu widzenia celu pracy najważniejszą cechą tej architektury jest powtarzalność eksperymentów. Ten sam schemat przetwarzania obowiązuje niezależnie od tego, czy analizowana jest różnica pomiędzy rodzinami detektorów, wariantami rozmiarowymi modeli, konfiguracjami trackerów czy środowiskami CPU i GPU.

4.2. Modele detekcji — konfiguracja i trening

W zaimplementowanym systemie trzy analizowane modele detekcji — YOLOv11, RF-DETR oraz Faster R-CNN — zostały zintegrowane jako wymienne warianty tej samej warstwy detekcyjnej. Oznacza to, że z perspektywy całej aplikacji pełnią one tę samą funkcję, natomiast różnią się architekturą, sposobem treningu, wymaganym formatem danych oraz charakterystyką inferencji.

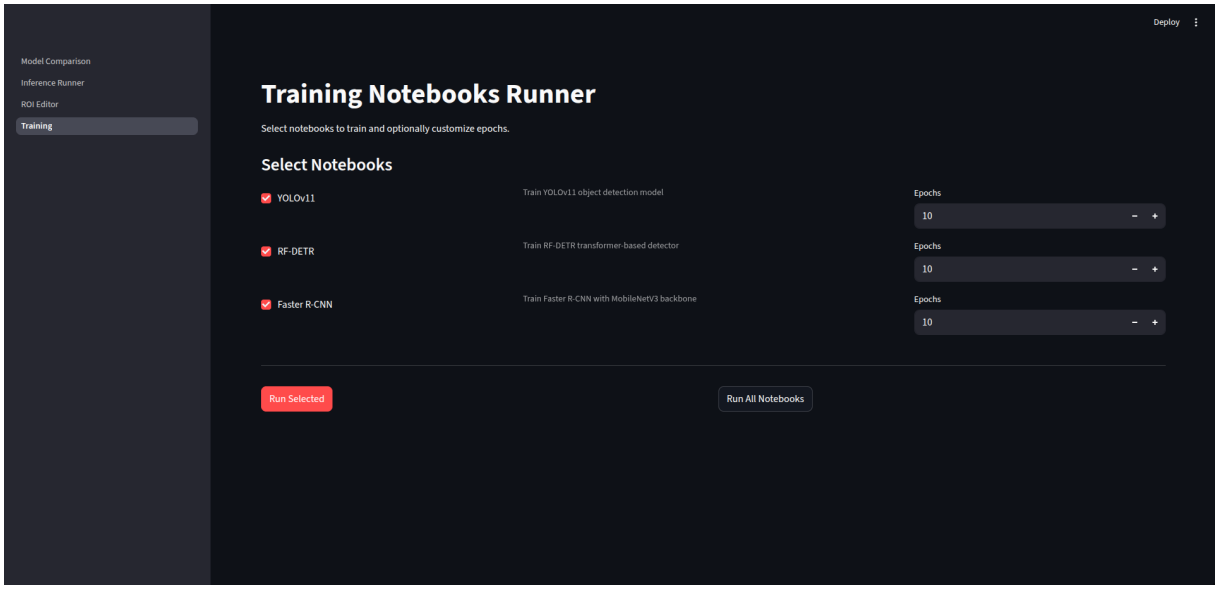
Istotną cechą rozwiązania jest to, że logika treningu została przygotowana w dedykowanych notatnikach Jupyter, ale sam proces uczenia nie jest odseparowany od aplikacji webowej. Przeciwnie, notatniki mogą być uruchamiane bezpośrednio z poziomu aplikacji Streamlit, w zakładce Training, dzięki czemu interfejs webowy pełni rolę warstwy sterującej eksperymentami, natomiast notebooki stanowią ich zaplecze implementacyjne. Interfejs zakładki Training, pełniącej rolę warstwy sterującej uruchamianiem notatników treningowych dla analizowanych modeli detekcji, zilustrowano na rysunku 4.2.

Takie podejście wynika z faktu, że porównywane detektory należą do różnych ekosystemów programistycznych i wymagają odmiennych narzędzi treningowych. YOLOv11 korzysta z frameworka Ultralytics i danych w formacie YOLO, natomiast RF-DETR oraz Faster R-CNN wykorzystują inne biblioteki i są trenowane na danych zapisanych w formacie COCO. Z tego względu zastosowanie notebooków jako warstwy wykonawczej upraszcza obsługę modelowo-specyficznych zależności, a jednocześnie pozwala zachować jednolity punkt uruchamiania eksperymentów z poziomu aplikacji.

Z punktu widzenia architektury systemu ważne było również ujednolicenie sposobu wykorzystania wytrenowanych modeli podczas inferencji. Niezależnie od użytej architektury, końcowe *checkpoints* są przypisywane do odpowiednich konfiguracji aplikacji, a następnie wykorzystywane w tym samym *pipeline* przetwarzania obrazu. Dzięki temu możliwe jest

bezpośrednie porównywanie modeli w identycznym scenariuszu badawczym, bez konieczności przebudowy dalszych etapów systemu.

Ważną rolę odgrywa także organizacja artefaktów eksperymentalnych. Wyniki treningu, takie jak zapisane wagi, pliki konfiguracyjne, logi oraz metryki ewaluacyjne, są przechowywane w uporządkowanej strukturze katalogów, co ułatwia odtwarzanie eksperymentów i późniejsze uzasadnienie wyboru konkretnego modelu do badań porównawczych. Zapewnia to reprodukowalność eksperymentów, co jest szczególnie istotne przy porównaniach obejmujących różne architektury, ich warianty rozmiarowe oraz środowiska wykonawcze..



Rysunek 4.2: Widok zakładki *Training* w aplikacji webowej — panel uruchamiania notatników treningowych dla modeli YOLOv11, RF-DETR oraz Faster R-CNN.

4.2.1. YOLOv11

W niniejszej pracy YOLOv11 pełni rolę podstawowego detektora jednoetapowego oraz głównego punktu odniesienia dla porównań z RF-DETR i Faster R-CNN. Model ten został wybrany ze względu na korzystny kompromis pomiędzy szybkością działania a jakością detekcji, a także ze względu na jego praktyczną przydatność w zastosowaniach zbliżonych do czasu rzeczywistego.

Trening modelu został zrealizowany z wykorzystaniem transfer learningu. W praktyce oznacza to rozpoczęcie uczenia od wcześniej wytrenowanych wag YOLOv11, a następnie dostrojenie modelu do właściwego dla tej pracy zadania detekcji dwóch klas przesyłek: box oraz mailer.

Konfigurację zbioru danych zdefiniowano w pliku `datasets/yolov11/data.yaml`, określającym ścieżki do podzbiorów oraz mapowanie klas. Dane przygotowano w formacie zgodnym z wymaganiami frameworka Ultralytics, co umożliwiło ich bezpośrednie użycie w procesie treningowym. Kwestie preprocessingu i augmentacji przedstawiono wcześniej,

dlatego w niniejszym podrozdziale ograniczono się do aspektów bezpośrednio związanych z konfiguracją uczenia.

Kod treningowy YOLOv11 znajduje się w dedykowanym notatniku Jupyter, jednak jego uruchomienie może odbywać się z poziomu aplikacji webowej. Oznacza to, że użytkownik nie musi ręcznie otwierać notebooka, aby rozpocząć eksperyment, lecz może wywołać proces treningowy bezpośrednio z interfejsu Streamlit w zakładce Training. Z perspektywy implementacyjnej aplikacja stanowi więc warstwę sterującą, a notebook realizuje właściwe operacje związane z konfiguracją, pobraniem danych, treningiem oraz zapisem wyników.

Sam proces uczenia wykorzystuje mechanizmy udostępniane przez framework Ultralytics i może być parametryzowany m.in. liczbą epok, rozmiarem obrazu wejściowego, wielkością batcha czy wyborem checkpointu początkowego [43]. W głównym przebiegu eksperymentalnym zastosowano konfigurację obejmującą 12 epok, *batch size* równy 16, rozmiar obrazu 640, włączone AMP oraz deterministyczne ustawienie ziarna losowości. Taki sposób parametryzacji pozwala zachować powtarzalność eksperymentów i ułatwia późniejsze porównanie wyników z innymi modelami.

Istotną zaletą *pipeline*'u YOLOv11 jest pełna ścieżka zapisu artefaktów treningowych. Obejmuje ona nie tylko końcowe *checkpoints*, lecz również pliki argumentów, zestawienia metryk, krzywe uczenia oraz macierze pomyłek, co umożliwia analizę jakości modelu po zakończeniu treningu. W dalszej części systemu do inferencji wykorzystywany jest checkpoint `best.pt`, czyli model odpowiadający najlepszemu wynikowi walidacyjnemu.

Integracja modelu z warstwą inferencji została zrealizowana przez odpowiedni adapter, który ładuje wytrenowane wagi, dobiera urządzenie wykonawcze i wykonuje predykcję dla kolejnych klatek wideo. Otrzymane wyniki są następnie mapowane do wspólnej struktury detekcji wykorzystywanej przez pozostałe moduły systemu, dzięki czemu dalsze etapy przetwarzania nie zależą od konkretnej biblioteki detekcyjnej.

YOLOv11 wyróżnia się w projekcie również tym, że może współpracować z natywnym mechanizmem śledzenia obiektów dostępnym w ekosystemie Ultralytics. Dzięki temu algorytmy, takie jak ByteTrack i BoT-SORT mogą być uruchamiane bezpośrednio na bazie tego modelu, bez potrzeby budowania dodatkowej zewnętrznej warstwy pośredniczącej. Z praktycznego punktu widzenia czyni to YOLOv11 nie tylko szybkim detektorem, ale również wygodnym punktem integracji detekcji i trackingu w eksperymentach związanych ze zliczaniem przesyłek na taśmie transportowej.

4.2.2. RF-DETR

W opracowanym systemie RF-DETR został zintegrowany jako jeden z wymiennych modeli detekcji obiektów wykorzystywanych we wspólnej warstwie detekcyjnej całego *pipeline*'u analizy przesyłek. Dzięki temu model ten nie funkcjonuje jako osobna gałąź eksperymentalna, lecz jako równorzędna alternatywa dla YOLOv11 oraz Faster R-CNN, co umożliwia ich porównywanie w identycznych warunkach przetwarzania. Z perspektywy architektury systemu ma to istotne znaczenie, ponieważ dalsze etapy przetwarzania, takie jak filtrowanie detekcji, ograniczenie do obszaru zainteresowania, śledzenie obiektów, zliczanie, wizualizacja oraz agregacja metryk, operują na zunifikowanej reprezentacji detekcji i pozostają niezależne od wewnętrznej implementacji konkretnego modelu.

RF-DETR pełni w niniejszej pracy rolę reprezentanta nowoczesnych, transformatorowych detektorów obiektów typu *end-to-end*, a jego obecność poszerza analizę o paradygmat odmienny zarówno od jednoetapowego YOLOv11, jak i dwuetapowego Faster R-CNN. Z punktu widzenia eksperymentów jest to istotne, ponieważ pozwala ocenić nie tylko klasyczne architektury oparte na sieciach konwolucyjnych, ale również współczesne podejście transformatorowe projektowane z myślą o korzystnym kompromisie pomiędzy skutecznością detekcji a opóźnieniem inferencji.

W warstwie implementacyjnej RF-DETR działa w tej samej pętli przetwarzania klatka po klatce, która została zastosowana dla całej aplikacji. Każda klatka wejściowa przechodzi przez sekwencję tych samych etapów: detekcję, filtrowanie po progu ufności, opcjonalne ograniczenie do obszaru ROI, śledzenie, zliczanie oraz końcową wizualizację i zapis wyników. Surowe wyjścia modelu są następnie mapowane do wspólnej struktury Detection używanej w projekcie, dzięki czemu RF-DETR może zostać włączony do *pipeline*'u bez konieczności modyfikowania dalszej logiki systemu. Taka abstrakcja jest szczególnie cenna w eksperymentach porównawczych, ponieważ umożliwia wymianę detektora przy zachowaniu niezmienionej reszty łańcucha przetwarzania.

Proces treningu RF-DETR został wydzielony do dedykowanego notatnika badawczego, natomiast aplikacja webowa pełni rolę warstwy orkiestrującej uruchamianie eksperymentów. Takie rozdzielenie odpowiedzialności upraszcza architekturę rozwiązania: interfejs użytkownika odpowiada za konfigurację eksperymentu, a logika specyficzna dla modelu pozostaje zamknięta w osobnym środowisku treningowym. Podejście to poprawia również powtarzalność badań, ponieważ ta sama procedura może być wielokrotnie uruchamiana dla kontrolowanych wartości parametrów, zwłaszcza liczby epok.

Istotną różnicą względem YOLOv11 jest wymagany format danych. RF-DETR oczekuje zbioru danych zapisanego w formacie COCO, z podziałem na katalogi train, valid i test, z których każdy zawiera własny plik `_annotations.coco.json`. Zastosowanie tego formatu jest zgodne z dokumentacją biblioteki rfdetr i stanowi naturalne środowisko pracy dla treningu modeli RF-DETR na danych niestandardowych.

W projekcie wykorzystano wariant RF-DETR Nano, inicjalizowany z wag wstępnie wytrenowanych, a następnie dostrajany do dwuklasowego zadania detekcji przesyłek. Sama procedura treningowa korzysta z wysokopoziomowego interfejsu biblioteki rfdetr, który wspiera trening na własnym zbiorze danych, zapis artefaktów do katalogu wyjściowego, wczesne zatrzymywanie oraz automatyczny wybór najlepszego checkpointu. W przyjętej konfiguracji zdefiniowano między innymi liczbę epok, *batch size* równy 8, gradient accumulation z parametrem `grad_accum_steps = 2` oraz *early stopping* z `patience = 5`, co pozwala utrzymać rozsądne wymagania obliczeniowe przy jednoczesnym zachowaniu warunków sprzyjających zbieżności modelu.

Dokumentacja RF-DETR wskazuje również, że łączenie *batch size* z `grad_accum_steps` pozwala utrzymać docelowy efektywny rozmiar partii nawet na kartach GPU o ograniczonej pamięci, a `checkpoint_best_total.pth` jest końcowym checkpointem wybieranym automatycznie do inferencji i benchmarków. Z tego względu rozwiązanie to dobrze wpisuje się w aplikacyjny charakter pracy, w której istotna jest nie tylko skuteczność modelu, ale

również możliwość powtarzalnego uruchamiania eksperymentów w praktycznym środowisku badawczym [34].

Na etapie inferencji RF-DETR został opakowany dedykowanym adapterem, którego zadaniem jest załadowanie wytrenowanego checkpointu, przygotowanie modelu do predykcji oraz konwersja surowych wyników do wspólnego formatu używanego przez resztę systemu. Istotnym szczegółem implementacyjnym jest odtworzenie liczby klas na podstawie metadanych checkpointu, a następnie ponowna inicjalizacja głowicy detekcyjnej tak, aby odpowiadała definicji klas używanej w zbiorze danych. Ogranicza to ryzyko rozbieżności pomiędzy strukturą wytrenowanego modelu a konfiguracją zadania wykorzystywanego podczas dalszych eksperymentów.

Podczas predykcji dla kolejnych klatek obraz jest najpierw konwertowany z przestrzeni barw BGR do RGB, a następnie przekazywany do procedury inferencyjnej wraz z aktualnie ustawionym progiem ufności. Wyniki modelu są dalej transformowane do postaci ramek ograniczających, wartości confidence oraz identyfikatorów klas, które są zgodne z formatem wymaganym przez pozostałe moduły aplikacji. W praktyce umożliwia to pełną wymienność modelu względem innych detektorów obsługiwanych w systemie.

W implementacji RF-DETR istotne znaczenie ma również normalizacja indeksów klas. Po predykcji identyfikatory klas są korygowane tak, aby odpowiadały kolejności etykiet używanych w aplikacji, tj. [`box`, `mailer`], a wartości niepoprawne są odrzucane. Choć jest to pozornie niewielki detal implementacyjny, jego poprawna realizacja warunkuje spójność całego *pipeline*'u, ponieważ błędne mapowanie klas prowadziłoby bezpośrednio do niepoprawnej wizualizacji oraz zafałszowania końcowych statystyk zliczania.

RF-DETR został również zintegrowany z warstwą śledzenia i zliczania obiektów. W tej konfiguracji model może współpracować z trackerami ByteTrack oraz SORT za pośrednictwem adaptera ujednolicającego komunikację pomiędzy wynikami detekcji a bibliotekami śledzenia. Dzięki temu detekcje RF-DETR są zamieniane na trajektorie posiadające trwałe identyfikatory `track_id`, które następnie służą do zliczania unikalnych przesyłek w materiale wideo. Sama logika zliczania nie opiera się na pojedynczych detekcjach klatkowych, lecz na zbiorach identyfikatorów już zaobserwowanych obiektów, utrzymywanych oddzielnie dla każdej klasy, co zmniejsza ryzyko wielokrotnego zliczania tej samej przesyłki.

Dodatkowo przed etapem śledzenia może zostać zastosowany opcjonalny, wielokątny obszar ROI, ograniczający analizę do właściwej przestrzeni roboczej taśmy transportowej. Rozwiązanie to pozwala zredukować wpływ detekcji pochodzących z obszarów nieistotnych dla eksperymentu, a tym samym poprawia stabilność późniejszego śledzenia i końcowego zliczania. Z praktycznego punktu widzenia RF-DETR jest więc oceniany nie jako izolowany model detekcyjny, lecz jako pełnoprawny komponent kompletnego systemu analizy przesyłek.

Integracja RF-DETR w aplikacji spełnia dwa zasadnicze cele badawcze. Po pierwsze, umożliwia jego bezpośrednie porównanie z YOLOv11 oraz Faster R-CNN w tym samym interfejsie użytkownika, przy tym samym *pipeline*'ie przetwarzania oraz w tym samym scenariuszu ewaluacyjnym. Po drugie, pozwala ocenić, czy detektor transformatorowy wnosi praktyczne korzyści do zadania analizy przesyłek wtedy, gdy współpracuje z warstwą śledzenia i zliczania, a nie wyłącznie na poziomie odseparowanych metryk detekcji.

4.2.3. Faster R-CNN

W zaimplementowanym systemie Faster R-CNN pełni rolę klasycznego detektora dwuetapowego, stanowiącego punkt odniesienia dla modeli YOLOv11 oraz RF-DETR. Jego uwzględnienie w badaniu pozwala porównać nowocześniejsze, szybsze architektury z rozwiązaniem znanym z wysokiej jakości lokalizacji obiektów. W niniejszej pracy model ten nie jest traktowany jako odrębny prototyp badawczy, lecz jako pełnoprawny element tego samego *pipeline*'u inferencyjnego co pozostałe detektory, dzięki czemu wszystkie architektury są oceniane w identycznych warunkach wejścia, filtrowania predykcji i dalszego przetwarzania.

W aplikacji wykorzystano wariant `fasterrcnn_mobilenet_v3_large_fpn` dostępny w bibliotece Torchvision [29]. Wariant ten łączy mechanizm detekcji Faster R-CNN z lekkim *backbone*'em MobileNetV3-Large oraz siecią Feature Pyramid Network, co jest korzystne w zadaniu obejmującym jednoczesną detekcję małych przesyłek typu mailer oraz większych paczek pudełkowych. Jest to szczególnie istotne w zastosowaniach logistycznych, w których w jednej scenie współwystępują obiekty o wyraźnie zróżnicowanych gabarytach i cechach wizualnych. Podczas inicjalizacji modelu liczba klas jest odczytywana z pliku `training_outputs/faster_r_cnn/training_config.json`, a następnie wykorzystywana do odtworzenia architektury zgodnej z checkpointem zapisanym po treningu. Dzięki temu moduł inferencyjny pozostaje spójny z konfiguracją używaną na etapie uczenia.

Obsługa modelu została zamknięta w klasie `FasterRCNNDetector` zdefiniowanej w pliku `src/inference.py`, natomiast wybór konkretnego detektora odbywa się przez wspólną funkcję fabrykującą `get_detector(model_type, dataset, device, confidence)`. Rozwiązanie to upraszcza integrację Faster R-CNN z resztą aplikacji, ponieważ kolejne moduły systemu nie muszą rozróżniać, z jakiej architektury pochodzą aktualne predykcje. Sam proces treningu został zaimplementowany w notatniku `training_notebooks/train_faster_rcnn.ipynb`, który może być uruchamiany z poziomu zakładki Training przy użyciu mechanizmu parametryzacji notebooków. Taki sposób organizacji eksperymentów odpowiada podejściu stosowanemu przy dostosowywaniu modeli detekcyjnych Torchvision do własnych zbiorów danych i ułatwia powtarzalne odtwarzanie wyników.

Na etapie inferencji każda klatka obrazu jest najpierw konwertowana z przestrzeni BGR do RGB, a następnie przekształcana do tensora wejściowego. Model zwraca słownik zawierający ramki ograniczające, wartości confidence oraz identyfikatory klas, które po filtracji według zadanego progu ufności są poddawane normalizacji. Istotnym krokiem implementacyjnym jest korekta indeksów klas wynikająca z faktu, że w implementacji Torchvision indeks 0 odpowiada klasie tła; po odjęciu tego przesunięcia etykiety są mapowane na kolejność klas używaną w aplikacji, tj. [`box`, `mailer`].

Po zakończeniu detekcji wyniki są zamieniane na wspólny obiekt `Detection`, zawierający współrzędne ramek, wyniki ufności, identyfikatory klas, nazwy klas oraz opcjonalnie identyfikatory śledzenia. Następnie mogą zostać przefiltrowane względem zadanego obszaru ROI, a potem przekazane do warstwy śledzenia i zliczania obiektów. W praktyce oznacza to, że Faster R-CNN nie funkcjonuje w pracy wyłącznie jako model opisany w literaturze, lecz jako w pełni zintegrowany komponent eksperymentalnego systemu ana-

lizy przesyłek, którego działanie można oceniać zarówno pod kątem jakości detekcji, jak i wpływu na końcowe wyniki śledzenia, zliczania oraz wydajności przetwarzania.

4.3. Algorytmy śledzenia obiektów i zliczania

Warstwa śledzenia i zliczania obiektów stanowi końcowy etap przetwarzania wideo w zaimplementowanym systemie. Jest ona wykonywana po detekcji obiektów oraz po wstępnej filtracji predykcji, a przed wizualizacją wyników i eksportem metryk. Przyjęte rozwiązanie odpowiada paradygmatowi *tracking-by-detection*, w którym detektor generuje obserwacje dla kolejnych klatek, natomiast tracker odpowiada za ich powiązanie w czasie i nadanie obiektom trwałych identyfikatorów.

Z perspektywy implementacyjnej przetwarzanie odbywa się klatka po klatce, a wyniki wszystkich wspieranych modeli detekcji są najpierw sprowadzane do wspólnej reprezentacji wewnętrznej `Detection`. Struktura ta przechowuje współrzędne ramki ograniczającej, wartość confidence, identyfikator klasy, nazwę klasy oraz opcjonalnie identyfikator śledzenia. Taka normalizacja oddziela warstwę detekcji od warstwy śledzenia, dzięki czemu dalsze etapy *pipeline*'u pozostają niezależne od konkretnej architektury modelu i mogą działać identycznie dla YOLOv11, RF-DETR oraz Faster R-CNN.

Przed przekazaniem detekcji do trackera stosowana jest filtracja mająca na celu zwiększenie stabilności dalszego śledzenia i ograniczenie błędów zliczania. W pierwszej kolejności odrzucane są predykcje o zbyt niskiej pewności, zgodnie z zadanym progiem confidence. Następnie, jeżeli dla danego nagrania zdefiniowano obszar ROI, uwzględniane są wyłącznie te detekcje, których środek ramki znajduje się wewnątrz wielokąta opisującego roboczy obszar taśmy. W praktyce ogranicza to wpływ tła i redukuje liczbę krótkotrwałych, niestabilnych trajektorii inicjalizowanych poza właściwym strumieniem przesyłek.

Powiązanie modeli detekcyjnych z algorytmami śledzenia zostało zrealizowane jawnie na poziomie konfiguracji systemu. Dla modelu YOLOv11 wykorzystywane są algorytmy ByteTrack oraz BoT-SORT, natomiast dla modeli RF-DETR i Faster R-CNN dostępne są konfiguracje z ByteTrack oraz SORT. Sam wybór tych metod nie jest przypadkowy: SORT stanowi lekkie i klasyczne rozwiązanie referencyjne, ByteTrack poprawia ciągłość śledzenia dzięki wykorzystaniu także słabszych detekcji, natomiast BoT-SORT reprezentuje bardziej rozbudowane podejście do asocjacji i utrzymywania tożsamości obiektów.

W systemie zastosowano dwa sposoby integracji warstwy śledzenia z detektorem. W przypadku YOLOv11 używany jest adapter natywny, który korzysta bezpośrednio z mechanizmu `track()` udostępnianego przez ekosystem Ultralytics. Oznacza to, że detekcja i śledzenie są wykonywane w jednym, spójnym przebiegu, bez konieczności osobnego wywołania zewnętrznego trackera dla tej samej klatki. Z kolei dla modeli RF-DETR i Faster R-CNN wykorzystywany jest adapter pośredniczący, który najpierw pobiera gotowe detekcje, następnie konwertuje je do formatu akceptowanego przez zewnętrzną bibliotekę śledzenia, a po zakończeniu asocjacji odwzorowuje wyniki z powrotem do wspólnej struktury `Detection`. W takim wariancie tracker działa jako warstwa nałożona na wyjście detektora, dlatego jakość tworzonych trajektorii jest szczególnie silnie zależna od stabilności predykcji między kolejnymi klatkami.

Zliczanie obiektów w aplikacji nie jest realizowane na podstawie przecięcia wirtualnej linii ani wejścia do zdefiniowanej strefy, lecz bezpośrednio na podstawie trwałości identyfikatorów nadawanych przez tracker. Dla każdej z dwóch klas, tj. `box` oraz `mailer`, utrzymywane są osobne zbiory wcześniej zaobserwowanych identyfikatorów śledzenia. Jeżeli w danej klatce tracker zwróci poprawny `track_id`, zostaje on dopisany do zbioru przypisanego do odpowiedniej klasy. Dzięki temu system raportuje równolegle dwie wielkości: bieżącą liczbę obiektów widocznych w aktualnej klatce po filtracji oraz łączną liczbę unikalnych przesyłek zaobserwowanych od początku analizy nagrania.

Takie podejście jest proste obliczeniowo i dobrze pasuje do scenariusza taśmy transportowej, gdzie obiekty poruszają się w stosunkowo uporządkowany sposób. Jednocześnie poprawność wyniku końcowego zależy bezpośrednio od jakości śledzenia. Jeżeli tracker zachowuje spójny identyfikator dla tej samej przesyłki, obiekt zostaje doliczony tylko raz. Jeżeli jednak dojdzie do fragmentacji trajektorii lub przełączenia identyfikatora, ta sama fizyczna przesyłka może zostać policzona wielokrotnie. Z tego względu dokładność zliczania należy interpretować nie jako własność samej logiki licznika, lecz jako rezultat działania całego łańcucha detekcja–śledzenie.

Istotnym elementem implementacji jest również zapisywanie artefaktów inferencji, które później stanowią podstawę analizy eksperymentalnej. System generuje adnotowane nagrania wyjściowe z narysowanymi ramkami i identyfikatorami trajektorii, a także pliki metryk zawierające między innymi statystyki klatkowe, liczności klas, średni czas inferencji oraz FPS. Umożliwia to jednoczesną ocenę jakościową i ilościową poszczególnych konfiguracji model–tracker, a także ich porównanie w dalszej części pracy, zwłaszcza w eksperymentach dotyczących poprawności zliczania oraz różnic między środowiskami CPU i GPU.

4.3.1. ByteTrack i BoT-SORT dla YOLOv11

W zaimplementowanym systemie model YOLOv11 wyróżnia się natywną integracją z algorytmami śledzenia w ekosystemie Ultralytics. W odróżnieniu od RF-DETR i Faster R-CNN, które wymagają zewnętrznej warstwy śledzenia, YOLOv11 umożliwia detekcję i śledzenie w jednym kroku, co upraszcza *pipeline* i pozwala porównywać strategie śledzenia przy tym samym detektorze.

Integrację zrealizowano poprzez klasę `YoloNativeTrackerAdapter` w module `src/tracking.py`. Adapter ten, z flagą `requires_detection` ustawioną na `False`, wywołuje metodę `model.track()` z plikiem konfiguracyjnym `bytetrack.yaml` lub `botsort.yaml` oraz parametrem `persist=True` zapewniającym ciągłość stanu trackera między klatkami. Wyniki są parsowane do struktury `Detection`, gdzie `track_id` pozwala modułowi zliczania odróżnić nowe obiekty od wcześniej zaobserwowanych.

Filtracja przestrzenna ROI jest stosowana w fazie postprocessingu wyników trackera, co wynika z natywnej integracji modelu z ekosystemem Ultralytics — metoda model.track zwraca ujednolicony wynik obejmujący jednocześnie detekcje i identyfikatory śledzenia. Takie podejście zapobiega fragmentacji trajektorii, gdy obiekt chwilowo wychodzi poza ROI. System utrzymuje zbiory `seen_track_ids` dla klas `box` i `mailer`, a rozmiar tych

zbiorów determinuje końcowy wynik liczbowy — każda zmiana tożsamości bezpośrednio zwiększa liczbę zliczonych przesyłek.

Algorytm ByteTrack wykorzystuje przy asocjacji zarówno detekcje o wysokiej, jak i niskiej ufności. Właściwość ta jest istotna dla małych przesyłek typu `mailer`, które mogą być częściowo zasłonięte lub generować niestabilne predykcje — zamiast odrzucać słabsze detekcje, ByteTrack podtrzymuje ich trajektorie, ograniczając wielokrotne zliczanie tej samej przesyłki. BoT-SORT oferuje bardziej zaawansowaną asocjację, łączącą informacje o ruchu z dodatkowymi wskazówkami, co zwiększa odporność na przełączenia identyfikatorów w gęstych układach paczek.

Porównanie obu trackerów należy rozumieć jako analizę kompromisu między szybkością a stabilnością identyfikatorów. ByteTrack to rozwiązanie lżejsze, poprawiające ciągłość śledzenia przy niewielkim narzucie obliczeniowym. BoT-SORT wymaga większych zasobów, lecz oferuje silniejsze mechanizmy asocjacji. W części eksperymentalnej oba algorytmy są oceniane pod kątem równowagi między dokładnością zliczania a wydajnością w środowisku bliskim czasu rzeczywistemu.

Aplikacja generuje artefakty umożliwiające jakościową i ilościową ewaluację. Nagrania wynikowe zawierają adnotacje z trajektoriami (`sv.TraceAnnotator`), a metryki eksportowane do JSON obejmują czasy inferencji, FPS oraz statystyki predykcji. Dane te stanowią podstawę obiektywnego porównania ByteTrack i BoT-SORT pod względem poprawności zliczania i kosztu obliczeniowego.

4.3.2. ByteTrack i SORT dla RF-DETR i Faster R-CNN

W odróżnieniu od modelu YOLOv11, który posiada natywną integrację z algorytmami śledzenia zaimplementowanymi w ramach ekosystemu Ultralytics, architektury RF-DETR oraz Faster R-CNN funkcjonują w zaimplementowanym systemie ściśle jako wyizolowane detektory obiektów. Z tego względu zastosowanie paradygmatu śledzenia przez detekcję wymagało budowy zewnętrznej warstwy pośredniczącej. W kodzie aplikacji zadanie to realizuje dedykowany adapter `RoboflowTrackerAdapter`, który stosuje wzorzec projektowy strategii i utrzymuje detektor oraz tracker jako moduły rozdzielone, ale interoperacyjne. Adapter ten wymusza wcześniejsze wyznaczenie detekcji poprzez ustawienie flagi `requires_detection = True`, co oznacza, że etap śledzenia operuje wyłącznie na już przefiltrowanych i ustrukturyzowanych wynikach.

Kluczową różnicą architektoniczną względem przepływu danych dla modelu YOLOv11 jest kolejność wykonywania operacji w pętli przetwarzania klatki. Dla modeli RF-DETR i Faster R-CNN potok przetwarzania przebiega następująco: najpierw generowane są surowe detekcje, następnie odrzucane są predykcje o pewności poniżej ustalonego progu, potem eliminowane są obiekty znajdujące się poza wielokątem definiującym obszar roboczy (ROI) taśmy transportowej, a dopiero tak oczyszczony zbiór trafia do trackera.

Zastosowanie zewnętrznych algorytmów wymagało pełnego ujednolicenia formatów komunikacji pomiędzy warstwami. Wewnętrzne struktury detekcyjne (reprezentowane przez systemową klasę `Detection`) są wewnątrz adaptera mapowane na wysoce abstrakcyjny format operacyjny `sv.Detections` z biblioteki pomocniczej supervision [32].

Następnie, funkcja trackera dla danej klatki zwraca zbiory wzbogacone o globalne identyfikatory (`track_id`), które rzutowane są z powrotem do zunifikowanego formatu aplikacji z zachowaniem oryginalnego słownika klas. Izolacja ta gwarantuje, że pozostałe moduły *pipeline*'u — w tym logika zliczania, eksport metryk i wizualizacja — pozostają niewrażliwe na różnice technologiczne pod spodem i działają identycznie dla wszystkich modeli. Samo zliczanie przesyłek opiera się na zbiorach unikalnych, już zaobserwowanych identyfikatorów (`seen_track_ids`) w pamięci. Z tego względu każda pomyłka trackera i przedwczesne przełączenie tożsamości nieuchronnie przekłada się na błędne zliczenie fizycznej paczki.

Dla pary modeli RF-DETR i Faster R-CNN udostępniono śledzenie przy użyciu algorytmów SORT oraz ByteTrack z oficjalnych pakietów Roboflow. Świadomie pominięto tu rozwiązanie BoT-SORT, ponieważ jego otwarta implementacja powiązana jest ściśle z frameworkiem Ultralytics i brakuje jej modularnego, niezależnego od detektora odpowiednika. Stworzenie od zera pełnoprawnego wrappera dla BoT-SORT wykraczało poza ramy niniejszego środowiska ewaluacyjnego typu *proof-of-concept*.

Wybrane techniki śledzenia diametralnie różnią się tolerancją na jakość i stabilność predykcji. Algorytm SORT stanowi prostolinijną, lekką linię bazową, opartą w całości na estymacji filtra Kalmana oraz miarze przestrzennego nałożenia IoU. Brakuje mu wsparcia predykcyjnego w obliczu nagłych okluzji, co predysponuje go do częstego gubienia trajektorii w sytuacjach zaniku surowej detekcji z klatki. Ma to miejsce niezwykle często w przypadku modelu Faster R-CNN, który generuje mniej stabilne predykcje dla małych, podatnych na przesłonięcia obiektów (klasa `mailer`). Natomiast ByteTrack drastycznie polepsza retencję i spójność tożsamości obiektów w czasie, ponieważ uwzględnia w strategii asocjacji również ramki odrzucone jako predykcje o niskiej ufności, przypisując je w drugim etapie do niezmatchowanych trajektorii. Ta innowacja jest nadzwyczaj pożądana i kompensująca dla modeli transformatorowych (takich jak RF-DETR), których rozkład atencji i ostateczne wartości pewności bywają niestabilne dla przesyłek częściowo przykrytych. ByteTrack pozwala wyciągnąć rzeczywiste trajektorie z marginalizowanego w innej sytuacji „szumu", redukując fragmentację ścieżek jedynie kosztem marginalnego wzrostu obciążenia obliczeniowego.

Dzięki zachowanej powtarzalności środowiska opartego na stałych, zewnętrznych trackerach, ewentualne różnice w statystykach końcowego zliczenia można przypisać rygorystycznie czystej jakości i stabilności warstw detekcyjnych badanych architektur, a nie odchyleniom wprowadzonym przez niejednolite parametry śledzenia.

4.4. Środowisko programistyczne

Środowisko programistyczne zaprojektowanego systemu detekcji i zliczania paczek zostało skonstruowane z naciskiem na modułowość, powtarzalność wyników oraz ścisłą separację zagadnień. Ze względu na badawczy charakter aplikacji, będącej rozwiązaniem typu *proof-of-concept*, architektura oprogramowania musiała wspierać szybką eksperymentację, bezproblemową wymianę modeli detekcji obiektów oraz rzetelną ewaluację ilościową. Głównym językiem implementacji jest Python 3.13, wybrany ze względu na jego dominu-

jącą pozycję i rozbudowany ekosystem w dziedzinie współczesnego widzenia komputerowego i głębokiego uczenia.

W celu zapewnienia rozwiązywania złożonych zależności związanych z bibliotekami uczenia maszynowego, projekt wykorzystuje uv jako główne narzędzie do zarządzania pakietami i środowiskami wirtualnymi. Narzędzie to stanowi szybką i powtarzalną alternatywę dla tradycyjnych rozwiązań, takich jak pip czy virtualenv, a obecność pliku blokującego (`uv.lock`) gwarantuje spójność środowiska na różnych etapach: od rozwoju, przez trening, aż po wdrożenie. Kontrola wersji jest realizowana za pomocą systemu Git, a repozytorium hostowane jest na platformie GitHub[1]. Strategia gałęzi okazała się kluczowa dla izolacji konfiguracji eksperymentalnych — gałąź `larger_models` służy do ewaluacji większych wariantów modeli.

4.4.1. Architektura kodu źródłowego i zapewnienie jakości

Z perspektywy inżynierii oprogramowania, baza kodu jest ściśle podzielona na dwie główne warstwy: logiki domenowej oraz warstwę prezentacji, co jest zgodne z zasadą separacji zagadnień. Rdzeń algorytmiczny systemu zlokalizowany jest w pakiecie `src/`, który zawiera m.in. abstrakcyjne klasy bazowe i implementacje wrapperów dla modeli wnioskowania (`inference.py`), zunifikowane struktury danych detekcji (`detection.py`), adaptery algorytmów śledzenia wieloobiektowego (`tracking.py`), główny potok przetwarzania klatka-po-klatce (`video_processor.py`) oraz moduł ekstrakcji metryk ilościowych (`metrics_extractor.py`). Izolacja logiki domenowej gwarantuje, że różne architektury detekcji mogą być oceniane w identycznych warunkach eksperymentalnych.

Warstwa prezentacji, zlokalizowana w katalogu `app/`, zawiera wyłącznie kod interfejsu użytkownika oparty na frameworku Streamlit i callbacki, nigdy nie wykonując wnioskowania bezpośrednio. Taka architektura nie tylko ułatwia testowanie jednostkowe logiki rdzeniowej, ale także pozwala na ponowne wykorzystanie tych samych modułów z pakietu `src` w notatnikach treningowych, eliminując duplikację kodu i ryzyko rozbieżności w transformacjach danych między treningiem a wnioskowaniem.

Jakość kodu w projektach badawczych, które ewoluują wraz ze zmieniającymi się hipotezami, bywa często pomijana, co może zagrozić powtarzalności eksperymentów. Aby zminimalizować to ryzyko, zintegrowano nowoczesne narzędzia analizy statycznej. Ruff pełni rolę szybkiego lintera i formattera, egzekwując zestawy reguł (E, F, UP, B, SIM, I) z rygorystycznym limitem długości linii wynoszącym 79 znaków. Z kolei narzędzie ty odpowiada za statyczne typowanie, co jest kluczowe w systemie operującym na tensorach i strukturach detekcyjnych, zapobiegając błędom typu runtime podczas długich sesji wnioskowania.

4.4.2. Frameworki uczenia maszynowego i widzenia komputerowego

Fundamentem obliczeń tensorowych i akceleracji sprzętowej jest biblioteka PyTorch, zapewniająca dynamiczny graf obliczeniowy niezbędny zarówno w fazie uczenia, jak i

[1] https://github.com/lukmiik/masters_thesis_parcels_object_detection_web_app

wnioskowania. Każda z trzech analizowanych architektur detekcji wykorzystuje dedykowane biblioteki, dostosowane do jej paradygmatu:

— **Faster R-CNN:** Klasyczny detektor dwuetapowy został zaimplementowany z wykorzystaniem pakietu TorchVision, który dostarcza zoptymalizowane moduły ekstrakcji cech, w tym wariant `fasterrcnn_mobilenet_v3_large_fpn`. TorchVision zostało wybrane, ponieważ jako oficjalna biblioteka PyTorch dostarcza gotowe implementacje Faster R-CNN z pre-trenowanymi wagami, co eliminuje ryzyko błędów implementacyjnych przy odtwarzaniu architektury referencyjnej.
— **YOLOv11:** Jednoetapowy potok detekcji czasu rzeczywistego jest napędzany przez framework Ultralytics. Biblioteka ta upraszcza proces uczenia transferowego i udostępnia zoptymalizowane silniki wnioskowania, a także natywne adaptery do algorytmów śledzenia. Ultralytics wybrano ze względu na natywną integrację treningu, ewaluacji i inferencji w jednym API, co skraca czas implementacji i eliminuje potrzebę pisania własnych pętli treningowych.
— **RF-DETR:** Architektura transformatorowa zintegrowana jest przy użyciu oficjalnego pakietu `rfdetr`, który opiera się na bibliotece HuggingFace Transformers. Pozwala to systemowi na wykorzystanie mechanizmów uwagi i globalnych relacji przestrzennych bez konieczności stosowania heurystyk typu non-maximum suppression. Oficjalny pakiet `rfdetr` jest jedyną referencyjną implementacją tej architektury udostępnioną przez autorów (Roboflow), co gwarantuje zgodność wyników z publikowanymi benchmarkami.

Przetwarzanie strumieni wideo o wysokiej przepustowości, typowych dla środowisk logistycznych, wymaga wydajnych narzędzi. System wykorzystuje OpenCV do dekodowania, zapisu i manipulacji ramkami wideo oraz konwersji przestrzeni barw (np. BGR-na-RGB) wymaganych przez detektory. Wybrano tą bibliotekę jako standard de facto do operacji wideo w środowiskach badawczych; jego C++ backend gwarantuje niski narzut przy dekodowaniu klatka po klatce w porównaniu z czystymi implementacjami Pythona. Jest ona wspierana przez PyAV jako niskopoziomowy backend kodowania. PyAV posłużył jako backend kodowania wyjściowych nagrań wideo, oferując bardziej stabilne API niż bezpośrednie wywołania ffmpeg w środowisku Python. Użyto biblioteki Pillow do dodatkowych transformacji obrazów. Mostkiem dla operacji numerycznych na tensorach i tablicach ramek ograniczających jest NumPy.

4.4.3. Śledzenie, ewaluacja i przetwarzanie danych

Wieloobiektowe śledzenie paczek, przekształcające detekcje klatka-po-klatce w ciągłe trajektorie czasowe, zaimplementowano z wykorzystaniem biblioteki trackers, dostarczającej implementacje algorytmów SORT i BoT-SORT. Kluczową rolę odegrała również biblioteka Supervision, która zapewnia niezależne od frameworka narzędzia do wizualizacji, adnotacji i integracji trackerów (w tym ByteTrack) z modelami, które nie posiadają natywnego wsparcia dla śledzenia — czyli RF-DETR i Faster R-CNN.

Ilościowa ewaluacja wyników detekcji i śledzenia opiera się na bibliotece TorchMetrics, która dostarcza ustandaryzowane implementacje miar, takich jak średnia precyzja uśred-

niona (mAP) oraz Intersection over Union (IoU), co gwarantuje rzetelne i powtarzalne porównania między modelami. Z kolei do analizy danych eksperymentalnych i generowania wykresów w Rozdziale 5 wykorzystano pakiety pandas, matplotlib oraz tabulate, a pasek postępu podczas intensywnych obliczeń realizowany jest przez bibliotekę tqdm.

4.4.4. Infrastruktura eksperymentalna i interfejs webowy

Zarządzanie eksperymentami treningowymi jest głęboko zintegrowane z interfejsem aplikacji webowej. Logika trenowania każdego detektora zawarta jest w dedykowanych notatnikach Jupyter:
`train_yolov11.ipynb`, `train_rf_detr.ipynb` oraz `train_faster_rcnn.ipynb`. Zamiast wymagać od użytkownika ręcznego uruchamiania notatników, system wykorzystuje narzędzie Papermill do ich programistycznego wykonania bezpośrednio z poziomu aplikacji, z parametrami (np. liczba epok, rozmiar batcha) przekazywanymi z interfejsu graficznego. Taka konstrukcja oddziela definicję procedury treningowej od jej orkiestracji, czyniąc proces w pełni udokumentowanym i powtarzalnym. Monitorowanie przebiegu uczenia w czasie rzeczywistym realizowane jest za pomocą TensorBoard. Wybrano tę bibliotekę jako narzędzie monitorowania treningu ze względu na natywną integrację z PyTorch i brak konieczności kont zewnętrznych, co jest istotne w środowisku bez stałego dostępu do internetu.

Zarządzanie zestawem danych odbywa się poprzez Roboflow Python SDK, który automatyzuje pobieranie odpowiedniej wersji adnotowanych danych z wykorzystaniem poświadczeń API przechowywanych w plikach `.env` i ładowanych przez bibliotekę python-dotenv. Konfiguracja ta gwarantuje, że trening, walidacja i wnioskowanie zawsze odwołują się do tych samych annotacji *ground truth*. Interfejs użytkownika, zaimplementowany jako wielostronicowa aplikacja Streamlit, zawiera m.in. zakładkę ROI Editor. Wykorzystuje ona komponent `streamlit-drawable-canvas-fix`, pozwalając użytkownikowi na interaktywne zdefiniowanie wielokąta obszaru zainteresowania (ROI) na klatce wzorcowej, co jest krytyczne dla ograniczenia detekcji wyłącznie do obszaru taśmy transportowej.

4.5. Sprzęt i środowisko obliczeniowe

Wszystkie eksperymenty, w tym trening modeli, ewaluacja oraz inferencja, zostały przeprowadzone na lokalnej stacji roboczej, bez wykorzystania zewnętrznej infrastruktury chmurowej. Takie podejście było celowym wyborem metodologicznym, który gwarantował pełną kontrolę nad zmiennymi eksperymentalnymi. Działanie w odizolowanym środowisku lokalnym wykluczyło wpływ opóźnień sieciowych, wahającej się przepustowości czy wąskich gardeł współdzielonych zasobów chmurowych na zmierzone czasy wnioskowania i wskaźniki FPS (klatek na sekundę), zapewniając wiarygodne odzwierciedlenie surowych możliwości systemu.

Podstawową jednostką przetwarzającą stacji roboczej był procesor Intel Core i7-13700KF (16 rdzeni, 24 wątki, częstotliwość bazowa 3,4 GHz, 30 MB pamięci Smart Cache). W architekturze zaprojektowanego systemu CPU odpowiada za realizację potoków preprocessingu danych, obsługę interfejsu aplikacji webowej Streamlit oraz koordynację ogólnego

przebiegu pętli przetwarzania wideo. Ponadto, procesor ten wykonywał operacje matematyczne wymagane przez zaimplementowane algorytmy śledzenia (ByteTrack, BoT-SORT i SORT). Kluczową rolę CPU stanowiło również środowisko wykonawcze dla Eksperymentu III, w którym oceniano wykonalność i wydajność uruchomienia całego potoku detekcji obiektów wyłącznie na procesorze, co symuluje scenariusze wdrożeniowe na urządzenia brzegowe (edge-deployment) o ograniczonych zasobach sprzętowych.

Do intensywnych zadań głębokiego uczenia wykorzystano kartę graficzną NVIDIA GeForce RTX 4070 Dual, wyposażoną w 12 GB pamięci VRAM GDDR6X. Karta ta stanowiła główny akcelerator zarówno w fazie treningu, jak i inferencji modeli YOLOv11, RF-DETR oraz Faster R-CNN. Pojemność pamięci VRAM wymusiła określone konfiguracje treningowe — w szczególności konieczne było ostrożne zarządzanie rozmiarem paczki (*batch size*) oraz zastosowanie akumulacji gradientów (np. w przypadku modelu RF-DETR), aby zapobiec błędom braku pamięci (OOM — Out-Of-Memory) przy jednoczesnym zachowaniu optymalnej zbieżności.

Z kolei podczas ewaluacji RTX 4070 umożliwiał wysokoprzepustową regresję ramek ograniczających i klasyfikację w czasie rzeczywistym, stanowiąc górną linię odniesienia dla wydajności systemu. Wybór tego układu GPU podyktowany był silnym kompromisem między wydajnością obliczeniową a dostępnością w środowisku innym niż serwerowe, co sprawia, że konfiguracja ta odzwierciedla realistyczne środowisko programistyczne o wysokiej wydajności.

Stacja robocza została dodatkowo wyposażona w 32 GB pamięci RAM DDR5 oraz szybki dysk NVMe SSD, co zapobiegło powstawaniu wąskich gardeł podczas ładowania danych i zapisywania wyników pośrednich. System operacyjny stanowił Ubuntu 24.04 LTS. Środowisko programistyczne opierało się na natywnej instalacji stosu technologicznego obejmującego CUDA 13.0, cuDNN oraz framework PyTorch 2.9.1, aby uniknąć narzutu wydajnościowego wynikającego z konteneryzacji. Zarządzanie zależnościami w środowisku Python realizowano za pomocą narzędzia uv z zablokowanym grafem zależności, co gwarantowało powtarzalność konfiguracji. Backend PyTorch był domyślnie skonfigurowany do wykorzystania akceleracji GPU, z możliwością przełączenia na CPU poprzez flagę `--device cpu` w skryptach wnioskowania lub odpowiednią opcję w interfejsie aplikacji webowej. Zaimplementowano to w ujednoliconym mechanizmie wyboru urządzenia w module `src/inference.py`, co pozwoliło na bezproblemowe przełączanie się między GPU a CPU bez konieczności modyfikacji kodu. Trening wszystkich modeli wykonywano na GPU z wykorzystaniem operacji o zmniejszonej precyzji (FP16 — Mixed Precision)[2] w celu zmaksymalizowania przepustowości pamięci i skrócenia czasu uczenia.

Taki celowy projekt architektury oprogramowania umożliwił przeprowadzenie kontro-

[2] Mixed Precision (mieszana precyzja) to technika treningu sieci neuronowych polegająca na jednoczesnym stosowaniu dwóch formatów liczbowych: 16-bitowego (FP16, half precision) dla operacji obliczeniowych — takich jak mnożenia macierzy i konwolucje — oraz 32-bitowego (FP32, single precision) dla operacji wymagających wysokiej stabilności numerycznej, w szczególności dla kopii wag modelu. Takie podejście pozwala niemal o połowę zmniejszyć zużycie pamięci VRAM oraz przyspieszyć obliczenia na kartach graficznych wyposażonych w rdzenie Tensor Core (NVIDIA), przy zachowaniu jakości treningu porównywalnej z trybem pełnej precyzji. Stabilność treningu zapewnia mechanizm skalowania straty (loss scaling), który zapobiega znikaniu gradientów w reprezentacji FP16.

lowanego eksperymentu porównawczego (szczegółowo opisanego w sekcji 5.4), w którym ten sam potok wideo uruchamiano z identycznymi modelami i ustawieniami — raz na GPU RTX 4070, a raz na CPU i7-13700KF. Zmierzony wskaźnik FPS i opóźnienia dostarczyły bezpośrednich informacji o praktycznej użyteczności systemu w środowiskach przemysłowych opartych wyłącznie na CPU, stanowiąc istotną część odpowiedzi na postawione w pracy pytania badawcze. Skrypty i logi z testów wydajnościowych wyizolowano w dedykowanej gałęzi `cpu_gpu` w repozytorium kodu źródłowego. Należy zaznaczyć, że choć bezwzględne wartości wydajnościowe specyficzne dla opisanej konfiguracji sprzętowej mogą się różnić na innych platformach, wnioski dotyczące względnej wydajności i kompromisów między badanymi modelami pozostają trafne.

4.6. Aplikacja webowa — architektura i funkcjonalności

Rozwinięciem koncepcji zaprezentowanej w Rozdziale 4.1 jest aplikacja webowa, której zadaniem jest udostępnienie graficznego interfejsu użytkownika dla całego potoku detekcji, śledzenia i zliczania paczek. Aplikacja ta pełni rolę demonstracyjnego narzędzia typu *proof-of-concept*, umożliwiającego użytkownikowi końcowemu — bez konieczności bezpośredniej interakcji z terminalem czy notatnikami Jupyter — konfigurowanie eksperymentów, uruchamianie wnioskowania na wybranych plikach wideo, definiowanie wielokątów obszaru zainteresowania (ang. *Region of Interest*, ROI) oraz wizualne porównywanie wyników pomiędzy różnymi konfiguracjami modeli i trackerów.

Aplikacja została zbudowana przy użyciu frameworka Streamlit, który umożliwia szybkie tworzenie interaktywnych interfejsów webowych opartych na języku Python, bez konieczności pisania kodu HTML, CSS czy JavaScript. Takie podejście jest szczególnie uzasadnione w projektach badawczych, w których stos technologiczny jest w całości oparty na Pythonie — pozwala to zachować spójność z logiką domenową zgromadzoną w pakiecie `src/` i jednocześnie obniżyć barierę wejścia dla użytkowników końcowych. Jednocześnie Streamlit reprezentuje szerszą kategorię narzędzi do interaktywnej demonstracji modeli uczenia maszynowego, do której należą również rozwiązanie takie jak Gradio, służące udostępnianiu modeli w przystępny sposób w przeglądarce internetowej.

Działanie aplikacji zilustrowano także w formie nagrania demonstracyjnego, przedstawiającego interfejs systemu oraz przykład analizy wideo z użyciem konfiguracji RF-DETR + SORT. Nagranie jest dostępne pod adresem: https://github.com/lukmiik/masters_thesis_parcels_object_detection_web_app/blob/main/demo.mp4.

4.6.1. Architektura oprogramowania i separacja zagadnień

Fundamentalną zasadą projektową aplikacji jest ścisłe rozdzielenie warstwy interfejsu użytkownika od logiki domenowej, co stanowi zgodne z zasadą separacji zagadnień (ang. *separation of concerns*) podejście do architektury oprogramowania. W praktyce oznacza to, że kod odpowiedzialny za interakcję z użytkownikiem — renderowanie widżetów, obsługę zdarzeń oraz układ graficzny — nie zawiera żadnej logiki obliczeniowej związanej z detekcją obiektów, śledzeniem czy ekstrakcją metryk. Takie rozdzielenie zapobiega modelowaniu połączeń między elementami graficznymi a operacjami uczenia maszynowego,

które w kodzie badawczym często prowadzi do trudnego w utrzymaniu splątania zależności.

W strukturze aplikacji punkt wejścia `app/main.py` pełni rolę routera i rejestruje cztery podstrony zlokalizowane w katalogu `app/pages/`: Porównanie Modeli, Uruchamianie Wnioskowania, Edytor ROI oraz Trening. Moduły znajdujące się w `app/pages/` odpowiadają wyłącznie za interakcję z użytkownikiem, zarządzanie stanem sesji oraz wizualizację wyników. Natomiast wszystkie operacje obliczeniowe — dekodowanie klatek wideo, tensorowe przekształcenia, wnioskowanie modeli, śledzenie wielobiektowe oraz ekstrakcja metryk ilościowych — pozostają całkowicie wyizolowane w pakiecie `src/`.

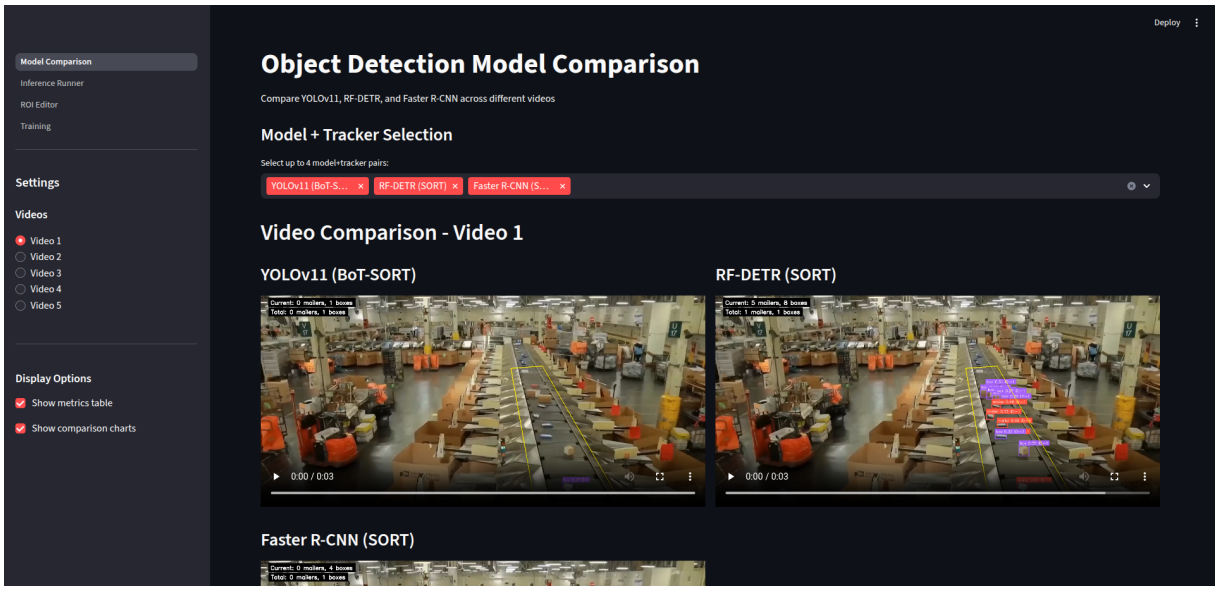
Tak skonstruowana architektura sprawia, że aplikacja webowa funkcjonuje jako „lekki klient" koordynujący wywołania tych samych funkcji z modułu `src/`, które są wykorzystywane w eksperymentach uruchamianych w trybie wsadowym. Dzięki temu każda zmiana w logice przetwarzania wideo, śledzenia czy ewaluacji automatycznie propaguje się do interfejsu webowego, eliminując ryzyko rozbieżności między wynikami prezentowanymi w aplikacji a rezultatami opisywanymi w Rozdziale 5.

Konfiguracja ścieżek dostępu do plików i katalogów jest scentralizowana w pliku `src/paths.py`, co zapewnia spójność operacji na plikach wejściowych (`input_videos/`), wagach modeli (`weights/`), wynikach przetwarzania (`outputs/`) oraz konfiguracji ROI (`roi.json`) we wszystkich komponentach systemu.

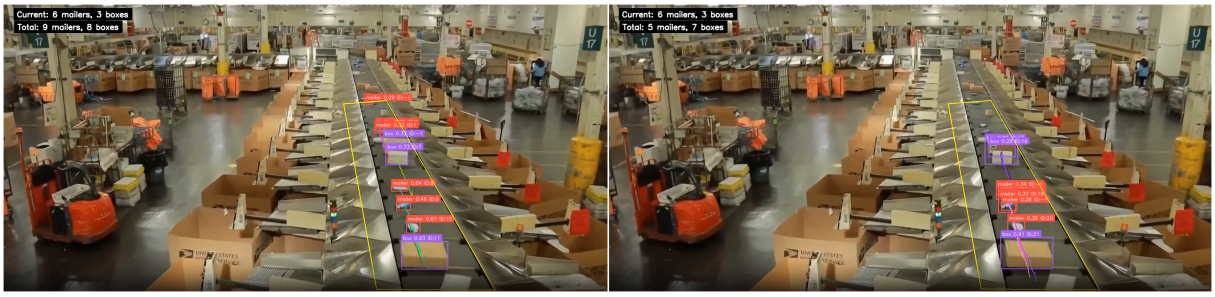
4.6.2. Moduły aplikacji i cykl życia eksperymentu

Aplikacja obejmuje pełny cykl życia eksperymentu poprzez cztery dedykowane moduły:

Porównanie Modeli (`view.py`). Ta strona stanowi domyślny punkt wejścia do aplikacji i jest podstawowym narzędziem do wizualnej oraz ilościowej oceny wszystkich testowanych konfiguracji model–detektor. Użytkownik wybiera w panelu bocznym plik wideo wejściowe, a następnie w głównej części interfejsu może skonfigurować do czterech kombinacji modelu detekcji i algorytmu śledzenia (np. YOLOv11 z ByteTrack, RF-DETR z SORT, Faster R-CNN z ByteTrack, YOLOv11 z BoT-SORT). Widok zakładki Model Comparison, umożliwiającej wybór konfiguracji model–tracker oraz porównanie adnotowanych materiałów wideo dla wybranego nagrania, przedstawiono na rysunku 4.3. Przykład wizualnego porównania adnotowanych klatek dla dwóch konfiguracji detekcyjno-śledzących w zakładce Model Comparison przedstawiono na rysunku 4.4.

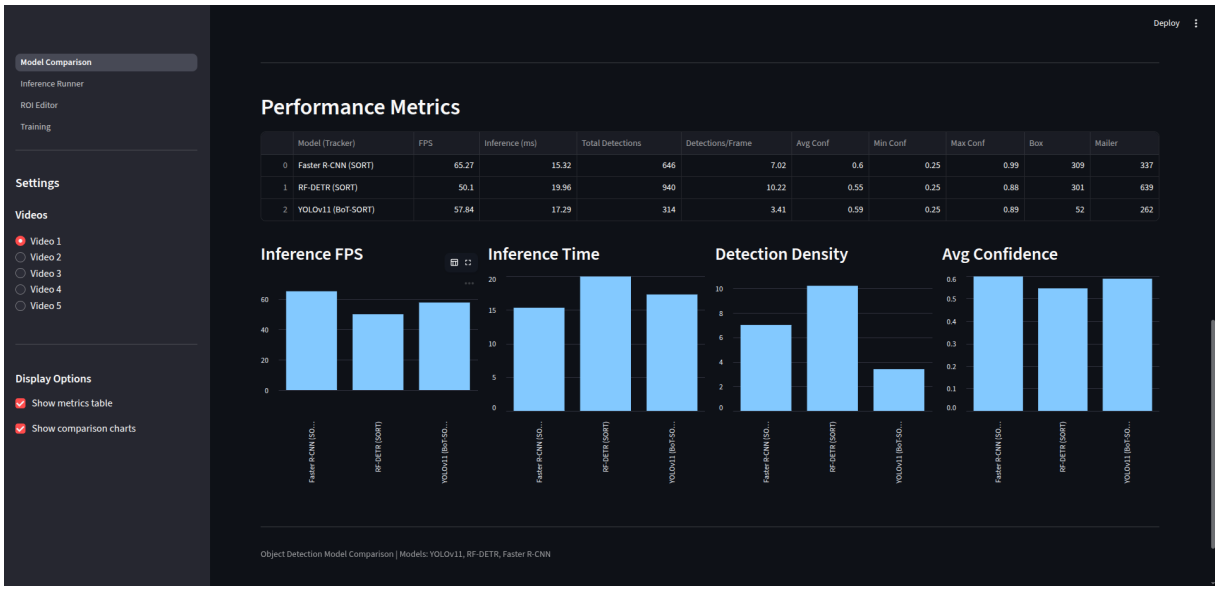


Rysunek 4.3: Widok zakładki *Model Comparison* w aplikacji webowej — wybór konfiguracji model–tracker oraz porównanie adnotowanych wideo dla wybranego nagrania.



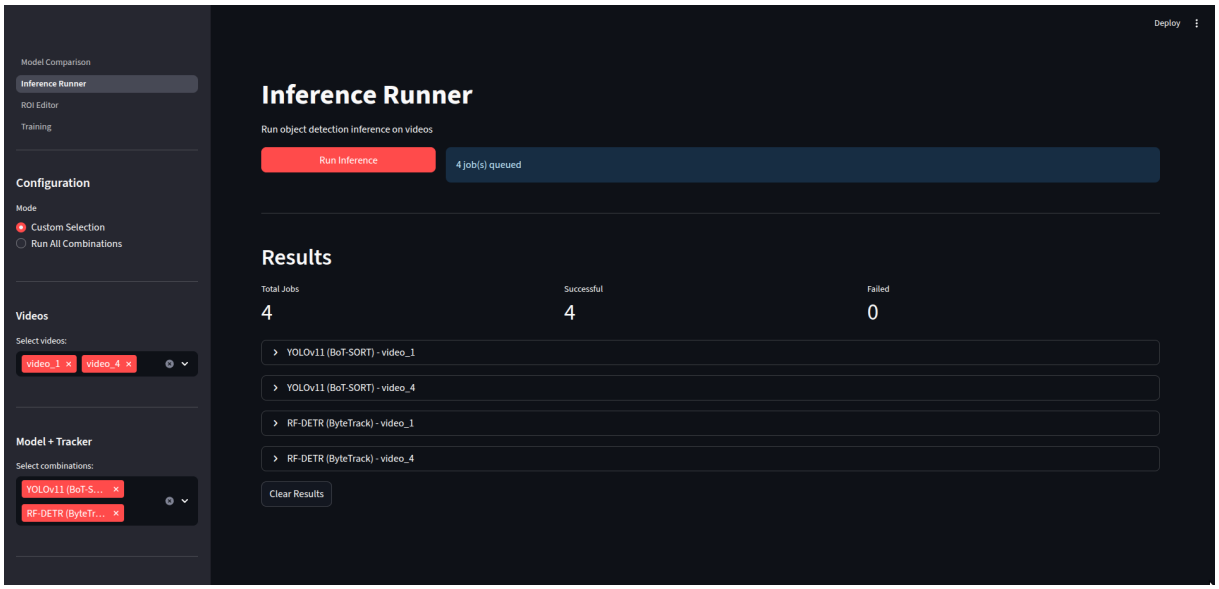
Rysunek 4.4: Porównanie adnotowanych klatek w zakładce *Model Comparison* — konfiguracje RF-DETR z ByteTrack oraz Faster R-CNN z ByteTrack dla tego samego nagrania.

Dla każdej włączonej kombinacji aplikacja dynamicznie ładuje adnotowane wideo wynikowe z katalogu `outputs/labeled_videos/` i renderuje je obok siebie, umożliwiając bezpośrednią ocenę wizualną stabilności śledzenia oraz precyzji ramek ograniczających. Ponadto, moduł agreguje dane z plików metryk w formacie JSON i konstruuje tabelę porównawczą oraz cztery wykresy słupkowe: FPS wnioskowania, Średni czas wnioskowania, Gęstość detekcji oraz Średnie zaufanie. Podejście to wykorzystuje mechanizm cachowania (`@st.cache_data`), co zapewnia płynną manipulację danymi nawet podczas analizy gęstych logów detekcyjnych [40]. Dane tabelaryczne i wykresy są identyczne z tymi, które są analizowane w Rozdziale 5, co umożliwia cytowanie wyników bezpośrednio w dyskusji naukowej. Sekcję Performance Metrics, zawierającą tabelę porównawczą oraz wykresy słupkowe FPS, czasu inferencji, gęstości detekcji i średniego zaufania, zilustrowano na rysunku 4.5.



Rysunek 4.5: Sekcja *Performance Metrics* w zakładce *Model Comparison* — tabela porównawcza oraz wykresy słupkowe FPS, czasu inferencji, gęstości detekcji i średniego zaufania.

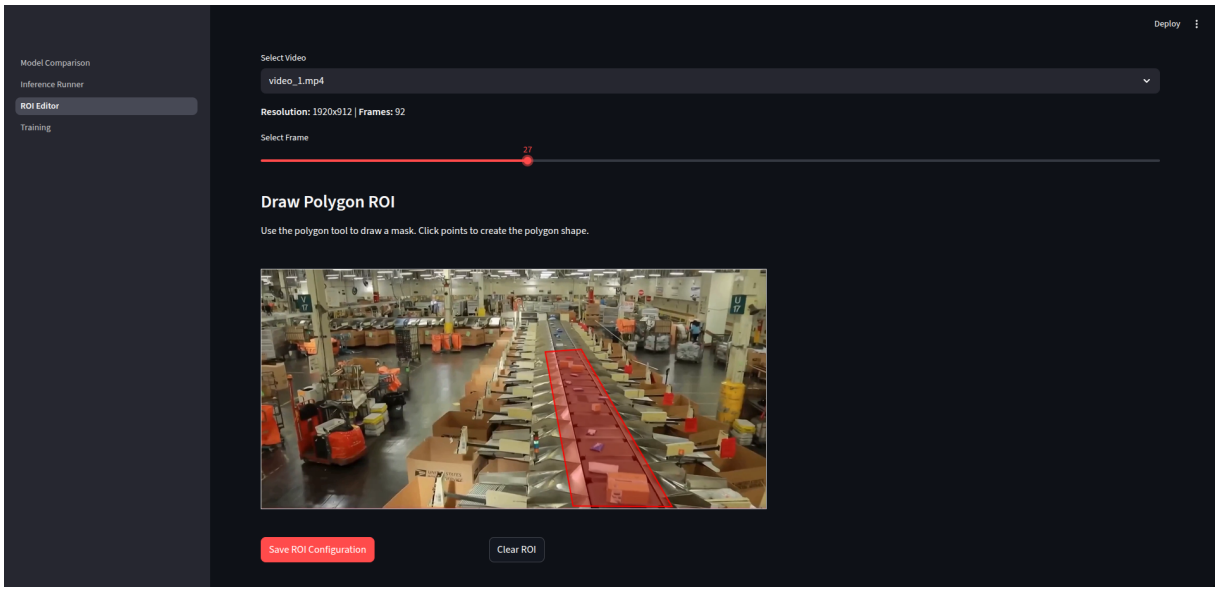
Uruchamianie Wnioskowania (`inference.py`). Ten moduł stanowi silnik wykonawczy interfejsu graficznego, pozwalający użytkownikowi planować i uruchamiać zadania detekcji i śledzenia na wybranych plikach wideo. Dostępne są dwa tryby pracy: tryb „Wybór ręczny", w którym użytkownik samodzielnie paruje model z trackerem, oraz tryb „Uruchom wszystkie kombinacje", automatycznie kolejkujący wszystkie dozwolone pary model–detektor obsługiwane przez backend systemu. Widok zakładki Inference Runner, wykorzystywanej do konfiguracji zadań wnioskowania oraz przeglądania podsumowania ich wykonania, przedstawiono na rysunku 4.6.



Rysunek 4.6: Widok zakładki *Inference Runner* w aplikacji webowej — konfiguracja zadań wnioskowania oraz podsumowanie ich wykonania.

Istotnym elementem interaktywnym jest suwak progu zaufania (ang. *confidence threshold*), który umożliwia precyzyjne dostrojenie czułości detekcji w czasie rzeczywistym. Po zainicjowaniu zadania moduł iteracyjnie przetwarza kolejkę, wywołując wewnętrzne funkcje `get_detector()` oraz `process_video()`. Stan aplikacji jest zarządzany poprzez `st.session_state`, co pozwala zachować wyniki pomiędzy kolejnymi renderowaniami interfejsu. Moduł wyświetla pasek postępu, podsumowanie wykonania oraz osadzone odtwarzacze wideo dla ukończonych zadań. Wielokąt ROI odpowiadający wybranemu plikowi wideo jest automatycznie ładowany z pliku `roi.json` i wstrzykiwany do potoku przetwarzania.

Edytor ROI (`roi_editor.py`). W przemysłowych zastosowaniach wizji maszynowej filtrowanie szumu tła jest kluczowe dla utrzymania wysokiej dokładności śledzenia oraz redukcji narzutu obliczeniowego. Edytor ROI adresuje to zagadnienie, zapewniając interaktywne narzędzie do rysowania wielokątnej maski bezpośrednio na wybranej klatce wideo. Do realizacji tego celu wykorzystywany jest komponent `streamlit-drawable-canvas`, który udostępnia płótno HTML5 pozwalające na precyzyjne rysowanie zamkniętych wielokątów na obszarze taśmy transportowej. Interaktywne definiowanie wielokątnego obszaru zainteresowania ROI na wybranej klatce nagrania, realizowane w zakładce ROI Editor, przedstawiono na rysunku 4.7.



Rysunek 4.7: Widok zakładki *ROI Editor* w aplikacji webowej — interaktywne definiowanie wielokątnego obszaru zainteresowania na wybranej klatce nagrania.

Moduł automatycznie skaluje współrzędne narysowane na płótnie do natywnej rozdzielczości wideo źródłowego, wykorzystując obliczony współczynnik skalowania. Tak skonfigurowany wielokąt jest serializowany i zapisywany do centralnego pliku `roi.json` wraz z surowym stanem płótna, umożliwiającym późniejsze wczytanie i edycję. Plik ten jest następnie czytany przez moduł Uruchamiania Wnioskowania i moduły przetwarzania podrzędnego, co gwarantuje, że raz zdefiniowany ROI jest automatycznie stosowany do wszystkich kolejnych uruchomień wnioskowania.

Orkiestracja Treningu (`training.py`). Moduł ten stanowi pomost między rozwojem eksperymentalnym a interfejsem produkcyjnym aplikacji. Umożliwia użytkownikowi inicjowanie dostrajania modeli YOLOv11, RF-DETR oraz Faster R-CNN bezpośrednio z przeglądarki internetowej. Zamiast implementować redundantne pętle treningowe, moduł ten wykorzystuje narzędzie Papermill do parametryzowanego wykonywania notatników Jupyter jako podrzędnych procesów systemowych. Poprzez eksponowanie hiperparametrów treningowych — takich jak liczba epok — w interfejsie użytkownika i wstrzykiwanie ich do notatników, aplikacja zapewnia, że proces uczenia modelu pozostaje w pełni powtarzalny, dokumentowany oraz odizolowany od wątku serwera webowego. Wyniki wykonanych notatników są zapisywane w katalogu `training_outputs/`, a postęp oraz status zakończenia (sukces lub komunikat błędu wraz ze standardowym wyjściem błędów) są wyświetlane w czasie rzeczywistym w interfejsie aplikacji.

4.6.3. Stos technologiczny i optymalizacje

Aplikacja została zbudowana w całości w języku Python. Oprócz frameworka Streamlit wykorzystuje OpenCV do dekodowania strumieni wideo i ekstrakcji właściwości klatek, natomiast biblioteki Pillow (PIL), NumPy oraz pandas są intensywnie używane do transformacji obrazów i agregacji metryk.

W celu utrzymania responsywności interfejsu użytkownika, mimo ciężkich operacji wejścia–wyjścia związanych z przetwarzaniem wideo i parsowaniem plików JSON, aplikacja wykorzystuje mechanizmy memoizacji i cachowania. Zunifikowany model danych — który ściśle odwzorowuje wewnętrzne identyfikatory (np. `rf_detr`, `bytetrack`) na czytelne dla użytkownika etykiety prezentacyjne — zapewnia spójność we wszystkich wykresach, tabelach i odtwarzaczach wideo. Integracja tych narzędzi tworzy samowystarczalne demonstracyjne narzędzie, które enkapsuluje cały przepływ pracy: od konfiguracji sceny i treningu modeli, przez wykonanie wnioskowania, po analizę końcowych metryk śledzenia wieloobiektowego.

Rozdział 5

Eksperymenty i wyniki

Niniejszy rozdział przedstawia empiryczną ewaluację opracowanego systemu wizyjnego, przeznaczonego do detekcji oraz zliczania małych i dużych przesyłek pocztowych na taśmie transportowej. Głównym celem przeprowadzonych badań jest weryfikacja głównej hipotezy badawczej oraz udzielenie odpowiedzi na szczegółowe pytania (PB1–PB4) sformułowane w podrozdziale 1.2. Ewaluacja została przeprowadzona z wykorzystaniem autorskiej aplikacji webowej, która pełni rolę zintegrowanego środowiska eksperymentalnego (ang. *testbed*). Zastosowanie jednolitego potoku przetwarzania (*pipeline*) pozwoliło na systematyczne i w pełni kontrolowane porównanie trzech architektur detekcji, algorytmów śledzenia wieloobiektowego, różnych wariantów rozmiarowych modeli oraz dwóch platform sprzętowych.

5.1. Plan eksperymentów i zastosowane metryki

Metodologia eksperymentalna została ustrukturyzowana tak, aby wyraźnie oddzielić ewaluację modeli detekcyjnych od algorytmów śledzenia, co bezpośrednio odzwierciedla współczesny paradygmat *tracking-by-detection*. Głównym założeniem planu eksperymentalnego było przeprowadzenie pełnej faktoryzacji kluczowych komponentów decydujących o praktycznej skuteczności systemu, przy jednoczesnym zachowaniu stałych i ściśle kontrolowanych zmiennych środowiskowych.

5.1.1. Środowisko eksperymentalne i kontrolowane warunki

Kluczowym argumentem metodologicznym niniejszej ewaluacji jest niezmienność warunków brzegowych. Analiza porównawcza została przeprowadzona przy zachowaniu następujących stałych parametrów:

— **Zbiór testowy:** ewaluacja wykorzystuje spójny zbiór 5 nagrań wideo (od `video_1.mp4` do `video_5.mp4`) zarejestrowanych z prędkością 30 FPS. Łączny materiał to 966 klatek (rozłożonych w nagraniach o długości odpowiednio: 92, 180, 133, 232 i 329 klatek). Nagrania charakteryzują się bardzo zróżnicowanym zagęszczeniem obiektów — od luźnych scen (ok. 1,1 detekcji na klatkę) po bardzo gęste konfiguracje (do 14 obiektów na klatkę), co ma kluczowe znaczenie dla testowania odporności trackerów w trudnych warunkach wizualnych.

— **Stałe parametry potoku:** każda iteracja potoku przetwarzania wykorzystuje dokładnie to samo filtrowanie przestrzenne za pomocą obszaru zainteresowania (ROI)

zaczytywanego deterministycznie z pliku `roi.json`, identyczny próg ufności dla detekcji ($confidence = 0{,}25$) oraz ten sam koder wyjściowy (H.264, CRF 23).

— **Powtarzalność:** każde uruchomienie eksperymentu jest w pełni powtarzalne dzięki ujednoliconej pętli przetwarzania. Jedynymi modyfikowanymi zmiennymi pomiędzy uruchomieniami są wstrzykiwane instancje klas implementujących interfejsy `BaseDetector` oraz `BaseTracker`.

5.1.2. Macierz eksperymentalna i mapowanie pytań badawczych

Aby dogłębnie odpowiedzieć na postawione pytania badawcze, w planie eksperymentów zastosowano pełne podejście czynnikowe: detektor × *tracker* × sprzęt × rozmiar modelu. System generuje łącznie 45 unikalnych przebiegów przetwarzania (3 modele × 3 warianty śledzenia, wliczając brak śledzenia jako linię bazową, × 5 nagrań wideo).

Architektura systemu wymusiła asymetryczne przypisanie algorytmów śledzenia do modeli:

— YOLOv11 wykorzystuje natywne implementacje algorytmów BoT-SORT oraz ByteTrack zintegrowane w backendzie Ultralytics, co pozwala trackerom korzystać z wewnętrznego stanu modelu.

— RF-DETR oraz Faster R-CNN korzystają z algorytmów ByteTrack i SORT dostarczanych przez zewnętrzną bibliotekę `supervision`, operującą na ujednoliconym interfejsie detekcji. ByteTrack stanowi tu wspólny mianownik dla wszystkich detektorów, zapewniając ustandaryzowaną linię bazową do porównań międzyarchitektonicznych.

Macierz eksperymentalna została podzielona na cztery odrębne etapy, które bezpośrednio odpowiadają na pytania badawcze:

— **Eksperyment I (podrozdział 5.2):** odpowiada na pytanie PB1. Koncentruje się na porównaniu bazowych modeli detekcji: YOLOv11, RF-DETR oraz Faster R-CNN. Eksperyment ten jest realizowany na gałęzi `main` repozytorium z wykorzystaniem akceleracji GPU.

— **Eksperyment II (podrozdział 5.3):** odpowiada na pytanie PB2. Ocenia wpływ skalowania rozmiaru modeli. Uwaga metodologiczna: podstawowa gałąź repozytorium (`main`) zawiera warianty lekkie (YOLOv11n, RF-DETR Nano, MobileNetV3). Gałąź o nazwie `larger_models` została wykorzystana do przechowania logów i wag wariantów większych (np. YOLOv11s, RF-DETR Small). Eksperyment ten bezpośrednio zestawia modele bazowe z ich bardziej złożonymi, cięższymi odpowiednikami.

— **Eksperyment III (podrozdział 5.4):** odpowiada na pytanie PB3. Zestawia wydajność inferencji w środowisku bazującym na CPU (gałąź `cpu_gpu`) z wynikami uzyskanymi na GPU (gałąź `main`), utrzymując identyczne konfiguracje modeli i nagrań w celu izolacji wpływu sprzętu obliczeniowego.

— **Eksperyment IV (podrozdział 5.5):** odpowiada na pytanie PB4. Ocenia dokładność algorytmów śledzenia i zliczania (BoT-SORT, ByteTrack, SORT).

5.1.3. Zastosowane metryki

Ponieważ projektowany system ma charakter aplikacji typu *proof-of-concept*, plan ewaluacji świadomie dzieli metryki na wskaźniki offline (z fazy treningu) oraz metryki inferencji online. Wyniki wszystkich przebiegów są agregowane w zbiorczym pliku `comparison_summary.csv`.

A. Metryki jakości detekcji offline (faza treningu). Ewaluowane przede wszystkim na potrzeby Eksperymentów I i II (PB1, PB2), stanowią standardowe metryki ewaluacyjne wyznaczane na zbiorach walidacyjnych:
— mAP@0.5 oraz mAP@0.5:0.95 (średnia precyzja uśredniona),
— Precyzja (*Precision*), Czułość (*Recall*) oraz krzywe PR i F1, służące do oceny balansu między fałszywymi alarmami a pominiętymi detekcjami.

B. Metryki wydajności i inferencji online (PB1, PB2, PB3). Kalkulowane dla każdej klatki podczas przetwarzania potokowego:
— `avg_inference_time_ms`: średni czas wymagany na pojedynczą iterację potoku (detekcja + śledzenie + filtracja przestrzenna).
— `avg_inference_fps`: wynikowa przepustowość, służąca jako punkt odniesienia do określenia, czy próg działania w czasie rzeczywistym ($\geqslant$ 30 FPS) został osiągnięty.
— Statystyki predykcji: łączna liczba detekcji, gęstość detekcji na klatkę, średnia pewność modelu na klasę (`avg_confidence_per_class`) oraz rozkład klas w nagraniu.

C. Metryki jakości zliczania i weryfikacja wizualna (PB4). Z uwagi na fakt, że nadrzędnym celem biznesowym systemu w zastosowaniach logistycznych jest zliczanie paczek, ostateczną miarą sukcesu algorytmu jest dokładność zliczania. Jednakże, ze względu na znane ograniczenia algorytmów śledzenia (takie jak fragmentacja trajektorii czy przełączanie identyfikatorów), poleganie wyłącznie na zliczeniu unikalnych identyfikatorów (`seen_track_ids`) mogłoby prowadzić do zafałszowanych wniosków. Przykładowo, wielokrotne zgubienie i ponowne wykrycie tej samej paczki mogłoby sztucznie zawyżyć wynik końcowy, maskując niską skuteczność detektora. Dlatego weryfikacja opiera się na bezpiecznym podejściu mieszanym:
— **Prawda podstawowa** (*Ground Truth*, GT): rzeczywista liczba fizycznych przesyłek (y) w każdym nagraniu wideo została ustalona manualnie przed rozpoczęciem eksperymentów.
— **Manualna weryfikacja jakościowa:** zasadnicza ocena poprawności działania algorytmów śledzenia opiera się na analizie wzrokowej (eksperckiej) wygenerowanych nagrań z naniesionymi adnotacjami. Pozwala to na bezpośrednie zaobserwowanie w czasie rzeczywistym, w których momentach *tracker* gubi obiekt, tworzy fałszywe ścieżki z szumu tła lub przypisuje nowe ID tej samej przesyłce po chwilowym przesłonięciu.
— **Referencyjny błąd zliczania:** system w celach referencyjnych oblicza końcowy błąd bezwzględny ($\epsilon = |\hat{y} - y|$), jednak wynik ten jest zawsze interpretowany ściśle w kontekście manualnej analizy nagrania. Ma to na celu wykluczenie sytuacji, w których poprawny końcowy wynik liczbowy jest jedynie przypadkowym efektem nałożenia się na siebie błędów detekcji i trackingu.

D. Ograniczenia metodologiczne. Zgodnie z założeniami zdefiniowanymi w podrozdziale 1.3, niniejszy system jest prototypem demonstracyjnym. W związku z tym plan eksperymentalny nie obejmuje zaawansowanych metryk oceny śledzenia wielu obiektów, takich jak MOTA czy IDF1. W zbiorze danych testowych nie generowano pełnych, ręcznych adnotacji tożsamości dla każdej klatki wideo. Ewaluacja rygorystycznie mierzy zatem końcową dokładność zliczania — najważniejszą metrykę biznesową dla potoków logistycznych — traktując wierność odtworzenia pełnej trajektorii czasoprzestrzennej jako element wtórny wobec ostatecznego wyniku liczbowego.

5.2. Eksperyment I — porównanie modeli detekcji

Niniejszy podrozdział odnosi się do pierwszego pytania badawczego (PB1), koncentrując się na ocenie bazowej wydajności trzech różnych architektur detekcji obiektów. Głównym celem jest ustalenie, który z modeli oferuje optymalny kompromis pomiędzy dokładnością detekcji a szybkością wnioskowania (inferencji) podczas analizy paczek przesuwających się na taśmie transportowej. Porównywane modele — YOLOv11n, RF-DETR Nano oraz Faster R-CNN (z wykorzystaniem *backbone*'a MobileNetV3-Large FPN) — reprezentują trzy odmienne paradygmaty w widzeniu komputerowym: detekcję jednoetapową, detekcję transformatorową *end-to-end* oraz klasyczną detekcję dwuetapową, co zostało teoretycznie zarysowane w Rozdziale 2.

5.2.1. Konfiguracja eksperymentu

Zbiór testowy składał się z 5 sekwencji wideo (`video_1` do `video_5`), zawierających łącznie 966 klatek zarejestrowanych z częstotliwością 30 klatek na sekundę (FPS). Rozkład klatek w poszczególnych nagraniach wynosił odpowiednio: 92, 180, 133, 232 oraz 329. Aby zapewnić rzetelne porównanie, potok przetwarzania (*pipeline*) wykorzystywał identyczne parametry dla wszystkich modeli: próg ufności (*confidence threshold*) ustawiono na poziomie 0,25, a ponadto zastosowano jednorodną maskę Obszaru Zainteresowania (Region of Interest — ROI), która odfiltrowywała zakłócenia z tła poza taśmociągiem. Modele trenowano na tym samym niestandardowym zbiorze danych (szczegółowo opisanym w Rozdziale 3), obejmującym dwie klasy: `box` (duże paczki) oraz `mailer` (małe paczki, takie jak koperty bąbelkowe i poliwers).

5.2.2. Metryki offline: jakość detekcji

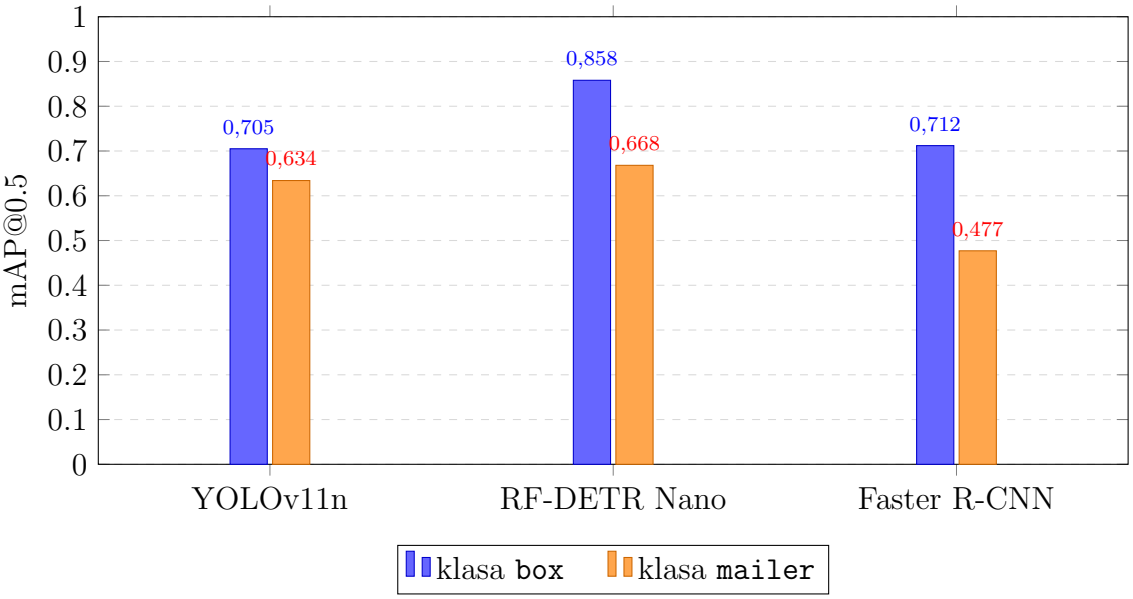
Ocena offline mierzy czyste możliwości predykcyjne modeli na zbiorze walidacyjnym po zakończeniu procesu treningu. Wyniki, zestawione w Tabeli 5.1, uwypuklają znaczące rozbieżności w sposobie, w jaki różne architektury radzą sobie z zadaniem detekcji przesyłek.

Tabela 5.1: Porównanie metryk jakości detekcji offline na zbiorze walidacyjnym dla ocenianych modeli.

Model	**mAP@0.5**	**mAP@0.5:0.95**	**Precyzja**	**Pełność (Recall)**
YOLOv11n	0,668	0,467	0,799	0,624
RF-DETR Nano	**0,763**	**0,552**	**0,870**	**0,690**
Faster R-CNN	0,570	0,356	0,570	0,418

RF-DETR Nano okazuje się bezapelacyjnym zwycięzcą pod względem jakości detekcji offline, osiągając najwyższe wyniki we wszystkich miarach, w tym mAP@0.5 na poziomie 0,763. Ta przewaga dokładności może być przypisana zdolności architektury transformatorowej do modelowania globalnych relacji przestrzennych i radzenia sobie z okluzjami bez polegania na heurystycznym przetwarzaniu końcowym, takim jak Non-Maximum Suppression.

Szczegółowa analiza per-klasa ujawnia jednak wyraźną asymetrię. RF-DETR Nano działa znacznie lepiej w przypadku dużych paczek (klasa `box`, mAP@0.5 = 0,858) w porównaniu do małych (klasa `mailer`, mAP@0.5 = 0,668). Ta rozbieżność potwierdza dobrze znany fakt, że małe, odkształcalne obiekty, takie jak koperty, dostarczają mniej wizualnych cech i pozostają trudnym problemem klasyfikacyjnym nawet dla najdokładniejszych architektur. Z kolei Faster R-CNN osiągnął najniższe metryki wśród wszystkich modeli, co odzwierciedla ograniczenia zastosowania lekkiego *backbone*'a MobileNetV3 w dwuetapowych detektorach dla specyficznych zbiorów logistycznych. Porównanie wartości mAP@0.5 dla klas `box` i `mailer` w trzech ocenianych modelach detekcji przedstawiono na rysunku 5.1.



Rysunek 5.1: Porównanie wartości mAP@0.5 dla klas `box` (duże paczki) oraz `mailer` (małe paczki) w trzech ocenianych modelach detekcji. Luka pomiędzy klasami wynosi odpowiednio +7,1 pp dla YOLOv11n, +19,0 pp dla RF-DETR Nano oraz +23,5 pp dla Faster R-CNN. Wszystkie modele wykrywają duże paczki lepiej niż małe, lecz wysoka dokładność globalna RF-DETR Nano nie przekłada się automatycznie na równomierność detekcji obu klas.

5.2.3. Metryki online: wydajność inferencji na GPU

Podczas gdy metryki offline kładą nacisk na teoretyczną dokładność, metryki online oceniają praktyczną użyteczność modeli w ciągłym strumieniu wideo. Tabela 5.2 przedstawia średnią wydajność wnioskowania na przestrzeni 5 testowych nagrań.

Tabela 5.2: Średnia wydajność inferencji online na GPU dla ocenianych modeli.

Model	Średni FPS	Średni czas [ms]	Łączna l. detekcji	Średnia ufność
YOLOv11n	**∼193**	**∼5,3**	3 201	0,678
Faster R-CNN	∼63	∼16,3	7 249	0,606
RF-DETR Nano	∼54	∼19,1	6 139	**0,733**

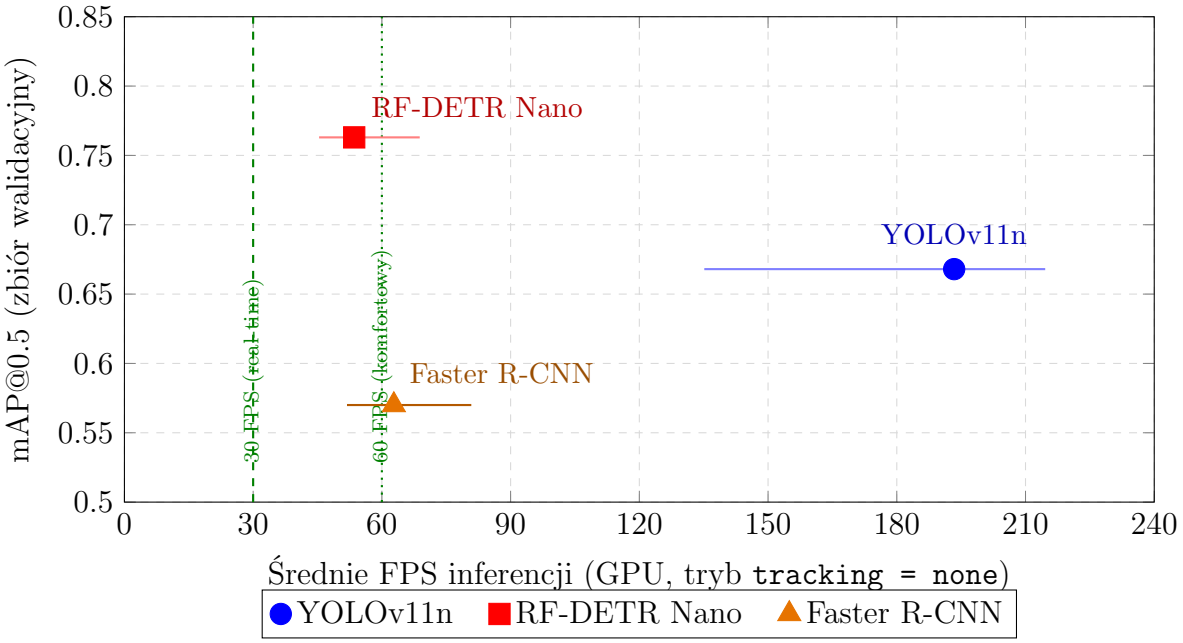
Wszystkie trzy modele z powodzeniem przekraczają próg przetwarzania w czasie rzeczywistym wynoszący 30 FPS, akcelerowane przez GPU. Niemniej jednak, YOLOv11n demonstruje potężną przewagę obliczeniową, osiągając średnio około 193 FPS. Czyni go to około trzykrotnie szybszym od konkurentów, co potwierdza wysoką efektywność nowoczesnych detektorów jednoetapowych. Jego przepustowość oznacza, że nawet przy analizie pojedynczego strumienia wideo detektor pozostawia znaczny zapas mocy obliczeniowej GPU, który może zostać wykorzystany na dodatkowe procesy, takie jak śledzenie, wizualizacja czy obsługa aplikacji webowej.

Analiza stabilności FPS w poszczególnych nagraniach (Tabela 5.3) uwidacznia interesujące zjawisko sprzętowe. YOLOv11n odnotował zauważalny spadek wydajności podczas przetwarzania `video_1` (135,1 FPS) w porównaniu do wysoce stabilnych ∼200–214 FPS osiąganych w kolejnych nagraniach. Jest to klasyczna manifestacja zjawiska rozgrzewania (*warm-up*) jąder CUDA, gdzie karta graficzna wymaga początkowych cykli na alokację pamięci i optymalizację grafów wykonawczych przed osiągnięciem szczytowej przepustowości. Modele RF-DETR i Faster R-CNN wykazywały większą stabilność, ale przy znacznie niższych wartościach bezwzględnych.

Tabela 5.3: Szczegółowa wydajność inferencji dla poszczególnych nagrań (FPS / czas inferencji w ms).

Wideo (l. klatek)	**YOLOv11n**	**RF-DETR Nano**	**Faster R-CNN**
`video_1` (92)	135,1 / 7,4 ms	45,4 / 22,0 ms	66,3 / 15,1 ms
`video_2` (180)	204,9 / 4,9 ms	68,8 / 14,5 ms	80,8 / 12,4 ms
`video_3` (133)	200,8 / 5,0 ms	55,7 / 18,0 ms	58,3 / 17,1 ms
`video_4` (232)	214,6 / 4,7 ms	52,2 / 19,2 ms	56,6 / 17,7 ms
`video_5` (329)	211,4 / 4,7 ms	45,7 / 21,9 ms	51,9 / 19,3 ms

Co więcej, całkowita liczba wygenerowanych detekcji rzutuje światło na fundamentalne różnice behawioralne pomiędzy architekturami. Faster R-CNN i RF-DETR generują znacznie większą objętość ramek ograniczających (odpowiednio 7 249 i 6 139) w porównaniu do YOLOv11n (3 201). Biorąc pod uwagę, że Faster R-CNN notuje jednocześnie najniższy średni wynik ufności (0,606), sugeruje to jego skłonność do generowania dużej liczby słabych predykcji lub fałszywych pozytywów. Natomiast YOLOv11n okazuje się znacznie bardziej selektywny, odrzucając wielokrotne i słabe detekcje na wczesnym etapie, co skutkuje mniejszą, ale bardziej pewną i stabilną pulą predykcji. W kontekście systemów zliczających jest to cecha wysoce pożądana, ponieważ zmniejsza ryzyko niestabilności algorytmów śledzenia i błędów podwójnego zliczania tych samych obiektów. Zależność pomiędzy średnią szybkością inferencji na GPU a jakością detekcji mierzoną mAP@0.5 zilustrowano na rysunku 5.2.



Rysunek 5.2: Wizualizacja kompromisu (*trade-off*) pomiędzy szybkością inferencji (średnie FPS na GPU) a jakością detekcji (mAP@0.5 na zbiorze walidacyjnym). Poziome odcinki ilustrują rozrzut FPS pomiędzy 5 nagraniami testowymi (zakres min–max). Pionowe linie odniesienia wyznaczają próg czasu rzeczywistego (30 FPS) oraz próg komfortowy (60 FPS). Pareto-front tworzą wyłącznie YOLOv11n (najszybszy) oraz RF-DETR Nano (najdokładniejszy); Faster R-CNN znajduje się na froncie zdominowanym, oferując jednocześnie niższą szybkość niż YOLOv11n i niższą dokładność niż RF-DETR Nano.

5.2.4. Dyskusja i odpowiedź na PB1

Wyniki Eksperymentu I dostarczają jednoznacznej odpowiedzi na pierwsze pytanie badawcze (PB1). Faster R-CNN reprezentuje najgorszy kompromis spośród przetestowanych architektur; oferuje niższą jakość detekcji niż RF-DETR Nano (mAP@0.5: 0,570 vs 0,763) i jest jednocześnie znacznie wolniejszy od YOLOv11n (∼63 FPS vs ∼193 FPS). Jego wydajność ilustruje ograniczenia lekkich sieci dwuetapowych w specjalistycznych zastosowaniach logistycznych, w których mechanizm propozycji regionów generuje nadmierną liczbę kandydatów o niskiej pewności.

Jeśli absolutnym priorytetem systemu logistycznego jest czysta jakość detekcji i odporność na okluzje — w szczególności dla trudniejszych klas paczek — RF-DETR Nano jest wyborem przewyższającym inne rozwiązania. Jako model oparty na transformatorach zapewnia najwyższą dokładność, jednak okupuje to najwolniejszym czasem wnioskowania.

Biorąc jednak pod uwagę optymalny kompromis pomiędzy dokładnością a szybkością działania, bezapelacyjnym zwycięzcą jest YOLOv11n. Utrzymuje on bardzo akceptowalny poziom dokładności (mAP@0.5 = 0,668) przy niespotykanej wydajności czasu rzeczywistego (∼193 FPS) oraz wysoce selektywnych, pewnych predykcjach. Połączenie tych cech pozostawia potężny zapas mocy obliczeniowej, który może zostać przeznaczony na intensywne algorytmy śledzenia wieloobiektowego (takie jak BoT-SORT czy ByteTrack) oraz obsługę warstwy aplikacji webowej. Czyni to architekturę jednoetapową najbardziej prak-

tycznym wyborem dla rozwiązań typu *proof-of-concept* w środowiskach przemysłowych, gdzie wymagana jest zarówno przepustowość, jak i skalowalność.

5.3. Eksperyment II — porównanie dużych i małych wariantów modeli

Niniejszy eksperyment adresuje drugie pytanie badawcze (PB2), dotyczące tego, w jakim stopniu rozmiar modelu wpływa na jakość detekcji i czy większe warianty modeli zapewniają istotną poprawę wyników względem mniejszych odpowiedników w zadaniu wykrywania paczek na taśmie transportowej. Analiza skupia się wyłącznie na dwóch architekturach, dla których dostępne są wstępnie wytrenowane punkty kontrolne (*checkpoints*) o różnej skali: YOLOv11 oraz RF-DETR. Detektor Faster R-CNN został celowo wykluczony z tego porównania, ponieważ zastosowana w pracy konfiguracja jego kręgosłupa (*backbone*) nie posiadała alternatywnego wariantu rozmiarowego, co uniemożliwia analizę skalowania w ramach niniejszego badania.

Przyjęto hipotezę roboczą zakładającą, że większe warianty modeli powinny zapewniać poprawę dokładności detekcji dzięki zwiększonej pojemności reprezentacyjnej, co jest zjawiskiem powszechnie obserwowanym w modelach głębokiego uczenia.

5.3.1. Konfiguracja eksperymentu

Eksperyment porównuje warianty *nano* i *small* dla obu architektur:

— YOLOv11: wariant *nano* (∼2,6 mln parametrów) względem wariantu *small* (∼9,4 mln parametrów),

— RF-DETR: wariant *Nano* (∼30,5 mln parametrów; 2,3 ms) względem wariantu *Small* (∼32,1 mln parametrów; 3,5 ms).

Obie rodziny modeli zostały wytrenowane przy użyciu identycznych potoków przetwarzania (*pipeline*) oraz tego samego zbioru danych (`mailers-and-boxes-v10` z klasami `box` i `mailer`), co zapewnia spójność eksperymentalną. Modele YOLOv11 były trenowane przez 12 epok z rozmiarem wsadu (*batch size*) równym 16 i rozdzielczością wejściową 640×640. Z kolei modele RF-DETR wykorzystywały wsad o rozmiarze 8 z akumulacją gradientu w 2 krokach (efektywny wsad 16), rozdzielczość 384×384 oraz mechanizm wczesnego zatrzymania (*early stopping*) z cierpliwością ustawioną na 5 epok. Taka kontrolowana konfiguracja jest zgodna z dobrymi praktykami w porównawczej ocenie systemów detekcji obiektów. Cały proces treningu został przeprowadzony na karcie NVIDIA GeForce RTX 4070 z wykorzystaniem mieszanej precyzji (FP16).

Ocena online została przeprowadzona za pomocą aplikacji webowej, na tym samym zestawie pięciu nagrań testowych wideo przy progu ufności (*confidence threshold*) 0,25 i z użyciem wspólnej maski ROI. Wyniki dla wariantów *nano* pozyskano z gałęzi `main` repozytorium (plik `outputs/metrics/comparison_summary.csv`), natomiast wyniki dla wariantów *small* pochodzą z gałęzi `larger_models`, co gwarantuje pełną izolację zmiennej niezależnej, jaką jest rozmiar modelu.

5.3.2. Metryki offline: jakość detekcji na zbiorze walidacyjnym

Wstępna ewaluacja na zbiorze walidacyjnym wykazała, że oba warianty osiągają wartość mAP@0.5 bliską 1,0. Na tym poziomie nasycenia różnice w wydajności praktycznie znikają, co czyni zbiór testowy niewłaściwym do zniuansowanego porównania. W związku z tym analizę oparto na wynikach ze zbioru walidacyjnego.

Tabela 5.4 podsumowuje wydajność obu wariantów na zbiorze walidacyjnym. W przypadku YOLOv11 przejście od wariantu *nano* do *small* przynosi jedynie marginalny wzrost mAP@0.5 (+0,007, tj. ok. +1%) oraz nieznaczniejszą poprawę w bardziej restrykcyjnej metryce mAP@0.5–0.95 (+0,021, tj. ok. +4,5%). Warto zauważyć, że poprawie tej towarzyszy przesunięcie kompromisu precyzja–pełność: pełność (*recall*) rośnie, ale kosztem spadku precyzji. Sugeruje to, że większy model generuje więcej detekcji, w tym również tych o niższej pewności, co jest zjawiskiem typowym dla detektorów jednoetapowych, gdzie skalowanie często faworyzuje pełność kosztem precyzji.

Tabela 5.4: Porównanie metryk jakości detekcji offline na zbiorze walidacyjnym dla wariantów YOLOv11 i RF-DETR.

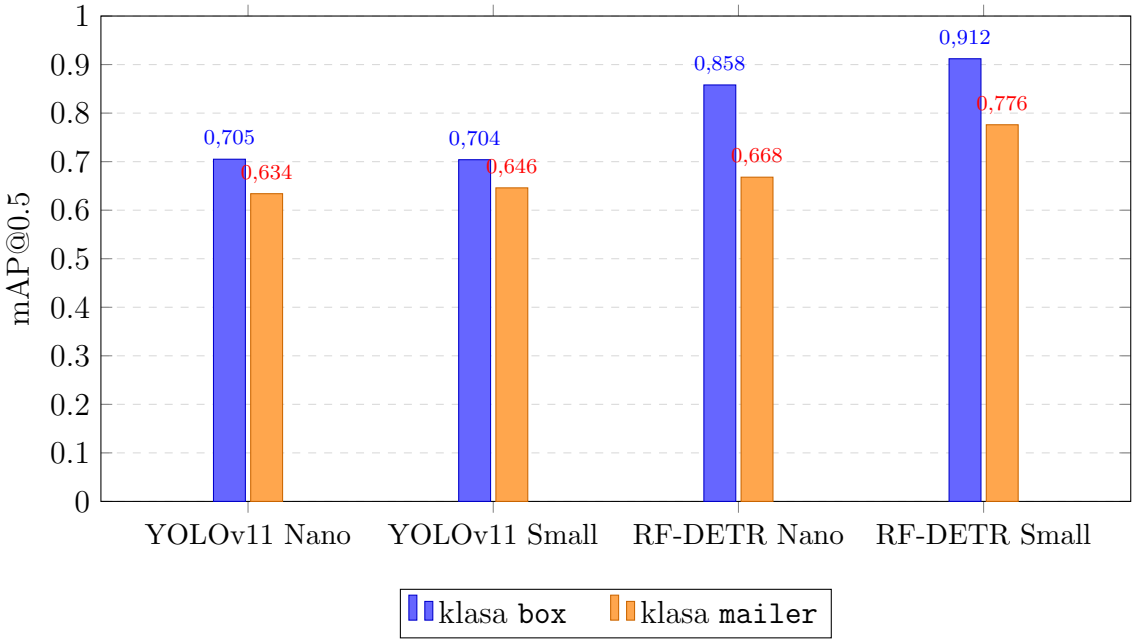
Architektura	**Wariant**	**Precyzja**	**Pełność**	**mAP@0.5**	**mAP@0.5–0.95**
YOLOv11	Nano (∼2,6 M)	0,799	0,624	0,668	0,467
YOLOv11	Small (∼9,4 M)	0,740	0,664	0,675	0,488
RF-DETR	Nano	0,870	0,690	0,763	0,552
RF-DETR	Small	**0,907**	**0,800**	**0,844**	**0,619**

W wyraźnym kontraście, dla architektury RF-DETR skok jakościowy jest znacznie bardziej wyrazisty. Wartość mAP@0.5 rośnie o +0,081 (+10,6%), a mAP@0.5–0.95 poprawia się o +0,067 (+12,1%). Jednocześnie poprawie ulega zarówno precyzja, jak i pełność, co wskazuje, że dodatkowa pojemność wariantu *Small* jest efektywnie wykorzystywana do poprawy reprezentacji cech bez wprowadzania istotnego szumu.

Analiza podziału na klasy dla modelu RF-DETR (Tabela 5.5) ukazuje, że zyski nie są równomiernie rozłożone. Klasa `mailer` — reprezentująca małe, często odkształcalne koperty i koperty bąbelkowe — pozostaje trudniejszą kategorią. Wariant *Small* poprawia mAP@0.5 dla tej klasy o +0,108 (+16,2%), co jest istotnie większym wzrostem niż dla klasy `box`. Wynik ten podkreśla przewagę architektur opartych na transformatorach w modelowaniu złożonych relacji przestrzennych i radzeniu sobie z trudnymi instancjami. Porównanie wartości mAP@0.5 dla klas `box` i `mailer` na zbiorze walidacyjnym dla wariantów *nano* i *small* architektur YOLOv11 oraz RF-DETR przedstawiono na rysunku 5.3.

Tabela 5.5: Metryki offline per klasa dla wariantów RF-DETR (zbiór walidacyjny).

Wariant	Klasa	Precyzja	Pełność	mAP@0.5	mAP@0.5–0.95
Nano	`box`	0,985	0,690	0,858	0,643
Nano	`mailer`	0,755	0,690	0,668	0,461
Small	`box`	0,987	0,800	0,912	0,692
Small	`mailer`	0,827	0,800	0,776	0,546



Rysunek 5.3: Porównanie wartości mAP@0.5 dla klas `box` (duże paczki) oraz `mailer` (małe paczki) na zbiorze walidacyjnym dla wariantów *nano* i *small* architektur YOLOv11 oraz RF-DETR. Luka pomiędzy klasami wynosi odpowiednio +7,1 pp dla YOLOv11 Nano, +5,8 pp dla YOLOv11 Small, +19,0 pp dla RF-DETR Nano oraz +13,6 pp dla RF-DETR Small. Skalowanie YOLOv11 nieznacznie zmniejsza dystans między klasami, lecz nie poprawia istotnie jakości detekcji dużych paczek; w przypadku RF-DETR przejście do wariantu *Small* przynosi wyraźną poprawę dla obu klas, przy czym największy zysk obserwowany jest dla trudniejszej klasy `mailer` (+0,108).

5.3.3. Metryki online: wydajność inferencji i gęstość detekcji

Ewaluacja w dynamicznym scenariuszu online na GPU dostarcza kluczowych informacji o praktycznych kompromisach wdrożeniowych. Tabela 5.6 przedstawia średnie metryki inferencji z wykorzystaniem trackera ByteTrack.

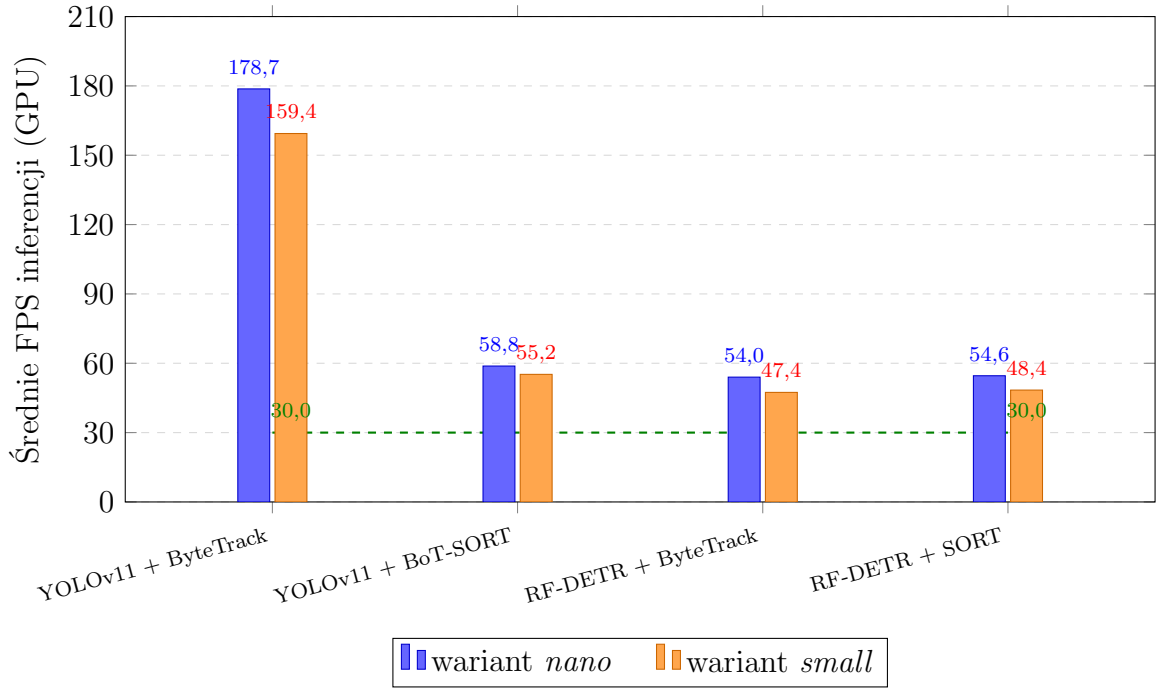
Przejście na większy wariant przewidywalnie wpływa na spadek wydajności. YOLOv11 notuje spadek FPS o około 10,8%, a RF-DETR o około 12,2%. Proporcjonalny narzut obliczeniowy jest zbliżony, mimo że bezwzględne wartości FPS różnią się ponad trzykrotnie. Wskazuje to, że skalowanie wpływa na czas inferencji w sposób przewidywalny i umiar-

Tabela 5.6: Średnia wydajność inferencji online na GPU dla wariantów modeli (*tracker*: *ByteTrack*).

Architektura	**Wariant**	**Śr. FPS**	**Śr. czas [ms]**	**Śr. ufność**	**Łączna l. detekcji**
YOLOv11	Nano	**178,7**	**5,6**	0,724	2 817
YOLOv11	Small	159,4	6,3	0,593	3 088
RF-DETR	Nano	54,0	18,6	0,761	4 631
RF-DETR	Small	47,4	21,3	**0,781**	**4 698**

kowany. Wpływ rozmiaru modelu na średni FPS inferencji dla wariantów *nano* i *small* w podziale na zastosowane trackery przedstawiono na rysunku 5.4.

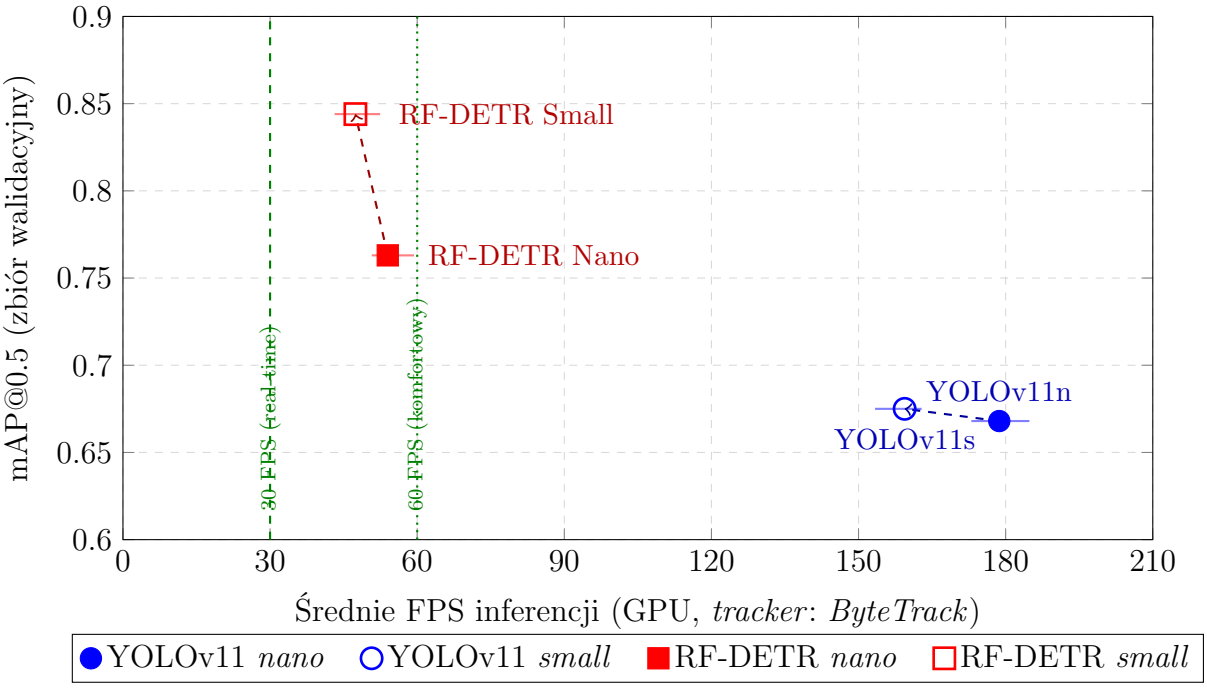
Interesującą obserwację dostarcza analiza średniego wyniku ufności (*confidence*). Dla YOLOv11 wariant *small* generuje niższą średnią ufność (0,593 vs 0,724 dla *nano*), mimo wyższego mAP. Oznacza to szerszą dystrybucję detekcji o umiarkowanej pewności, co wzmacnia wcześniejszą obserwację o wzroście pełności kosztem precyzji. W przeciwieństwie do tego, RF-DETR *Small* wykazuje wyższą średnią ufność (0,781 vs 0,761), co sugeruje bardziej stabilne i pewne predykcje.



Rysunek 5.4: Porównanie średniego FPS inferencji na GPU dla wariantów *nano* i *small* obu architektur w podziale na zastosowane algorytmy śledzenia. Wartości uśrednione na 5 nagraniach testowych (zielona linia przerywana wyznacza próg czasu rzeczywistego 30 FPS). Względny spadek przepustowości po przejściu na wariant *small* pozostaje umiarkowany i mieści się w przedziale ok. 6–13% niezależnie od *trackera*. Wszystkie konfiguracje, w tym warianty *small*, utrzymują się komfortowo powyżej progu czasu rzeczywistego.

Analiza zagregowanego rozkładu detekcji per klatka (sumowanych po wszystkich klat-

kach 5 nagrań testowych z *trackerem* ByteTrack) uwidacznia dalsze różnice w zachowaniu modeli. Większy wariant YOLOv11 wykrywa znacznie więcej obiektów klasy `box` (1 084 vs 558, +94%), przy jednoczesnym spadku detekcji klasy `mailer` (2 004 vs 2 265, −11,5%). Szczególnie ilustrujący jest przypadek `video_1.mp4`, gdzie YOLOv11n wykrył 304 obiekty (50 `box`, 254 `mailer`), podczas gdy YOLOv11s wykrył zaledwie 99 (75 `box`, 24 `mailer`). Niższa efektywna ufność wariantu *small* na trudnych instancjach powoduje, że są one tracone po zastosowaniu progu ufności. Zjawisko to musi być starannie rozważone, gdyż w zastosowaniach zliczających tłumienie trudnych detekcji może prowadzić do poważnych błędów. Z kolei RF-DETR utrzymuje wysoce stabilny rozkład klas niezależnie od rozmiaru modelu (2 105/2 526 dla wariantu *nano* oraz 2 137/2 561 dla wariantu *small* odpowiednio dla klas `box` i `mailer`), co jest prawdopodobnie efektem działania globalnego mechanizmu uwagi, pozwalającego na lepsze wyłapywanie kontekstu. Kompromis pomiędzy szybkością inferencji a jakością detekcji dla wariantów *nano* i *small* obu analizowanych architektur zilustrowano na rysunku 5.5.



Rysunek 5.5: Wizualizacja kompromisu (*trade-off*) pomiędzy szybkością inferencji (średnie FPS na GPU, tracker: ByteTrack) a jakością detekcji (mAP@0.5 na zbiorze walidacyjnym) dla wariantów *nano* i *small* obu architektur. Wykres rozszerza analizę z Rysunku 5.2 (Eksperyment I) o większe warianty modeli. Poziome odcinki ilustrują rozrzut FPS pomiędzy 5 nagraniami testowymi (zakres min–max), a pionowe linie odniesienia wyznaczają progi czasu rzeczywistego (30 FPS) i komfortowy (60 FPS). Przerywane strzałki obrazują przejście od wariantu *nano* do *small*: dla YOLOv11 jest ono niemal poziome (marginalny wzrost dokładności), natomiast dla RF-DETR wyraźnie skierowane w górę (znaczący skok mAP@0.5 o +0,081). Pareto-front tworzą wyłącznie YOLOv11n (najszybszy) oraz RF-DETR Small (najdokładniejszy, 0,844); pozostałe warianty są zdominowane.

5.3.4. Dyskusja i odpowiedź na PB2

Otrzymane wyniki dostarczają zniuansowanej odpowiedzi na pytanie badawcze PB2. Wpływ rozmiaru modelu na jakość detekcji jest silnie zależny od architektury bazowej. Zjawisko to jest spójne z szerszymi ustaleniami opisywanymi w literaturze, wskazującymi, że korzyści ze skalowania modelu są uwarunkowane jego strukturą.

1. **RF-DETR**: większy wariant przynosi istotną i praktycznie znaczącą poprawę jakości detekcji (od +8% do +12% w metrykach mAP), w szczególności dla trudniejszej klasy `mailer`. Kosztem jest akceptowalny i umiarkowany spadek FPS rzędu 12%. Mechanizm globalnej uwagi pozwala transformatorowi na znacznie efektywniejsze wykorzystanie zwiększonej pojemności do modelowania drobnych, rozproszonych cech wizualnych, kluczowych przy detekcji małych i częściowo zasłoniętych przesyłek.
2. **YOLOv11**: różnica w jakości jest marginalna (+1–4% mAP). Co więcej, wariant *small* wprowadza negatywne efekty uboczne w dynamicznych scenariuszach śledzenia, takie jak niższa ogólna ufność i znaczny spadek detekcji małych paczek (`mailer`) na rzecz wykrywania dużych (`box`) w złożonych sekwencjach wideo.

Odpowiadając wprost na PB2: zwiększenie rozmiaru modelu poprawia jakość detekcji, jednak wielkość i praktyczna użyteczność tej poprawy zależą od zastosowanej architektury. W przypadku architektur transformatorowych (RF-DETR) hipoteza o pozytywnym wpływie skalowania znajduje mocne potwierdzenie. Dla jednoetapowych modeli konwolucyjnych (YOLOv11) zyski są marginalne, a w kontekście systemu zliczającego na taśmie transportowej, wariant *nano* pozostaje rozwiązaniem wystarczającym, tańszym obliczeniowo i bardziej stabilnym w zakresie rozkładu klas detekcji. Ostateczny wybór konfiguracji nie może opierać się na pojedynczej, izolowanej metryce, lecz musi wynikać ze wspólnej analizy jakości detekcji, przepustowości inferencji oraz stabilności zachowania w zadaniu docelowym, jakim jest śledzenie i zliczanie obiektów.

5.4. Eksperyment III — porównanie inferencji na CPU vs. GPU

Niniejszy eksperyment stanowi bezpośrednią odpowiedź na trzecie pytanie badawcze PB3, dotyczące różnicy wydajności pomiędzy inferencją realizowaną na CPU i GPU oraz praktycznej użyteczności wdrożenia systemu wyłącznie na procesorze. Szczegółowy opis środowiska obliczeniowego przedstawiono wcześniej w podrozdziale 4.4, natomiast plan eksperymentów oraz zastosowane metryki omówiono w podrozdziale 5.1, dlatego w tym miejscu ograniczono opis wyłącznie do elementów istotnych z perspektywy porównania obu platform wykonawczych.

W celu zachowania spójności metodologicznej na CPU i GPU wykorzystano identyczne wytrenowane wagi modeli, te same implementacje trackerów oraz niezmienne parametry potoku przetwarzania, w szczególności próg ufności detekcji i tę samą maskę ROI.

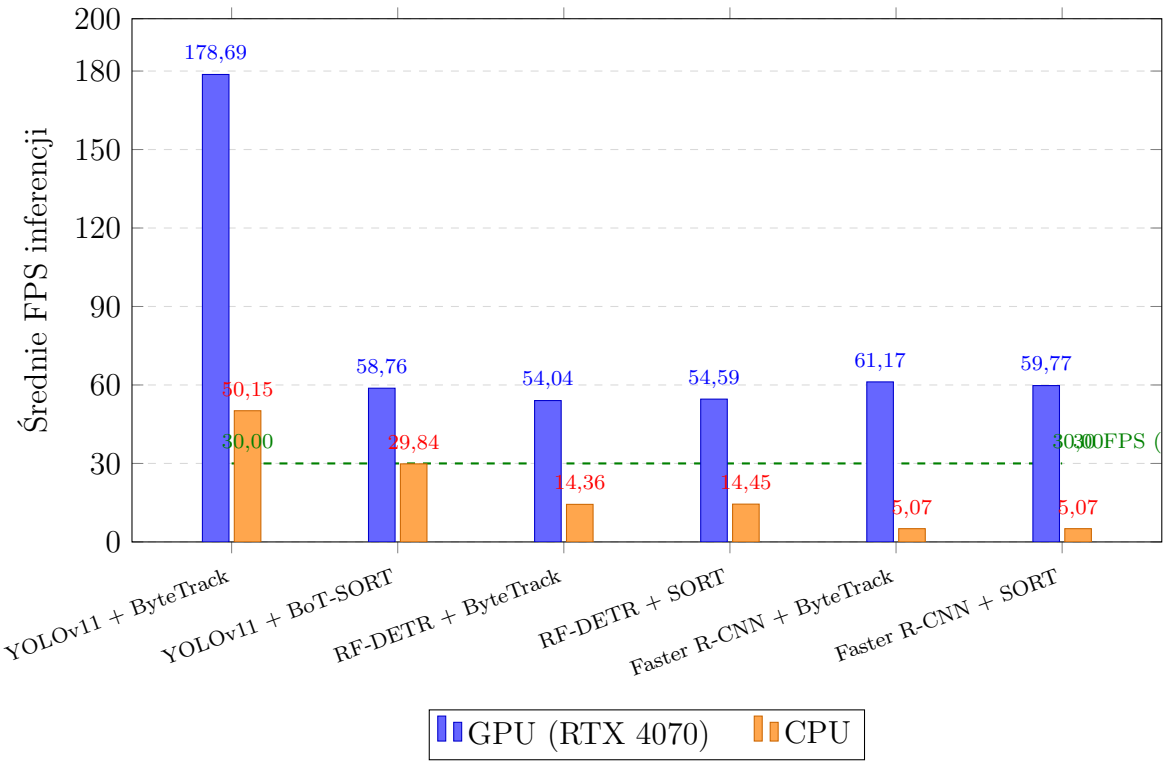
W analizie przyjęto metrykę `avg_inference_fps`, która odzwierciedla przepustowość całego potoku wykonywanego dla pojedynczej klatki, obejmującego detekcję, śledzenie oraz filtrację przestrzenną w obszarze zainteresowania. Z pomiaru wyłączono narzut związany z operacjami wejścia-wyjścia na plikach wideo, co jest zgodne z typowymi praktykami benchmarkingu w badaniach z zakresu widzenia komputerowego. Dodatkowo wszystkie

uruchomienia wykonano w kontrolowanych i powtarzalnych warunkach, tak aby jedyną zmienną niezależną pozostawała platforma sprzętowa. W celu ilościowego porównania obu platform wykonawczych zestawiono średnią wydajność inferencji dla wszystkich analizowanych konfiguracji model–*tracker* na GPU i CPU w tabeli 5.7.

Tabela 5.7: Średnia wydajność inferencji dla analizowanych konfiguracji model–*tracker* na GPU i CPU.

Model	**Tracker**	**GPU śr. FPS**	**CPU śr. FPS**	**GPU śr. [ms]**	**CPU śr. [ms]**	**Przyspieszenie (GPU/CPU)**
Faster R-CNN	ByteTrack	61,17	5,07	16,56	200,56	~12,1×
Faster R-CNN	SORT	59,77	5,07	16,83	199,46	~11,9×
RF-DETR	ByteTrack	54,04	14,36	18,58	69,70	~3,75×
RF-DETR	SORT	54,59	14,45	18,59	69,25	~3,73×
YOLOv11	ByteTrack	**178,69**	**50,15**	**5,60**	**20,19**	~3,6×
YOLOv11	BoT-SORT	58,76	29,84	17,10	33,70	~1,97×

Wyniki z tabeli 5.7 pokazują, że każda konfiguracja osiąga wyższy FPS na GPU, jednak skala przyspieszenia zależy od zastosowanego detektora i *trackera*. Uzyskane wyniki pokazują bardzo wyraźną przewagę GPU w każdym analizowanym wariancie. Wszystkie konfiguracje uruchamiane na karcie graficznej przekraczają próg 30 FPS, a więc spełniają warunek pracy w czasie rzeczywistym. Szczególnie wyróżnia się konfiguracja YOLOv11 z *trackerem ByteTrack*, która osiąga blisko 179 FPS, co oznacza niemal sześciokrotne przekroczenie progu czasu rzeczywistego. Tak duża przewaga GPU nad CPU jest zgodna z ogólną charakterystyką współczesnych akceleratorów, które bardzo dobrze radzą sobie z równoległymi operacjami macierzowymi występującymi w sieciach głębokich. Porównanie średniego FPS inferencji na GPU i CPU dla wszystkich analizowanych konfiguracji model–*tracker* przedstawiono na rysunku 5.6.



Rysunek 5.6: Porównanie średniego FPS inferencji na GPU oraz CPU dla wszystkich sześciu analizowanych konfiguracji model–*tracker*. Zielona pozioma linia przerywana wyznacza próg czasu rzeczywistego (30 FPS). Wszystkie konfiguracje na GPU komfortowo przekraczają ten próg, podczas gdy na CPU jedynie YOLOv11 (z ByteTrack zdecydowanie, z BoT-SORT na granicy progu) zachowuje praktyczną użyteczność. Architektury cięższe (Faster R-CNN, RF-DETR) na CPU nie spełniają wymagań pracy w czasie rzeczywistym, co potwierdza tezę o konieczności akceleracji sprzętowej dla detektorów innych niż lekkie modele jednoetapowe.

Wyniki uzyskane na CPU prowadzą do bardziej zróżnicowanych wniosków. Faster R-CNN osiąga zaledwie około 5 FPS, co całkowicie wyklucza praktyczne zastosowanie tej architektury w systemie zliczania przesyłek działającym w czasie rzeczywistym. RF-DETR wypada zauważalnie lepiej, jednak około 14 FPS nadal pozostaje wynikiem zbyt niskim dla scenariusza ciągłej analizy obrazu z przenośnika. Jedynie YOLOv11 zachowuje praktyczną użyteczność na CPU: w połączeniu z ByteTrack osiąga około 50 FPS, natomiast z BoT-SORT utrzymuje się na granicy czasu rzeczywistego, osiągając około 30 FPS. Taki rezultat dobrze wpisuje się w obserwacje z literatury, zgodnie z którymi lekkie, jednoetapowe detektory są znacznie lepiej przystosowane do wdrożeń na tańszych platformach brzegowych niż modele cięższe lub bardziej złożone architektonicznie.

Porównanie względnego przyspieszenia ujawnia ponadto, że skala korzyści z wykorzystania GPU silnie zależy od architektury modelu. Największy zysk obserwowany jest dla Faster R-CNN, który przyspiesza ponad dwunastokrotnie, co można wiązać z wysokim kosztem obliczeniowym charakterystycznym dla detektorów dwuetapowych. W przypadku RF-DETR wzrost wydajności wynosi około 3,7×, natomiast dla YOLOv11 około 3,6× przy ByteTrack i około 2× przy BoT-SORT. Oznacza to, że im cięższy i mniej zoptymali-

zowany do pracy na CPU jest model, tym bardziej niezbędne staje się wsparcie akceleracji GPU. Wnioski te są spójne z szerszymi analizami efektywnej inferencji modeli głębokiego uczenia.

Istotnym elementem eksperymentu jest również wpływ algorytmu śledzenia na końcową przepustowość systemu. W przypadku szybkiego detektora YOLOv11 zmiana trackera z ByteTrack na BoT-SORT powoduje istotny spadek FPS zarówno na CPU, jak i na GPU. Na CPU wydajność obniża się z około 50 FPS do około 30 FPS, natomiast na GPU z około 179 FPS do około 59 FPS. Sugeruje to, że przy wystarczająco szybkim detektorze koszt śledzenia przestaje być pomijalny i staje się widocznym składnikiem całkowitego czasu przetwarzania. Zjawisko to można tłumaczyć dodatkowymi operacjami wykonywanymi przez BoT-SORT, w szczególności modułem ReID, który zwiększa koszt obliczeniowy całego potoku.

Odwrotna sytuacja występuje dla wolniejszych modeli, tj. Faster R-CNN i RF-DETR. W ich przypadku różnice pomiędzy ByteTrack i SORT pozostają minimalne i nie przekraczają kilku procent. Oznacza to, że dominującym składnikiem opóźnienia jest sam etap detekcji, a narzut *trackera* zostaje w praktyce „ukryty" przez wolniejszą inferencję modelu bazowego. Podobny efekt obserwuje się w systemach typu *tracking-by-detection*, w których wpływ warstwy śledzenia jest tym bardziej widoczny, im szybszy jest detektor dostarczający wejściowe predykcje.

Analiza konfiguracji skrajnych dodatkowo uwydatnia skalę różnic pomiędzy badanymi wariantami. Najszybsza konfiguracja, tj. YOLOv11 z ByteTrack uruchomione na GPU, osiągnęła 184,84 FPS, co odpowiada około 5,41 ms na klatkę. Najwolniejsza konfiguracja, tj. Faster R-CNN z ByteTrack uruchomione na CPU, spadła do 4,44 FPS, czyli około 225,20 ms na klatkę. Oznacza to około 41-krotną różnicę wydajności w obrębie tego samego systemu aplikacyjnego, wynikającą wyłącznie z doboru modelu i platformy sprzętowej.

Jednocześnie eksperyment potwierdził, że zmiana urządzenia wykonawczego nie prowadzi do istotnego pogorszenia jakości samych predykcji. Dla tych samych par model–*tracker* liczba detekcji, średnie poziomy ufności oraz rozkład klas pozostawały bardzo zbliżone pomiędzy uruchomieniami na CPU i GPU. Niewielkie odchylenia numeryczne można wiązać z różnicami w arytmetyce zmiennoprzecinkowej oraz z drobnymi przesunięciami czasowymi w aktualizacji trajektorii obiektów przez *tracker*. Z praktycznego punktu widzenia oznacza to, że wybór CPU lub GPU wpływa przede wszystkim na przepustowość systemu, a nie na merytoryczną poprawność detekcji.

Na podstawie przeprowadzonego eksperymentu można sformułować jednoznaczną odpowiedź na pytanie PB3. Wdrożenie systemu wyłącznie na CPU jest praktycznie uzasadnione jedynie w przypadku lekkiego modelu YOLOv11 oraz stosunkowo taniego obliczeniowo *trackera*, takiego jak *ByteTrack*. Dla architektur cięższych, w szczególności Faster R-CNN i RF-DETR, zastosowanie GPU jest warunkiem koniecznym do osiągnięcia czasu rzeczywistego. Ostatecznie wybór platformy sprzętowej należy więc traktować jako decyzję optymalizacyjną dotyczącą kosztu wdrożenia i wymaganej przepustowości, a nie jako czynnik wpływający na samą jakość detekcji.

5.5. Eksperyment IV — porównanie *trackerów* w kontekście dokładności zliczania

Niniejszy eksperyment odpowiada na czwarte pytanie badawcze PB4 i dotyczy ustalenia, który algorytm śledzenia zapewnia najwyższą poprawność zliczania paczek we współpracy z poszczególnymi modelami detekcji. W odróżnieniu od klasycznych benchmarków MOT celem nie była tu optymalizacja metryk, takich jak MOTA czy IDF1, lecz ocena końcowego efektu użytkowego systemu, czyli poprawności łącznej liczby paczek policzonych na nagraniach z taśmy transportowej.

Porównanie przeprowadzono dla sześciu dopuszczalnych konfiguracji model–*tracker*, wynikających bezpośrednio z architektury zaimplementowanej aplikacji. Model YOLOv11 współpracuje z *trackerami* BoT-SORT i *ByteTrack*, natomiast RF-DETR oraz Faster R-CNN z *trackerami ByteTrack* i SORT, co łącznie daje sześć analizowanych zestawień. Jednocześnie należy podkreślić, że wspólna nazwa „ByteTrack" nie oznacza tu identycznej implementacji w każdym przypadku, ponieważ YOLOv11 korzysta z natywnego backendu Ultralytics, a pozostałe dwa detektory z odrębnej warstwy trackingowej zintegrowanej w aplikacji.

Eksperyment wykonano w warunkach kontrolowanych, aby możliwie dobrze odizolować wpływ samej warstwy śledzenia. Dla wszystkich konfiguracji wykorzystano ten sam zestaw pięciu nagrań testowych, ten sam obszar ROI zapisany w `roi.json` oraz ten sam próg ufności równy 0,25, zgodnie z ogólnym planem eksperymentów przyjętym w pracy.

Kluczowe znaczenie ma przy tym sama logika zliczania. System nie liczy przecięć linii ani pojedynczych detekcji klatkowych, lecz gromadzi zbiory unikalnych identyfikatorów śledzenia osobno dla klas `box` i `mailer`, a wynik końcowy definiuje jako liczność tych zbiorów. Taka konstrukcja powoduje, że każdy rozpad trajektorii lub zmiana ID może prowadzić do zawyżenia wyniku, natomiast połączenie dwóch obiektów w jeden track może skutkować niedoszacowaniem. Dodatkowym źródłem błędu jest niestabilność etykiety klasy, ponieważ ten sam obiekt może w różnych klatkach zostać dopisany do różnych zbiorów identyfikatorów, jeśli detektor naprzemiennie rozpoznaje go jako `box` i `mailer`.

Szczegółowe zestawienie liczby obiektów klasy `box` dla poszczególnych nagrań i konfiguracji przedstawiono w tabeli 5.8, natomiast odpowiadające mu zestawienie dla klasy `mailer` w tabeli 5.9.

Tabela 5.8: Liczba obiektów klasy `box` w podziale na nagrania i konfiguracje.

Wideo	**GT**	**YOLOv11 + BoT-SORT**	**YOLOv11 + ByteTrack**	**RF-DETR + ByteTrack**	**RF-DETR + SORT**	**Faster R-CNN + ByteTrack**	**Faster R-CNN + SORT**
`video_1`	7	6	8	8	19	7	16
`video_2`	9	11	12	17	17	17	22
`video_3`	8	4	4	15	19	27	37
`video_4`	2	5	5	3	3	3	8
`video_5`	4	13	12	8	13	8	22
Suma	30	39	41	51	71	62	105

Tabela 5.9: Liczba obiektów klasy `mailer` w podziale na nagrania i konfiguracje.

Wideo	**GT**	**YOLOv11 + BoT-SORT**	**YOLOv11 + ByteTrack**	**RF-DETR + ByteTrack**	**RF-DETR + SORT**	**Faster R-CNN + ByteTrack**	**Faster R-CNN + SORT**
`video_1`	11	4	4	9	22	5	14
`video_2`	8	19	21	12	16	19	32
`video_3`	9	20	20	10	15	41	52
`video_4`	4	5	5	5	6	7	9
`video_5`	13	33	32	14	25	22	47
Suma	45	81	82	50	84	94	154

Do ilościowej oceny błędu zliczania zastosowano trzy uzupełniające się metryki: średni błąd bezwzględny (*MAE*), średni bezwzględny błąd procentowy (*MAPE*) oraz błąd systematyczny (ang. *bias*). Wszystkie miary uśredniane są po nagraniach testowych, dzięki czemu każde wideo wnosi porównywalny wkład do wyniku końcowego, niezależnie od liczby zarejestrowanych w nim przesyłek. Niech $N = 5$ oznacza liczbę nagrań testowych, $y_i^{(c)}$ — rzeczywistą liczbę obiektów klasy $c \in \{\texttt{box}, \texttt{mailer}\}$ w nagraniu i (wartości *ground truth* z Tabel 5.8 i 5.9), a $\hat{y}_i^{(c)}$ — liczbę unikalnych identyfikatorów śledzenia tej samej klasy zwróconą przez daną konfigurację detektor–*tracker*. Łączną liczbę obiektów w nagraniu i definiujemy jako $y_i = y_i^{(\texttt{box})} + y_i^{(\texttt{mailer})}$ oraz analogicznie $\hat{y}_i = \hat{y}_i^{(\texttt{box})} + \hat{y}_i^{(\texttt{mailer})}$.

Średni błąd bezwzględny dla pojedynczej klasy c obliczany jest jako średnia arytmetyczna z bezwzględnych różnic per nagranie:

$$\mathrm{MAE}^{(c)} = \frac{1}{N} \sum_{i=1}^{N} \left| \hat{y}_i^{(c)} - y_i^{(c)} \right|. \tag{5.1}$$

Wariant „łącznie” nie jest sumą $\mathrm{MAE}^{(\texttt{box})} + \mathrm{MAE}^{(\texttt{mailer})}$, lecz liczony jest na sumarycznej liczbie paczek per nagranie, co lepiej oddaje błąd biznesowy całego potoku zliczającego:

$$\mathrm{MAE} = \frac{1}{N} \sum_{i=1}^{N} |\hat{y}_i - y_i|, \qquad \mathrm{MAPE} = \frac{100\%}{N} \sum_{i=1}^{N} \frac{|\hat{y}_i - y_i|}{y_i}. \tag{5.2}$$

Błąd systematyczny obliczany jest jako średnia arytmetyczna z różnic ze znakiem per nagranie i pozwala rozróżnić systematyczne przeszacowanie (> 0) od niedoszacowania (< 0):

$$\mathrm{Bias} = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i). \tag{5.3}$$

Zbiorcze wartości błędów zliczania dla wszystkich analizowanych konfiguracji detektor–*tracker*, obejmujące MAE dla klas `box` i `mailer`, MAE łączne, MAPE łączne oraz błąd systematyczny, przedstawiono w tabeli 5.10.

Tabela 5.10: Zestawienie błędów zliczania dla analizowanych konfiguracji.

Konfiguracja	MAE (box)	MAE (mailer)	MAE (łącznie)	MAPE łącznie [%]	Błąd syst. (łącznie)
YOLOv11 + BoT-SORT	3,80	10,00	12,20	79,9	+9,00
YOLOv11 + ByteTrack	3,80	10,20	12,00	78,8	+9,60
RF-DETR + ByteTrack	**4,20**	**1,80**	**5,60**	**37,2**	+5,20
RF-DETR + SORT	8,20	7,80	16,00	99,1	+16,00
Faster R-CNN + ByteTrack	6,40	12,20	18,60	117,7	+16,20
Faster R-CNN + SORT	15,00	21,80	36,80	239,4	+36,80

5.5.1. Dyskusja wyników i odpowiedź na PB4

Najbardziej ogólny wniosek z eksperymentu jest jednoznaczny: wszystkie analizowane konfiguracje wykazują dodatni błąd systematyczny, a więc system ma wyraźną tendencję do przeszacowywania liczby paczek. Taki obraz wyników jest spójny z przyjętą logiką zliczania opartą na unikalnych identyfikatorach śledzenia, ponieważ ponowne nadanie nowego ID już wcześniej zaobserwowanemu obiektowi skutkuje jego ponownym zliczeniem. Z perspektywy metod śledzenia oznacza to, że dominującym mechanizmem błędu nie jest samo niewykrycie obiektu, lecz fragmentacja trajektorii i przełączanie identyfikatorów.

Najlepsze wyniki uzyskała konfiguracja RF-DETR + ByteTrack, która osiągnęła najniższy błąd MAE łącznie, najniższy MAPE oraz najniższy dodatni błąd systematyczny. Szczególnie istotna jest przewaga tej konfiguracji dla klasy `mailer`, czyli klasy trudniejszej wizualnie ze względu na mniejsze rozmiary, bardziej płaską geometrię i mniejszą wyrazistość cech. Otrzymany wynik sugeruje, że w zadaniu zliczania paczek sama nazwa *trackera* nie wystarcza do oceny jakości całego rozwiązania, ponieważ ostateczny rezultat zależy od współdziałania stabilności detekcji i stabilności utrzymywania tożsamości obiektu w czasie.

Porównanie ByteTrack i SORT dla detektorów RF-DETR oraz Faster R-CNN również wypada jednoznacznie na korzyść ByteTrack. W obu przypadkach zastąpienie ByteTrack przez SORT prowadzi do wyraźnego wzrostu błędu zliczania, co można interpretować jako skutek większej podatności SORT na krótkotrwałe zaniki detekcji i rozpad trajektorii. W przeciwieństwie do prostszego podejścia asocjacyjnego, ByteTrack wykorzystuje również detekcje o niższej pewności do podtrzymania śledzenia, dzięki czemu lepiej radzi sobie z chwilowymi spadkami confidence i częściowymi przesłonięciami.

W przypadku modelu YOLOv11 różnice między BoT-SORT a ByteTrack okazały się natomiast bardzo małe. Obydwie konfiguracje uzyskały niemal identyczne wartości MAE, MAPE i błędu systematycznego, co oznacza, że w badanym scenariuszu dodatkowe mechanizmy oferowane przez BoT-SORT nie przełożyły się na istotną poprawę końcowej dokładności zliczania. W praktyce oznacza to, że dla tego detektora główne źródło błędu leży raczej po stronie jakości i stabilności samych predykcji klas niż w niedoskonałości warstwy asocjacji między klatkami.

Istotne jest także to, że eksperyment IV nie mierzy śledzenia w oderwaniu od reszty pipeline'u. Końcowy błąd zliczania jest tutaj wypadkową jakości detektora, odporności *trackera*, filtrowania do obszaru ROI oraz przyjętej polityki zliczania opartej na unikalnych identyfikatorach. Z tego względu najbardziej uzasadniona odpowiedź na pytanie PB4 brzmi następująco: w badanym środowisku najwyższą poprawność zliczania zapewnia kon-

figuracja RF-DETR + ByteTrack, natomiast *trackera* należy oceniać nie jako izolowany algorytm, lecz jako element pełnej konfiguracji detektor–*tracker*.

5.6. Dyskusja wyników — interpretacja, porównanie z literaturą, analiza błędów

Uzyskane wyniki pokazują, że w badanym zadaniu nie istnieje jedna konfiguracja, która byłaby najlepsza jednocześnie pod względem jakości detekcji, szybkości inferencji oraz poprawności końcowego zliczania. Całość eksperymentów ujawnia więc wyraźny kompromis między dokładnością lokalizacji obiektów, przepustowością systemu i stabilnością śledzenia, co dobrze wpisuje się w obserwacje opisywane w literaturze dotyczącej współczesnych systemów detekcji obiektów w zastosowaniach przemysłowych i logistycznych.

Z perspektywy jakości detekcji offline najlepszy wynik osiągnął RF-DETR Nano, uzyskując mAP@0.5 równe 0,763, podczas gdy YOLOv11n osiągnął 0,668, a Faster R-CNN 0,570. Oznacza to, że architektura transformatorowa najskuteczniej radziła sobie z samym zadaniem lokalizacji i klasyfikacji obiektów w statycznej ocenie walidacyjnej. Taki rezultat można interpretować jako potwierdzenie, że podejścia end-to-end oparte na transformerach są szczególnie użyteczne w scenach z częściowymi przesłonięciami, zróżnicowaną skalą obiektów i większą złożonością przestrzenną.

Jednocześnie w ocenie online na GPU zdecydowanie najszybszym modelem okazał się YOLOv11n, osiągając średnio około 193 FPS, wobec około 63 FPS dla Faster R-CNN i około 54 FPS dla RF-DETR Nano. W praktyce oznacza to, że YOLOv11n zapewnia najwyższy zapas wydajności obliczeniowej i z inżynierskiego punktu widzenia stanowi najbardziej użyteczny kompromis dla systemu typu *proof-of-concept* analizującego ciągły strumień wideo. Dodatkowo model ten generował znacznie mniej detekcji na materiałach testowych niż pozostałe architektury, co sugeruje bardziej selektywny profil predykcji i mniejsze ryzyko przekazywania niestabilnych ramek do warstwy śledzenia.

W kontekście zliczania ma to istotne znaczenie, ponieważ nadmiar słabych lub krótkotrwałych detekcji zwiększa prawdopodobieństwo fragmentacji trajektorii i w konsekwencji prowadzi do wielokrotnego policzenia tej samej paczki. Tę interpretację wzmacnia fakt, że Faster R-CNN, który wygenerował najwięcej detekcji i jednocześnie miał najniższe średnie zaufanie predykcji, okazał się najsłabszym kompromisem między jakością a użytecznością czasu rzeczywistego. W badanym scenariuszu klasyczna architektura dwuetapowa pełni więc raczej rolę punktu odniesienia niż praktycznie konkurencyjnego rozwiązania wdrożeniowego.

Analiza wyników per klasa pokazuje również, że małe przesyłki pozostały trudniejsze do wykrywania niż duże dla wszystkich badanych architektur. Luka między klasami `box` i `mailer` w mAP@0.5 wyniosła 7,1 punktu procentowego dla YOLOv11s, 19,0 dla RF-DETR Nano oraz 23,5 dla Faster R-CNN. Jest to zgodne z literaturą dotyczącą detekcji małych obiektów, w której jako główne źródła trudności wskazuje się małą liczbę pikseli opisujących obiekt, częściowe okluzje, szum tła oraz większą podatność na spadki pewności detekcji.

Wyniki eksperymentu skalowania modeli dodatkowo pokazują, że wpływ zwiększenia rozmiaru modelu zależy od architektury bazowej. Dla YOLOv11 przejście z wariantu nano do small przyniosło jedynie marginalny wzrost mAP@0.5 z 0,668 do 0,675, natomiast dla RF-DETR przejście z wariantu nano do small dało wyraźną poprawę z 0,763 do 0,844. Szczególnie istotne jest to, że w przypadku RF-DETR zysk był bardzo wyraźny dla trudniejszej klasy `mailer`, co sugeruje, że dodatkowa pojemność modelu transformatorowego została efektywnie wykorzystana do modelowania drobniejszych i bardziej niestabilnych cech wizualnych.

Osobną warstwę interpretacji wnosi eksperyment śledzenia i zliczania, ponieważ praktyczna wartość systemu zależy nie tylko od jakości detekcji klatkowej, ale także od zachowania tożsamości obiektów w czasie. Wszystkie analizowane konfiguracje wykazały dodatni błąd systematyczny zliczania, a więc system miał systematyczną tendencję do przeszacowywania liczby paczek zamiast ich pomijania. Najlepszy wynik końcowy osiągnęła konfiguracja RF-DETR + ByteTrack, dla której MAE łączne wyniosło 5,60, MAPE 37,2%, a błąd systematyczny 5,20, natomiast najgorsza okazała się konfiguracja Faster R-CNN + SORT z MAE 36,80, MAPE 239,4% i błędem systematycznym 36,80.

Taki profil błędów wskazuje, że dominującym mechanizmem nie było tło generujące losowe fałszywe alarmy, lecz fragmentacja trajektorii i ponowne nadawanie nowych identyfikatorów tym samym fizycznym obiektom. Wynika to bezpośrednio z przyjętej logiki zliczania, w której wynik końcowy opiera się na liczbie unikalnych identyfikatorów *trackera*, a nie na przecięciach linii lub innym zdarzeniu przestrzennym. W takiej konstrukcji każdy ID switch albo chwilowa utrata ścieżki może natychmiast zawyżyć końcowy wynik, podczas gdy błąd przeciwny pojawia się wtedy, gdy różne paczki zostają błędnie złączone w jedną trajektorię [48].

Na tej podstawie można stwierdzić, że dokładność zliczania w badanym zadaniu była ograniczana co najmniej w takim samym stopniu przez spójność czasową śledzenia, jak przez samą jakość lokalizacji przestrzennej. Najprawdopodobniej głównym źródłem niestabilności były małe przesyłki klasy `mailer`, które cechowały się niższą jakością detekcji i większą podatnością na chwilowe spadki confidence. Taka interpretacja pozostaje zgodna zarówno z wynikami tej pracy, jak i z literaturą, według której małe obiekty są szczególnie wrażliwe na częściowe przesłonięcia, deformacje i zakłócenia tła, a *trackery* typu *tracking-by-detection* silnie dziedziczą niestabilność po warstwie detekcyjnej [38].

Wyniki porównania *trackerów* sugerują również, że detektor i *tracker* nie powinny być optymalizowane niezależnie od siebie. W badanym zbiorze *ByteTrack* wyraźnie poprawiał wyniki dla RF-DETR i Faster R-CNN względem SORT, natomiast w przypadku YOLOv11 różnica między ByteTrack i BoT-SORT była niewielka, co wskazuje, że bardziej selektywny i stabilny detektor może ograniczać korzyści z bardziej rozbudowanej asocjacji. Oznacza to, że ostateczna skuteczność konfiguracji zależy nie tylko od klasy algorytmu śledzenia, ale od dopasowania jego mechanizmu asocjacji do charakterystyki błędów konkretnego detektora [9].

Na wiarygodność uzyskanych wyników pozytywnie wpływa fakt, że porównania przeprowadzono w kontrolowanych warunkach eksperymentalnych: z wykorzystaniem tych samych nagrań testowych, wspólnej maski ROI, identycznych progów confidence oraz

tych samych wytrenowanych wag tam, gdzie było to możliwe. Taki sposób prowadzenia eksperymentu jest zgodny z oczekiwaniami wobec części dyskusyjnej pracy dyplomowej, która powinna oceniać wiarygodność wyników i uzasadniać ich interpretację. Jednocześnie porównanie z literaturą należy traktować ostrożnie, ponieważ opublikowane systemy detekcji paczek różnią się konfiguracją kamer, gęstością sceny, sposobem anotacji oraz wykorzystaniem danych RGB, RGB-D lub dodatkowych etapów przetwarzania.

Ostatecznie wartość tej pracy polega nie tyle na wykazaniu uniwersalnej przewagi jednej architektury, ile na dostarczeniu kontrolowanej charakterystyki zachowania konfiguracji detektor–tracker w konkretnym scenariuszu sortowania przesyłek na taśmie transportowej. W świetle uzyskanych wyników można przyjąć, że jeśli priorytetem jest maksymalna szybkość i prostota wdrożenia, najlepszym wyborem pozostaje YOLOv11n, jeśli priorytetem jest najwyższa jakość detekcji offline, najkorzystniej wypada RF-DETR, natomiast jeśli celem jest poprawa samego zliczania, dalsza optymalizacja powinna koncentrować się nie tylko na detektorze, ale również na ograniczaniu fragmentacji ID, kalibracji confidence oraz bardziej odpornej logice liczenia.

Rozdział 6

Wnioski

Gwałtowny rozwój sektora e-commerce wymusza poszukiwanie wysoce wydajnych i zautomatyzowanych rozwiązań dla logistyki oraz sortowni paczek. Niniejsza praca przedstawia opracowanie i ewaluację koncepcyjnego systemu wizyjnego przeznaczonego do detekcji, śledzenia i zliczania przesyłek pocztowych na taśmie transportowej w czasie rzeczywistym, ze szczególnym uwzględnieniem wyzwań wynikających z wizualnej asymetrii oraz wzajemnych okluzji pomiędzy małymi, odkształcalnymi przesyłkami (mailer) a dużymi, sztywnymi paczkami. Przeprowadzone eksperymenty wykazały, że wybór architektury detektora powinien być traktowany jako kompromis projektowy pomiędzy jakością detekcji, szybkością inferencji a stabilnością zliczania w strumieniu wideo.

Systematyczne porównanie trzech paradygmatów detekcji — jednoetapowego, opartego na transformatorach *end-to-end* oraz dwuetapowego — uwydatniło krytyczny *trade-off* pomiędzy przepustowością obliczeniową a dokładnością lokalizacji. Uzyskane wyniki pokazują, że podczas gdy modele transformatorowe osiągają wyższą dokładność i odporność w gęstych scenach z dużym stopniem okluzji, architektury jednoetapowe pozostają najbardziej pragmatycznym wyborem dla szybkich aplikacji logistycznych ze względu na swoją wyjątkową szybkość wnioskowania i skalowalność. Zauważono również, że mniejsze i wizualnie mniej wyraziste paczki pozostają najbardziej problematycznym elementem analizy, co jest spójne z aktualnymi badaniami nad utrwalonymi trudnościami w detekcji małych obiektów w rzeczywistych scenach.

Niezawodne zliczanie zależy nie tylko od dokładności detekcji na poziomie pojedynczej klatki, ale także od zachowania tożsamości obiektu w czasie, co podkreśla praktyczne znaczenie solidnych potoków *tracking-by-detection* w logistycznej analizie wideo. Ewaluacja algorytmów śledzenia udowodniła, że asocjacja detekcji o niskiej pewności jest kluczowa w ograniczaniu fragmentacji trajektorii i redukcji błędu zliczania. Ponadto, ocena międzyplatformowa potwierdziła, że podczas gdy lekkie modele mogą utrzymać użyteczność operacyjną na standardowych procesorach CPU, wdrożenie cięższych architektur bezwzględnie wymaga akceleracji GPU, aby spełnić rygorystyczne ograniczenia przepustowości czasu rzeczywistego nakładane przez nowoczesne systemy przenośnikowe. Zestawienie tych wyników stanowi podstawę empiryczną dla weryfikacji hipotezy badawczej i szczegółowych wniosków przedstawionych w kolejnych podrozdziałach.

6.1. Podsumowanie osiągniętych celów

Wszystkie podstawowe cele badawcze i implementacyjne sformułowane w pracy zostały zrealizowane. Opracowano kompletny potok przetwarzania obejmujący przygotowanie i adnotację danych, trening modeli YOLOv11, RF-DETR oraz Faster R-CNN, ich integrację z algorytmami śledzenia wieloobiektowego ByteTrack, BoT-SORT i SORT, a także logikę zliczania i ewaluację całego rozwiązania. Całość uzupełniono dedykowaną aplikacją webową wspierającą orkiestrację eksperymentów, definiowanie obszaru ROI oraz wizualne porównywanie wyników. Uzyskane rezultaty stanowią bezpośrednią podstawę do weryfikacji hipotezy badawczej oraz sformułowania ograniczeń pracy i kierunków dalszego rozwoju systemu, omówionych w kolejnych podrozdziałach.

Oryginalny wkład własny niniejszej pracy obejmuje cztery elementy: (1) własny zbiór danych z 395 obrazami i 1403 ręcznymi adnotacjami dla domeny przesyłek pocztowych, niepochodzący z żadnego publicznie dostępnego benchmarku; (2) modularną architekturę pipeline'u z ujednoliconą warstwą normalizacji wyjść detektorów, umożliwiającą porównywalne zestawienie heterogenicznych konfiguracji detektor–tracker w jednym środowisku; (3) usystematyzowaną metodologię eksperymentalną obejmującą cztery niezależne eksperymenty oceniające jednocześnie jakość detekcji offline, wydajność inferencji i błąd końcowego zliczania; (4) dedykowaną aplikację webową integrującą trening, inferencję, interaktywny edytor ROI i wizualne porównywanie wyników konfiguracji.

6.2. Odpowiedź na hipotezę badawczą

Postawiona w pracy hipoteza badawcza została potwierdzona, choć jej interpretacja wymaga kilku istotnych doprecyzowań. Przeprowadzone eksperymenty wykazały, że możliwe jest opracowanie systemu detekcji, śledzenia i zliczania przesyłek, opartego na metodach głębokiego uczenia, który osiąga praktycznie użyteczną skuteczność oraz wydajność w kontrolowanym scenariuszu pracy na taśmie transportowej. Jednocześnie wyniki pokazały, że końcowa jakość rozwiązania nie jest cechą każdej testowanej konfiguracji, lecz wynika z właściwego doboru architektury detektora, wariantu modelu, algorytmu śledzenia oraz platformy obliczeniowej. Wniosek ten znajduje potwierdzenie w wynikach przedstawionych między innymi w Tabelach 5.1, 5.2, 5.4, 5.6, 5.7 i 5.10 oraz na Rysunkach 5.2, 5.5 i 5.6.

Z perspektywy samych modeli detekcji hipoteza została zweryfikowana pozytywnie. RF-DETR osiągnął najwyższą jakość detekcji offline, co wskazuje, że architektura transformatorowa dobrze radzi sobie z modelowaniem relacji przestrzennych oraz z trudniejszymi przypadkami, takimi jak częściowe okluzje i większe zagęszczenie obiektów. Z kolei YOLOv11n okazał się najbardziej korzystnym rozwiązaniem z punktu widzenia kompromisu pomiędzy dokładnością a szybkością działania, oferując bardzo wysoką przepustowość inferencji przy nadal satysfakcjonującej jakości detekcji, co czyni go szczególnie atrakcyjnym w zastosowaniach zbliżonych do czasu rzeczywistego. Faster R-CNN w badanym scenariuszu wypadł najsłabiej jako rozwiązanie praktyczne, ponieważ oferował mniej korzystny profil jakościowo-wydajnościowy niż pozostali konkurenci.

Wyniki eksperymentu dotyczącego skalowania modeli potwierdziły, że wpływ rozmiaru modelu jest silnie zależny od rodziny architektury. W przypadku RF-DETR przejście z wariantu nano do small przyniosło wyraźną poprawę metryk jakości, szczególnie dla klasy mailer, co sugeruje, że większa pojemność modelu została efektywnie wykorzystana do reprezentacji trudniejszych, małych i bardziej deformowalnych przesyłek. Odmienną charakterystykę zaobserwowano dla YOLOv11, gdzie wzrost rozmiaru modelu przełożył się jedynie na marginalny zysk w metrykach offline, a jednocześnie nie dawał jednoznacznej korzyści w scenariuszu praktycznego zliczania, dlatego większy model nie może być automatycznie uznany za lepszy wybór wdrożeniowy.

Istotna część hipotezy dotycząca platform sprzętowych również została potwierdzona. Porównanie CPU i GPU wykazało wyraźny podział między konfiguracjami lekkimi i cięższymi: modele bardziej wymagające obliczeniowo potrzebują akceleracji GPU, natomiast lekkie konfiguracje oparte na YOLOv11 zachowują operacyjną użyteczność także na CPU, choć kosztem niższej przepustowości. Oznacza to, że praktyczna możliwość wdrożenia zależy nie tylko od jakości modelu, ale również od dostępnego budżetu obliczeniowego i docelowego środowiska pracy systemu.

Uzyskane wyniki potwierdziły również, że końcowa dokładność zliczania zależy nie tylko od jakości detekcji, lecz także od stabilności śledzenia i poprawnej asocjacji obiektów między klatkami. Najlepsze wyniki zliczania osiągnęła konfiguracja łącząca RF-DETR z ByteTrack, co potwierdza, że wykorzystanie detekcji o niższej pewności może skutecznie ograniczać fragmentację trajektorii i zmniejszać ryzyko wielokrotnego zliczenia tej samej przesyłki. W konsekwencji hipotezę badawczą należy uznać za potwierdzoną w zakresie przyjętego scenariusza eksperymentalnego. Jednocześnie uzyskanych wniosków nie należy bezpośrednio uogólniać na inne sortownie, konfiguracje kamer, prędkości taśmy czy rozkłady przesyłek bez przeprowadzenia dodatkowej walidacji.

6.3. Ograniczenia systemu i pracy

Pomimo uzyskanych wyników, opracowany system oraz przeprowadzone badania podlegają szeregowi ograniczeń, które można zidentyfikować na różnych poziomach: dotyczących zbioru danych, architektury systemu, zakresu eksperymentalnego oraz kontekstu wdrożeniowego.

Ograniczenia zbioru danych i generalizacji. Zbiór danych wykorzystany w niniejszej pracy został zgromadzony na podstawie publicznie dostępnych źródeł wideo, a nie w ramach kontrolowanej procedury próbkowania. W rezultacie kolekcja ta może zawierać nierównomierną reprezentację określonych typów przesyłek, powtarzające się wzorce wizualne oraz artefakty specyficzne dla źródeł. Modele trenowane na takich danych mogą wykazywać pewien stopień obciążenia, który trudno jest wykryć w standardowej ocenie offline, lecz może ujawnić się jako spadek wydajności przy przeniesieniu systemu do innej sortowni, na inną konfigurację przenośnika lub w odmiennych warunkach oświetlenia. Obawę tę potęguje klasa mailer, obejmująca szeroki zakres odkształcalnych obiektów o niskim kontraście — cienkie koperty bąbelkowe, foliopaki oraz niewielkie kartony — których wygląd wizualny może znacząco różnić się w zależności od stopnia wypełnienia,

ułożenia na taśmie oraz kąta padania światła przemysłowego. Spójność adnotacji stanowi dodatkowy czynnik ograniczający: ponieważ granica między klasami box i mailer nie jest zawsze geometrycznie ostra, jakakolwiek niespójność w etykietowaniu wprowadza niejednoznaczność, która przejawia się niestabilnymi predykcjami i mniej wiarygodną oceną ilościową.

Trudności detekcji małych obiektów. Małe przesyłki pozostają najtrudniejszym elementem zadania detekcji we wszystkich przeprowadzonych eksperymentach. Obiekty tego typu zajmują ograniczony obszar obrazu, zawierają mniej szczegółów teksturalnych, słabsze krawędzie oraz niższy stosunek sygnału do szumu, co zwiększa wrażliwość na rozmycie ruchu, artefakty kompresji oraz błędy adnotacji. Ponadto, część klasy mailer składa się z odkształcalnych opakowań, które nie zachowują stałego kształtu między klatkami, co oznacza, że kontur obiektu może znacząco zmieniać się nawet w obrębie jednej sekwencji wideo, a refleksy powierzchni foliowych mogą dodatkowo utrudniać ekstrakcję cech. Wyzwania te są zgodne z dobrze udokumentowanymi wynikami w literaturze dotyczącej detekcji małych obiektów, gdzie luka w skuteczności między małymi a dużymi obiektami pozostaje jednym z najtrwalszych nierozwiązanych problemów w tej dziedzinie.

Błędy śledzenia i zliczania. Dokładność zliczania paczek zależy nie tylko od jakości detekcji w pojedynczych klatkach, lecz również od stabilności warstwy śledzenia. Wszystkie trzy *trackery* wykorzystane w tej pracy — SORT, ByteTrack oraz BoT-SORT — należą do paradygmatu *tracking-by-detection*, co oznacza, że przedłużające się braki detekcji, silne przesłonięcia lub gęste sąsiedztwo paczek mogą nadal prowadzić do fragmentacji trajektorii oraz przełączeń identyfikatorów, które bezpośrednio przekładają się na błędy zliczania. Strategia ByteTrack polegająca na włączaniu detekcji o niższej ufności zmniejsza, lecz nie eliminuje tego ryzyka; w silnie zatłoczonych scenach nawet metody wykorzystujące cechy wyglądu mogą mieć trudności z utrzymaniem spójnych tożsamości. Logika zliczania oparta na przekraczaniu wirtualnej linii jest wrażliwa na konfigurację ROI: nieprawidłowo umieszczona linia zliczająca lub ROI nieuwzględniająca pełnego zasięgu przestrzennego taśmy może wprowadzać systematyczne nadliczby lub niedoliczby, które nie znajdują odzwierciedlenia w metryce mAP. Z metodologicznego punktu widzenia praca koncentruje się na dokładności końcowego zliczania oraz metrykach na poziomie detektora, nie obejmuje jednak pełnych adnotacji klatka po klatce wymaganych dla rygorystycznej oceny śledzenia wielu obiektów, takich jak MOTA czy IDF1. W rezultacie ocena dostarcza wiarygodnego porównania praktycznej wydajności zliczania w badanym scenariuszu, jednak oferuje mniej pełny obraz długoterminowej spójności trajektorii niż pełny benchmark MOT.

Zakres eksperymentalny i generalizowalność. Eksperymenty zostały przeprowadzone dla określonego zbioru danych, konkretnej konfiguracji kamery oraz kontrolowanego środowiska rejestracji, co ogranicza bezpośrednią przenoszalność wyników na inne środowiska przemysłowe. Ocenie poddano jedynie trzy architektury detekcji (YOLOv11, RF-DETR, Faster R-CNN) oraz trzy algorytmy śledzenia (SORT, ByteTrack, BoT-SORT); pełna przestrzeń dostępnych metod jest znacznie szersza, a uzyskane wyniki mogą nie generalizować się na architektury nieobjęte porównaniem. W szczególności nowsze modele detekcyjne, podejścia wielomodalne łączące dane RGB z informacją o głębi lub ar-

chitektury *end-to-end* detekcji i śledzenia nie zostały zbadane, a wszystkie te kierunki reprezentują aktywne i obiecujące obszary badań.

Charakter *proof-of-concept*. Wszystkie eksperymenty zostały przeprowadzone w zamkniętym, kontrolowanym środowisku laboratoryjnym na łącznie 966 klatkach z pięciu nagrań wideo, co stanowi istotne ograniczenie interpretowalności wyników. Po pierwsze, ocena nie obejmuje zachowania systemu w trybie ciągłej pracy — nie wiadomo, czy stabilność FPS i dokładność zliczania utrzymują się po wielogodzinnej inferencji, gdy mogą wystąpić takie zjawiska jak wyciek pamięci czy dryfowanie rozkładu predykcji. Po drugie, wnioskowanie wykonywano na plikach wideo, a nie na żywym strumieniu z kamery; opóźnienia wynikające z bufora kamery i transferu danych mogą obniżyć mierzone wartości FPS w rzeczywistym wdrożeniu. Po trzecie, pomiary wydajności (m.in. 193 FPS dla YOLOv11n oraz 54 FPS dla RF-DETR Nano na GPU) uzyskano na dedykowanym akceleratorze bez dodatkowych obciążeń, podczas gdy w środowisku produkcyjnym GPU może być współdzielone z innymi procesami operacyjnymi. Uzyskane wartości metryk offline (mAP) oraz online (FPS) należy zatem traktować jako punkty odniesienia do porównania architektur w ramach zdefiniowanego eksperymentu, a nie jako prognozy rzeczywistej przepustowości systemu produkcyjnego. Brak integracji z zewnętrzną infrastrukturą operacyjną — czytnikami kodów kreskowych, modułami OCR, systemami zarządzania magazynem czy mechanizmami kierowania ruchem — jest celową decyzją zakresu pracy, która dodatkowo wzmacnia ograniczenie przenoszalności wyników: zachowanie systemu po podłączeniu do infrastruktury logistycznej może różnić się od tu prezentowanego ze względu na dodatkowe obciążenie obliczeniowe oraz konieczność obsługi wyjątków nieobecnych w nagraniach testowych.

Zależność sprzętowa i środowisko inferencji. Wnioskowanie z akceleracją GPU zostało zmierzone na pojedynczej karcie graficznej w środowisku laboratoryjnym. Wartości wydajności mogą różnić się na innych konfiguracjach sprzętowych, we wdrożeniach zwirtualizowanych lub konteneryzowanych, lub gdy GPU jest współdzielony z innymi procesami. Wyniki inferencji na CPU demonstrują, że lekkie konfiguracje mogą zachować użyteczność operacyjną na procesorach, jednak zmierzona przepustowość może nie być wystarczająca dla wysokowydajnych przemysłowych przenośników obsługujących większe gęstości paczek niż te uwzględnione w testach. Techniki kwantyzacji, przycinania lub destylacji modeli — które mogłyby znacząco poprawić przepustowość CPU — nie zostały zastosowane w tej pracy i stanowią naturalny kierunek przyszłej optymalizacji.

Podsumowując, przedstawiony system należy traktować jako zwalidowaną platformę eksperymentalną oraz solidną podstawę do dalszego rozwoju, a nie jako uniwersalnie wdrażalne rozwiązanie dla każdej instalacji przemysłowej. Uzyskane wyniki są wiążące przede wszystkim dla badanej konfiguracji eksperymentalnej i powinny stanowić punkt wyjścia do dalszych badań nad uogólnieniem zaproponowanego podejścia na inne środowiska operacyjne.

6.4. Kierunki dalszego rozwoju

Przedstawione w pracy wyniki stanowią solidny dowód słuszności koncepcji dla systemu detekcji i zliczania paczek pocztowych. Mimo zadowalających rezultatów istnieje kilka obiecujących kierunków badawczych, których realizacja mogłaby znacząco zwiększyć wartość praktyczną systemu i przybliżyć go do wdrożenia produkcyjnego.

Trening modelu i implementacja dla konkretnej sortowni. Kluczowym krokiem na drodze do wdrożenia przemysłowego jest zebranie danych i wytrenowanie modelu na materiale pochodzącym bezpośrednio z docelowej sortowni — z jej specyficzną konfiguracją kamer, oświetleniem, typami paczek i prędkością taśmy. Modele trenowane i ewaluowane w niniejszej pracy opierały się na zbiorze danych zgromadzonym na zasadzie dostępności źródeł, bez kontrolowanej procedury próbkowania, co ogranicza bezpośrednią przenaszalność wyników do innych środowisk operacyjnych. Metody nienadzorowanej i półnadzorowanej adaptacji dziedzinowej — w tym podejścia oparte na wyrównaniu cech oraz transformatorowe detektory z adaptacją domenową — wykazały obiecujące rezultaty w redukcji kosztownego re-etykietowania danych przy wdrożeniu w nowych warunkach [17]. W praktyce wzbogacanie zbioru bez konieczności ręcznego etykietowania tysięcy klatek umożliwiają modele generatywne, zdolne do syntetyzowania trudnych warunków oświetleniowych, ekstremalnego rozmycia ruchu czy deformacji kopert bąbelkowych, co czyni system znacznie odporniejszym na rzadkie anomalie wizualne.

Obsługa małych, odkształcalnych i częściowo zasłoniętych paczek. W przeprowadzonych eksperymentach cienkie koperty bąbelkowe oraz foliowe przesyłki stanowiły najtrudniejsze przypadki detekcji. Przyszłe badania powinny skupić się na treningu przy wyższych rozdzielczościach wejściowych, wzmocnionej agregacji cech wieloskalowych, celowanych politykach augmentacji oraz strategiach wyrównywania klas. Warto również zbadać, czy specjalistyczne procedury treningu dla trudnych przykładów zmniejszą liczbę fałszywych negatywów w zatłoczonych scenach. Perspektywicznie rozważenia wymaga zastąpienie lub uzupełnienie głowicy detekcji o segmentację instancji — rozwiązania takie jak Mask R-CNN dostarczają konturów obiektów na poziomie pikseli, co znacząco ułatwia separację stykających się lub nakładających się przesyłek.

Optymalizacja pod kątem wdrożeń na urządzeniach o ograniczonych zasobach. Jak wykazał Eksperyment III, konfiguracja sprzętowa ma istotny wpływ na praktyczną użyteczność systemu. Przyszłe badania powinny systematycznie porównywać techniki kompresji modeli — kwantyzację, przycinanie wag (ang. *pruning*) oraz destylację wiedzy — w celu ograniczenia kosztu inferencji w środowiskach CPU lub na urządzeniach brzegowych (ang. *edge*) pozbawionych dedykowanego GPU. Wyniki opisane w literaturze wskazują, że połączenie rzadkościowego treningu, przycinania kanałów i destylacji może podnieść przepustowość z ok. 26 FPS do ponad 68 FPS na sprzęcie o ograniczonych zasobach — bez istotnego spadku jakości predykcji [36]. Równie obiecujące są lekkie architektury transformatorowe ukierunkowane na inferencję w czasie rzeczywistym.

Fuzja sensoryczna i dane głębi. Obecny system opiera się wyłącznie na monofonicznym wideo RGB. Istotnym kierunkiem przyszłych prac jest integracja kamer RGB-D lub sensorów ze strukturyzowanym oświetleniem, które dostarczają estymaty głębi dla każdego piksela. Badania w zakresie detekcji paczek na taśmach transportowych poka-

zują, że detekcja granic oparta na RGB-D znacząco poprawia obsługę gęsto ułożonych, nieuporządkowanych lub silnie zasłoniętych przesyłek [15]. Analiza trzeciego wymiaru ułatwiłaby ponadto separację gęsto ułożonych foliopaków, w przypadku których dwuwymiarowe ramki ograniczające często znacząco na siebie nachodzą, destabilizując pracę trackerów takich jak ByteTrack czy BoT-SORT.

Śledzenie wielokamerowe i warstwa śledzenia. Przesyłki poruszające się w sortowni nierzadko ulegają długotrwałym zasłonięciom, a pojedyncza kamera nie zawsze zapewnia wystarczające pokrycie sceny. Architektura transformatorowa *end-to-end* umożliwiająca jednoczesne korelowanie obrazów z wielu perspektyw pozwoliłaby globalnie podtrzymywać tożsamość przesyłki na całej linii produkcyjnej [25]. Jednocześnie warstwa śledzenia i zliczania oferuje wyraźne możliwości doskonalenia: przyszłe prace powinny uzupełnić ocenę opartą na zliczaniu o pełniejsze metryki MOT oraz szczegółowo przeanalizować, w jaki sposób przełączenia tożsamości, krótkotrwałe zasłonięcia i fragmentacja trajektorii wpływają na końcowy wynik biznesowy.

Integracja z infrastrukturą logistyczną. Opracowana aplikacja celowo pomija integrację z zewnętrznymi systemami operacyjnymi. Naturalnym kierunkiem wdrożenia przemysłowego jest połączenie wizualnego potoku zliczania i detekcji z systemami zarządzania magazynem (WMS), czytnikami kodów kreskowych i modułami OCR, ramionami robotycznymi oraz automatycznymi mechanizmami kierowania ruchem przesyłek. Wyniki badań nad kompleksowymi inteligentnymi systemami przenośnikowymi potwierdzają wykonalność integracji detekcji opartej na głębokim uczeniu ze zliczaniem w czasie rzeczywistym, klasyfikacją kształtów i monitorowaniem przepływu w ramach jednego zunifikowanego potoku przemysłowego. Taka integracja przekształciłaby obecny system *proof-of-concept* w pełnoprawny element nowoczesnego zautomatyzowanego centrum sortowniczego.

Literatura

[1] Aharon, N., Orfaig, R., Bobrovsky, B.-Z.:
BoT-SORT: Robust Associations Multi-Pedestrian Tracking. 2022.
arXiv: 2206.14651 [cs.CV]. URL: https://arxiv.org/abs/2206.14651.

[2] Aldubaikhi, A., Patel, S.: "Advancements in Small-Object Detection (2023–2025): Approaches, Datasets, Benchmarks, Applications, and Practical Guidance".
W: *Applied Sciences* 15.22 (2025), s. 11882. DOI: 10.3390/app152211882.
URL: https://www.mdpi.com/2076-3417/15/22/11882 (term. wiz. 29. 04. 2026).

[3] Balakrishna, S., Yadao, S., Solanki, V. K.:
"Enhancing YOLOv11 for Real-Time Object Detection: Advanced Architectures and Edge-Optimized Training Pipeline".
W: *Annals of Computer Science and Information Systems*. T. 42.
Warsaw: Polish Information Processing Society (PTI), 2024, s. 89–96.
DOI: 10.15439/2024R115.
URL: https://annals-csis.org/Volume_42/drp/pdf/115.pdf.

[4] Bashar, M., Islam, S., Hussain, K. K., Hasan, M. B., Rahman, A. B. M. A., Kabir, M. H.:
In Pursuit of Many: A Review of Modern Multiple Object Tracking Systems. 2026.
arXiv: 2209.04796 [cs.CV]. URL: https://arxiv.org/abs/2209.04796.

[5] Bewley, A., Ge, Z., Ott, L., Ramos, F., Upcroft, B.:
"Simple Online and Realtime Tracking".
W: *2016 IEEE International Conference on Image Processing (ICIP)*.
Phoenix, AZ: IEEE, 2016, s. 3464–3468. DOI: 10.1109/ICIP.2016.7533003.

[6] Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., Zagoruyko, S.:
"End-to-End Object Detection with Transformers".
W: *Computer Vision – ECCV 2020*. Lecture Notes in Computer Science.
Cham: Springer International Publishing, 2020, s. 213–229.
DOI: 10.1007/978-3-030-58452-8_13. URL: https://www.ecva.net/papers/eccv_2020/papers_ECCV/html/832_ECCV_2020_paper.php.

[7] Crasto, N.: *Class Imbalance in Object Detection: An Experimental Diagnosis and Study of Mitigation Strategies*. 2024. arXiv: 2403.07113 [cs.CV].
URL: https://arxiv.org/abs/2403.07113.

[8] Dalal, N., Triggs, B.: "Histograms of Oriented Gradients for Human Detection".
W: *Proceedings of the 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*. T. 1.

Washington, DC: IEEE Computer Society, 2005, s. 886–893.
DOI: 10.1109/CVPR.2005.177.

[9] Del Pup, L.: "Object Tracking Evaluation: BoT-SORT & ByteTrack with YOLOv8. A Comparison of Accuracy and Computational Efficiency". Prac. mag. Stockholm: KTH Royal Institute of Technology, 2024. URL: https://www.diva-portal.org/smash/get/diva2:1886982/FULLTEXT01.pdf (term. wiz. 27. 04. 2026).

[10] Dendorfer, P., Ošep, A., Milan, A., Schindler, K., Cremers, D., Reid, I., Roth, S., Leal-Taixé, L.: "MOTChallenge: A Benchmark for Single-Camera Multiple Target Tracking". W: *International Journal of Computer Vision* 129.4 (2021), s. 845–881. DOI: 10.1007/s11263-020-01393-0.

[11] Fabbrizzi, S., Papadopoulos, S., Ntoutsi, E., Kompatsiaris, I.: "A Survey on Bias in Visual Datasets". W: *Computer Vision and Image Understanding* 223, art. 103552 (2022). DOI: 10.1016/j.cviu.2022.103552.

[12] Gao, K., Wang, X., Xu, Y., Sładkowski, A.: "Intelligent Logistics Express Parcel Real-Time Detection System Based on Improved YOLOv8". W: *Transport Problems* 20.1 (2025). DOI: 10.20858/tp.2025.20.1.15. URL: http://transportproblems.polsl.pl/pl/archiwum/2025/zeszyt1/2025t20z1_15.pdf (term. wiz. 27. 04. 2026).

[13] InPost: *Trading Update for Q4 and FY2025*. Sty. 2026. URL: https://inpost.eu/sites/default/files/2026-01/Trading%20update%20for%20Q4%202025%20and%20FY25_1.pdf (term. wiz. 23. 04. 2026).

[14] Khanam, R., Hussain, M.: *YOLOv11: An Overview of the Key Architectural Enhancements*. 2024. arXiv: 2410.17725 [cs.CV]. URL: https://arxiv.org/abs/2410.17725.

[15] Kim, S., Truong, V. D., Lee, K.-H., Yoon, J.: "Revolutionizing Robotic Depalletizing: AI-Enhanced Parcel Detecting with Adaptive 3D Machine Vision and RGB-D Imaging for Automated Unloading". W: *Sensors* 24.5 (2024). DOI: 10.3390/s24051476.

[16] Kishor, R.: "Performance Benchmarking of YOLOv11 Variants for Real-Time Delivery Vehicle Detection: A Study on Accuracy, Speed, and Computational Trade-Offs". W: *Asian Journal of Research in Computer Science* 17.12 (2024), s. 108–122. DOI: 10.9734/ajrcos/2024/v17i12532.

[17] Li, H., Zhang, R., Yao, H., Zhang, X., Hao, Y., Song, X., Li, X., Zhao, Y., Li, L., Chen, Y.: "DA-Ada: Learning Domain-Aware Adapter for Domain Adaptive Object Detection". W: *Advances in Neural Information Processing Systems (NeurIPS)*. 2024. URL: https://proceedings.neurips.cc/paper_files/paper/2024/file/bb71b5567ee985e0a4cee54ade19275c-Paper-Conference.pdf.

[18] Liu, L., Ouyang, W., Wang, X., Fieguth, P., Chen, J., Liu, X., Pietikäinen, M.: "Deep Learning for Generic Object Detection: A Survey". W: *International Journal of Computer Vision* 128.2 (2020), s. 261–318. DOI: 10.1007/s11263-019-01247-4.

[19] Lowe, D. G.: "Distinctive Image Features from Scale-Invariant Keypoints". W: *International Journal of Computer Vision* 60.2 (2004), s. 91–110. DOI: 10.1023/B:VISI.0000029664.99615.94.

[20] Lu, Z., Dai, N., Hu, X., Xu, K., Yuan, Y.: "Research on High-Speed Classification and Location Algorithm for Logistics Parcels Based on a Monocular Camera". W: *Scientific Reports* 14.1, art. 15901 (2024). DOI: 10.1038/s41598-024-66941-x. URL: https://www.nature.com/articles/s41598-024-66941-x (term. wiz. 26. 04. 2026).

[21] Ma, J., Ushiku, Y., Sagara, M.: "The Effect of Improving Annotation Quality on Object Detection Datasets: A Preliminary Study". W: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*. New Orleans, LA: IEEE, czer. 2022, s. 4850–4859. DOI: 10.1109/CVPRW56347.2022.00532. URL: https://openaccess.thecvf.com/content/CVPR2022W/VDU/html/Ma_The_Effect_of_Improving_Annotation_Quality_on_Object_Detection_Datasets_A_Preliminary_Study_CVPRW_2022_paper.html (term. wiz. 29. 04. 2026).

[22] Ma, Z., Chen, C., Chen, J., Jiang, Y.: "STC-SORT: A Dynamic Spatio-Temporal Consistency Framework for Multi-Object Tracking in UAV Videos". W: *Applied Sciences* 16.2, art. 1062 (2026). DOI: 10.3390/app16021062. URL: https://www.mdpi.com/2076-3417/16/2/1062 (term. wiz. 07. 05. 2026).

[23] Meest Post: *Report: Where Did Poles Send Parcels in 2025 and What Was Inside Them?* Lut. 2026. URL: https://www.meestpost.com/en/pl/news/what-s-new/report-where-did-poles-send-parcels-in-2025-and-what-was-inside-them (term. wiz. 23. 04. 2026).

[24] Naumann, A., Hertlein, F., Dörr, L., Furmans, K.: "TAMPAR: Visual Tampering Detection for Parcel Logistics in Postal Supply Chains". W: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. Waikoloa, HI: IEEE, sty. 2024, s. 8061–8071. URL: https://openaccess.thecvf.com/content/WACV2024/papers/Naumann_TAMPAR_Visual_Tampering_Detection_for_Parcel_Logistics_in_Postal_Supply_Chains_WACV_2024_paper.pdf (term. wiz. 29. 04. 2026).

[25] Niculescu-Mizil, A., Patel, D., Melvin, I.: "MCTR: Multi Camera Tracking Transformer". W: *Proceedings of the Winter Conference on Applications of Computer Vision (WACV) Workshops*. Lut. 2025, s. 874–884. URL: https://openaccess.thecvf.com/content/WACV2025W/RWS/html/Niculescu-

Mizil_MCTR_Multi_Camera_Tracking_Transformer_WACVW_2025_paper.html (term. wiz. 07. 05. 2026).

[26] Niu, S., Chen, Q., de Silva, C. W.: "Visual Sorting of Express Parcels Based on Multi-Task Deep Learning". W: *Sensors* 20.23, art. 6971 (2020). DOI: 10.3390/s20236971.

[27] Oquab, M., Darcet, T., Moutakanni, T., Vo, H. V., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El-Nouby, A., Assran, M., Ballas, N., Galuba, W., Howes, R., Huang, P.-Y., Li, S.-W., Misra, I., Rabbat, M., Sharma, V., Synnaeve, G., Xu, H., Jégou, H., Mairal, J., Labatut, P., Joulin, A., Bourdoukan, D.: "DINOv2: Learning Robust Visual Features without Supervision". W: *Transactions on Machine Learning Research* (2024). URL: https://openreview.net/forum?id=a68SUt6zFt (term. wiz. 07. 05. 2026).

[28] Park, K., Pal, A., Mok, J. H.: *A Two-Stage Detection-Tracking Framework for Stable Apple Quality Inspection in Dense Conveyor-Belt Environments*. 2026. arXiv: 2602.19278 [cs.CV]. URL: https://arxiv.org/abs/2602.19278.

[29] PyTorch Contributors: *torchvision.models.detection.fasterrcnn_mobilenet_v3_large_fpn*. PyTorch Vision Documentation. 2024. URL: https://docs.pytorch.org/vision/main/models/generated/torchvision.models.detection.fasterrcnn_mobilenet_v3_large_fpn.html (term. wiz. 30. 04. 2026).

[30] Ren, S., He, K., Girshick, R., Sun, J.: "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks". W: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 39.6 (2017), s. 1137–1149. DOI: 10.1109/TPAMI.2016.2577031.

[31] Robinson, I., Robicheaux, P., Popov, M., Ramanan, D., Peri, N.: "RF-DETR: Neural Architecture Search for Real-Time Detection Transformers". W: *The Thirteenth International Conference on Learning Representations (ICLR 2026)*. OpenReview.net, 2026. URL: https://openreview.net/forum?id=qHm5GePxTh (term. wiz. 24. 04. 2026).

[32] Roboflow: *Supervision*. Dokumentacja biblioteki Supervision, wersja 0.27.x. Roboflow Documentation. 2023. URL: https://supervision.roboflow.com (term. wiz. 01. 05. 2026).

[33] Roboflow: *RF-DETR: SOTA Real-Time Object Detection Model*. 2025. URL: https://github.com/roboflow/rf-detr (term. wiz. 23. 04. 2026).

[34] Roboflow: *Train an RF-DETR Model*. 2025. URL: https://blog.roboflow.com/train-deploy-rf-detr/ (term. wiz. 30. 04. 2026).

[35] Roboflow: *Roboflow: Computer Vision Tools for Developers and Enterprises*. Roboflow. 2026. URL: https://roboflow.com/ (term. wiz. 15. 05. 2026).

[36] Sabaghian, M., Keyvanrad, M. A., Moghadami, S. M.: *A Novel Compression Framework for YOLOv8: Achieving Real-Time Aerial Object*

Detection on Edge Devices via Structured Pruning and Channel-Wise Distillation. 2025. arXiv: 2509.12918 [cs.CV]. URL: https://arxiv.org/abs/2509.12918.

[37] Salunke, A. A.: "Boosting Object Detection Accuracy: A Comparative Study of Image Augmentation Techniques". W: *Global Journal of Computer Science and Technology* 23.1 (2023). URL: https://globaljournals.org/GJCST_Volume23/1-Boosting-Object-Detection-Accuracy.pdf (term. wiz. 30. 04. 2026).

[38] Santhoshkumar, R., Ramachandran, M., Ramesh, A., Banerjee, S.: "Comparative Evaluation of SORT, DeepSORT, and ByteTrack for Multiple Object Tracking in Highway Timelapse Videos". W: *International Journal of Sensor, Image, and Communication Systems* 1.1 (2023), s. 1–11. URL: https://vectoral.org/index.php/IJSICS/article/view/97 (term. wiz. 28. 04. 2026).

[39] Sata, S. P., Weidmann, M., Frasch, S., Albrecht, M., Westphal, H., Pusch, M. i in.: "Identification and Tracking of Multiple Objects on a Peristaltic Conveyor System". W: *2025 5th International Conference on Robotics, Automation and Artificial Intelligence (RAAI)*. Piscataway, NJ: IEEE, 2025, s. 309–317.

[40] Streamlit: *Caching*. Streamlit Documentation. 2025. URL: https://docs.streamlit.io/develop/concepts/architecture/caching (term. wiz. 01. 05. 2026).

[41] Tariq, M. F., Javed, M. A.: *Small Object Detection with YOLO: A Performance Analysis Across Model Versions and Hardware*. 2025. arXiv: 2504.09900 [cs.CV]. URL: https://arxiv.org/abs/2504.09900.

[42] Ultralytics: *YOLO11*. 2024. URL: https://docs.ultralytics.com/models/yolo11/ (term. wiz. 24. 04. 2026).

[43] Ultralytics: *Model Training with Ultralytics YOLO*. 2026. URL: https://docs.ultralytics.com/modes/train/ (term. wiz. 30. 04. 2026).

[44] Wang, H., Wang, Q., Yang, F., Zhang, W., Zuo, W.: *Data Augmentation for Object Detection via Progressive and Selective Instance-Switching*. 2019. arXiv: 1906.00358 [cs.CV]. URL: https://arxiv.org/abs/1906.00358.

[45] Wu, J., Duan, Y., Miao, Z. i in.: "Research on an Algorithm of Express Parcel Sorting Based on Deeper Learning and Multi-Information Recognition". W: *Computational Intelligence and Neuroscience* 2022 (2022), s. 1–11. DOI: 10.1155/2022/8040670. URL: https://pmc.ncbi.nlm.nih.gov/articles/PMC9460149/ (term. wiz. 07. 05. 2026).

[46] Wu, X., Sahoo, D., Hoi, S. C. H.: *Recent Advances in Deep Learning for Object Detection*. 2019. arXiv: 1908.03673 [cs.CV]. URL: https://arxiv.org/abs/1908.03673.

[47] Zebra Technologies: *Improving Every Process: The Recipe for High-Performance Warehousing*. 2025. URL: https://www.zebra.com/us/en/research-insights/warehousing-vision-study.html (term. wiz. 23. 04. 2026).

[48] Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Yuan, Z., Luo, P., Liu, W., Wang, X.: *ByteTrack: Multi-Object Tracking by Associating Every Detection Box.* 2022. arXiv: 2110.06864 [cs.CV]. URL: https://arxiv.org/abs/2110.06864.

[49] Zou, Z., Chen, K., Shi, Z., Guo, Y., Ye, J.: "Object Detection in 20 Years: A Survey". W: *Proceedings of the IEEE* 111.3 (2023), s. 257–276. DOI: 10.1109/JPROC.2023.3238524.